General

General

- Everything is on dl4cv.github.io (and Moodle)
- 4 credit points
- 2 hrs. lecture, ~2 hrs. tutorial (Tutorial covers new material)
- Communication through Moodle
- 3 homework assignments + Final Project
  Homework is demanding, but worth it
- Reserve duty etc.: We will support!

”If we have seen further, it is by standing on the shoulders of Giants"

- From basic to most recent SotA
- Slightly biased towards Weizmann research
- Intuition
- Hands-on
- Openness



We Assume you...

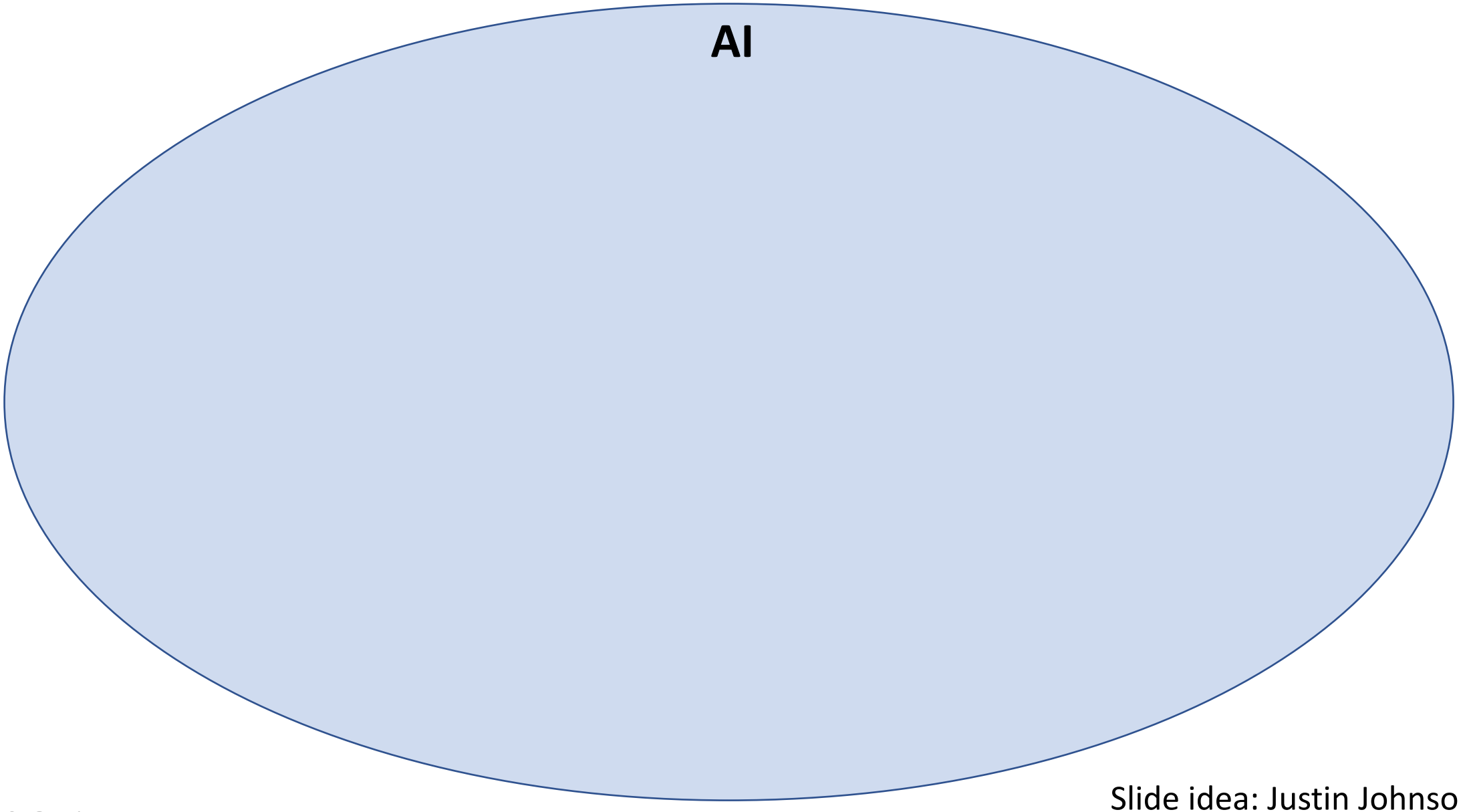
- Know Basic Calculus (e.g. know what is a Gradient).
- Know Basic Algebra (e.g. Vector spaces, Matrix multiplication, Eigen decomposition).
- Written code before (preferably Python).
- Bumped into Machine Learning (e.g. heard the term “Overfitting”).

Homework

Theory | **From Scratch** | **Applied**

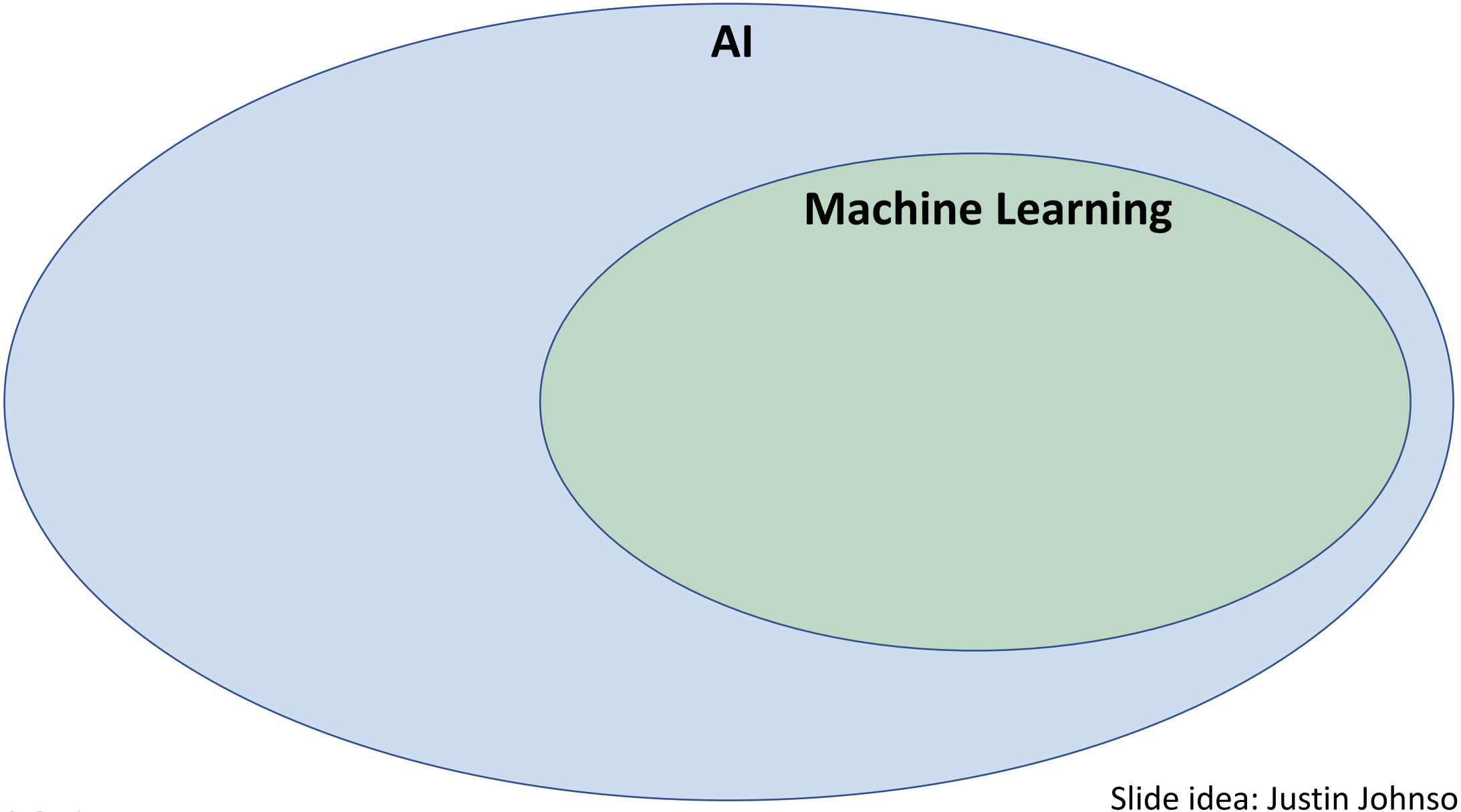
HW1 is online!

Road map



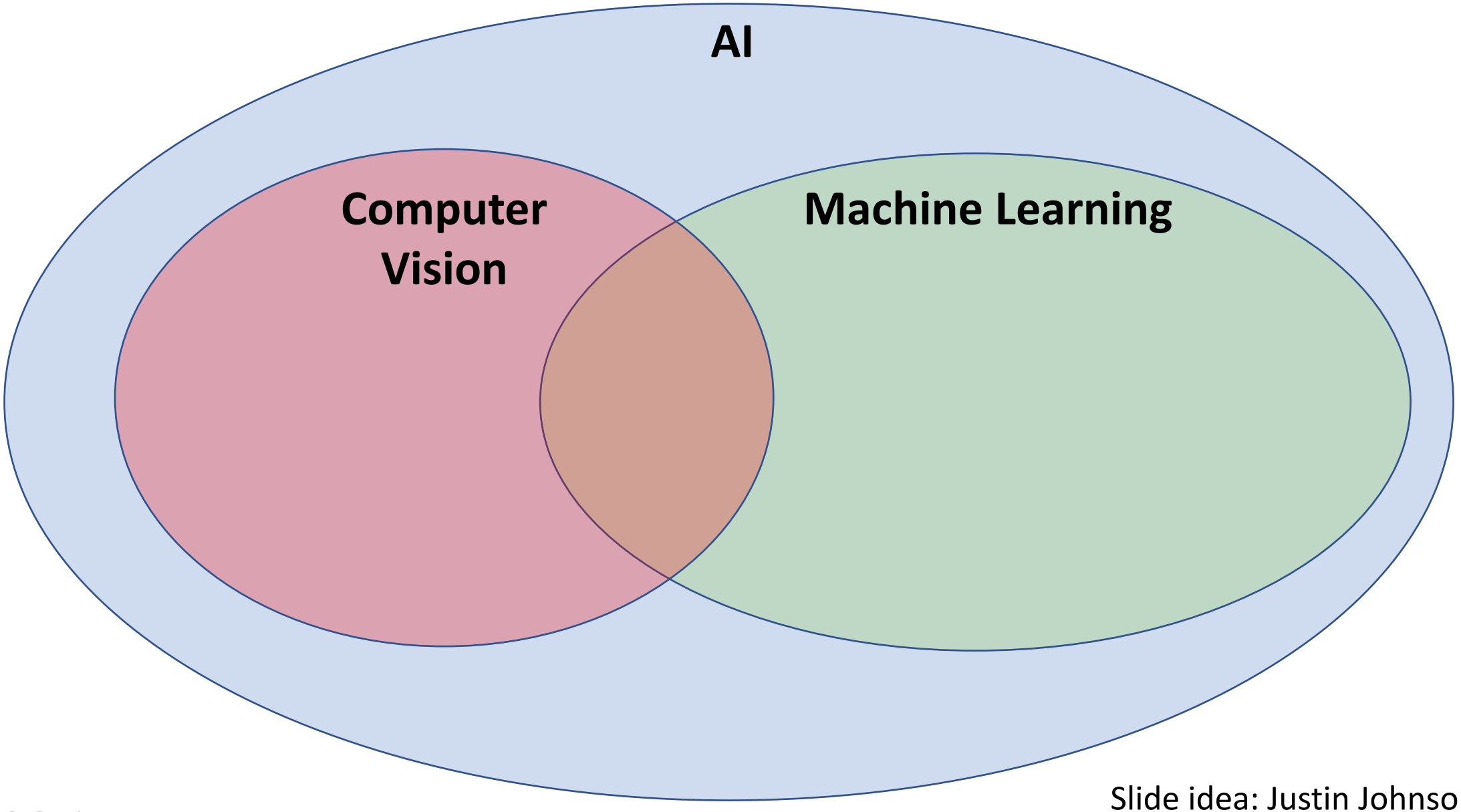
Slide idea: Justin Johnson

Road map



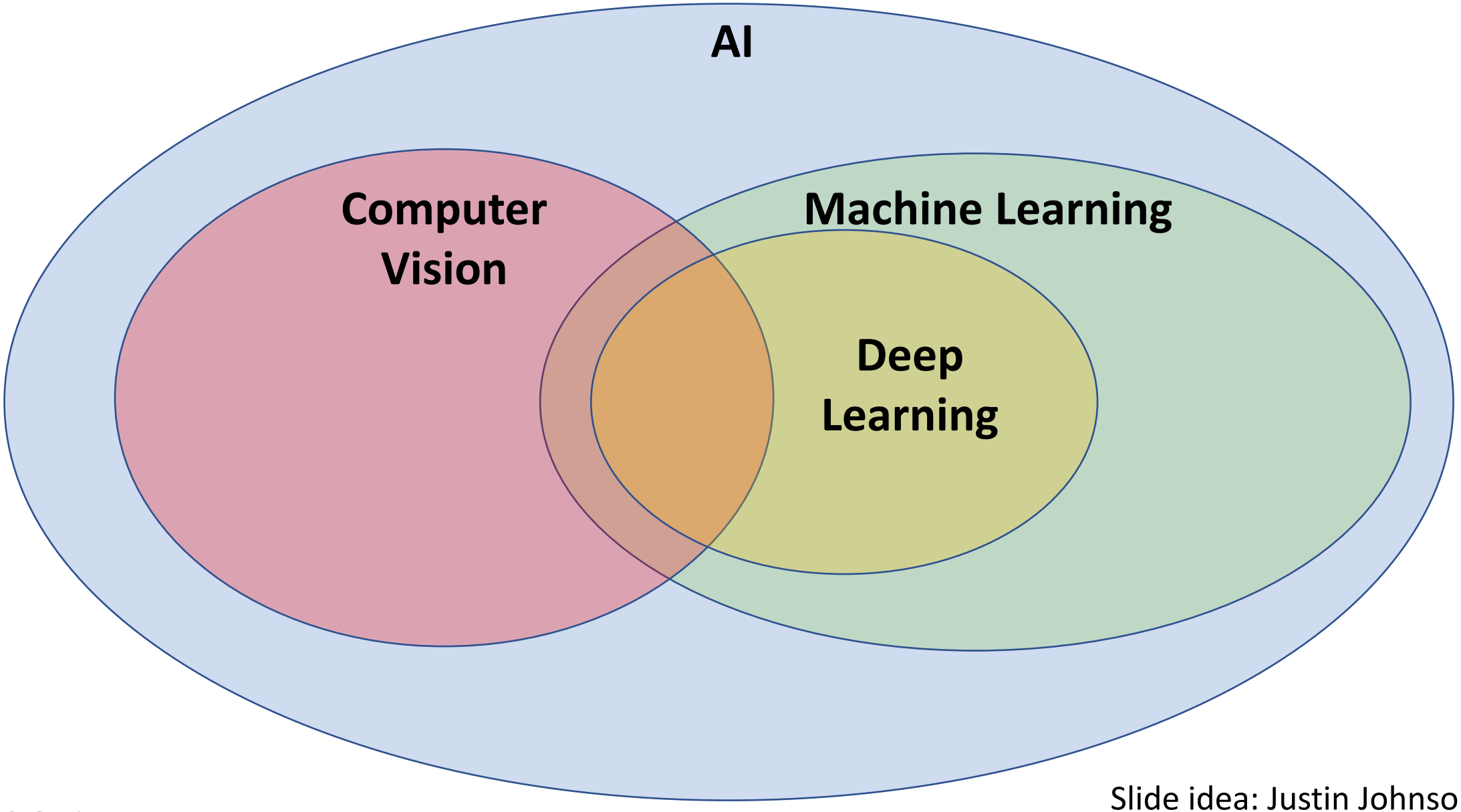
Slide idea: Justin Johnson

Road map



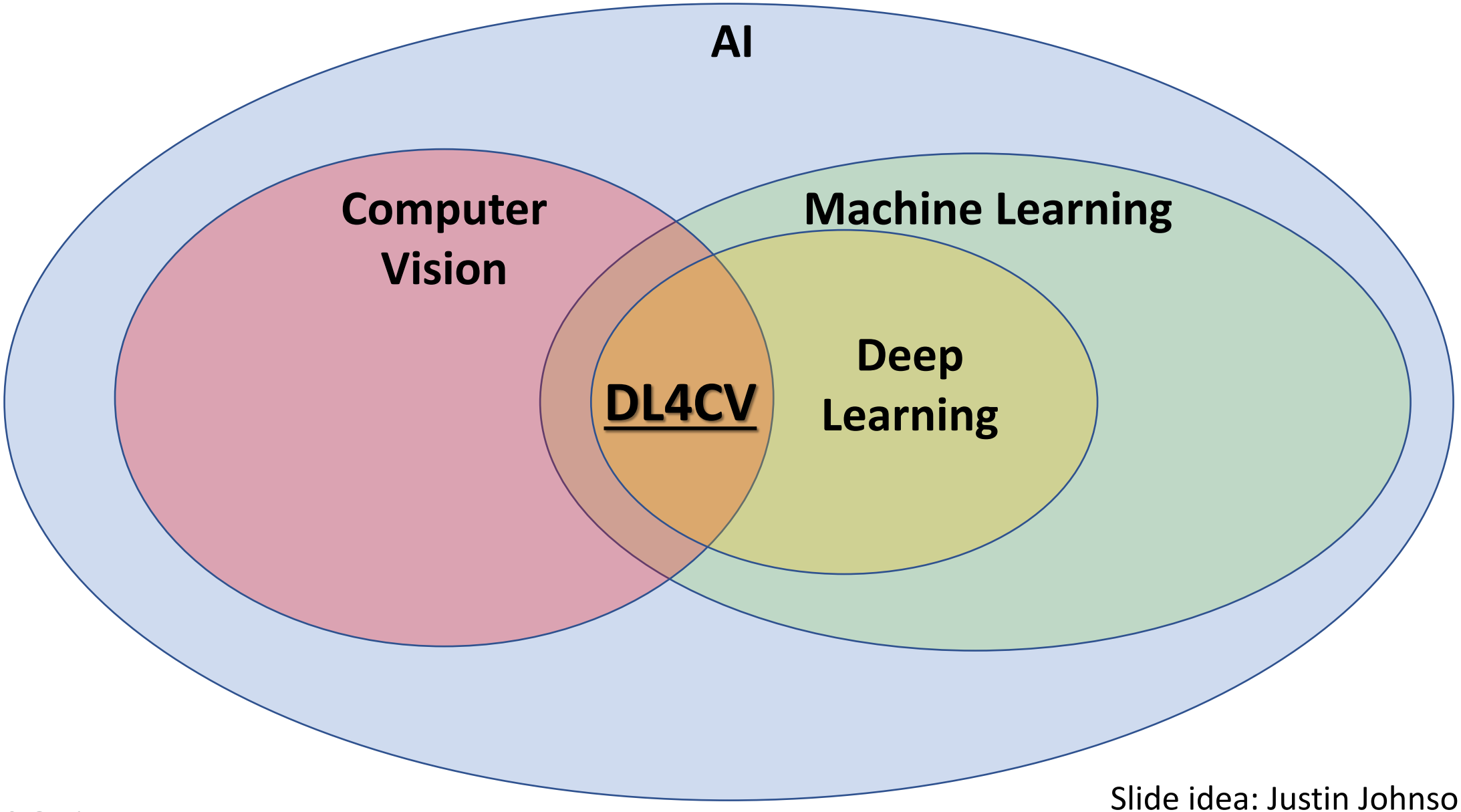
Slide idea: Justin Johnson

Road map



Slide idea: Justin Johnson

Road map



Slide idea: Justin Johnson

Today:

- Motivation and history (15%)
- Supervised learning (25%)
- Linear regression (20%)
- Gradient descent (25%)
- Feature transform (15%)

Deep Learning is powerful



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."



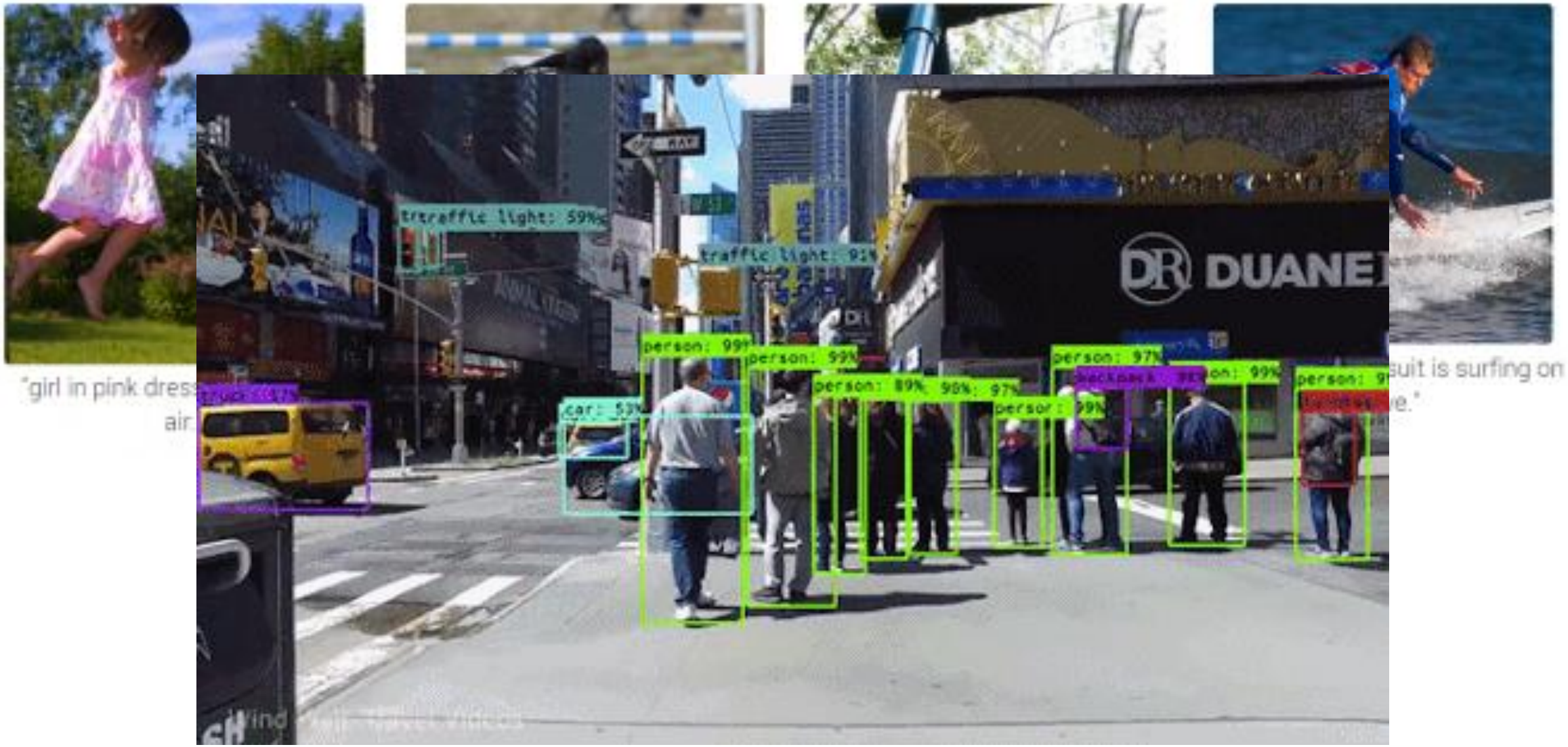
"young girl in pink shirt is swinging on swing."



"man in blue wetsuit is surfing on wave."

Andrej Karpathy, Li Fei-Fei, CVPR 2015 Deep Visual-Semantic Alignments for Generating Image Descriptions

Deep Learning is powerful



Andrej Karpathy, Li Fei-Fei, CVPR 2015 Deep Visual-Semantic Alignments for Generating Image Descriptions

Abhishek Bansal- DetectMe (GitHub)



You

generate the picture depicting the joke: a snail on the back of a turtle saying "woohoo"

You

generate the picture depicting the joke: a snail on the back of a turtle saying "woohoo"

ChatGPT



Here's the image depicting the joke: a snail on the back of a turtle, exclaiming "woohoo!" The scene captures the snail's enthusiasm contrasted with the turtle's calm and steady pace. I hope you find it amusing! Let me know if there's anything else I can assist you with!

You

generate the picture depicting the joke: a snail on the back of a turtle saying "woohoo"

ChatGPT



Here's the image depicting the joke: a snail on the back of a turtle, exclaiming "woohoo!" The scene captures the snail's enthusiasm contrasted with the turtle's calm and steady pace. I hope you find it amusing! Let me know if there's anything else I can assist you with!

You

thats very good, but I want it to look like the snail feels the speed. maybe some wind on his skin or anything else that indicates that he feels he is going fast

You

generate the picture depicting the joke: a snail on the back of a turtle saying "woohoo"

ChatGPT



Here's the image depicting the joke: a snail on the back of a turtle, exclaiming "woohoo!" The scene captures the snail's enthusiasm contrasted with the turtle's calm and steady pace. I hope you find it amusing! Let me know if there's anything else I can assist you with!

You

thats very good, but I want it to look like the snail feels the speed. maybe some wind on his skin or anything else that indicates that he feels he is going fast

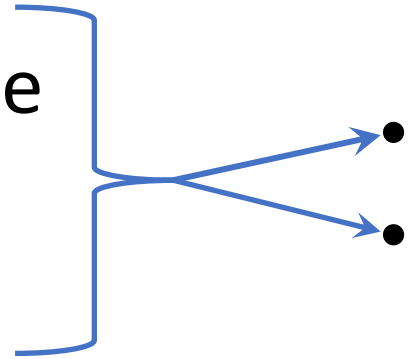
ChatGPT



Here's the revised image where the snail truly feels the thrill of speed! The motion blur and the wind effects emphasize the snail's perceived rapid movement, even though the turtle remains at its usual slow pace. I hope this captures the humor and essence of the joke better. Let me know if there's anything else I can assist you with!

History of:

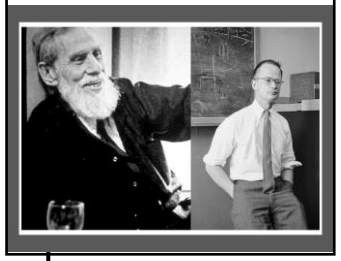
- Computer Science
- Mathematics
- Neuroscience



- Computer vision
- Machine learning



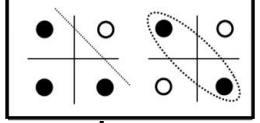
McCulloch Pitts
Non learned
Neuron



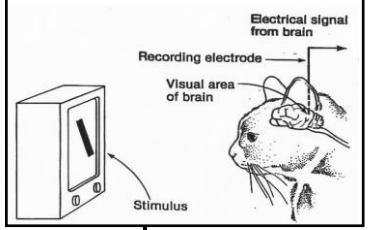
XOR kills
perceptron

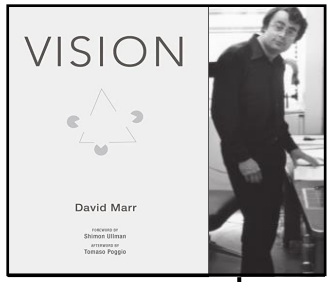


M. Minsky – S. Papert



Hubel & Wiesel





1957

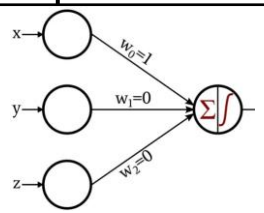
1970

1943

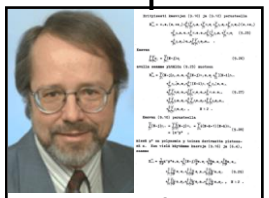
1959

1969

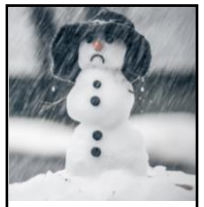
1979



Perceptron
(Rosenblatt)
learned weights

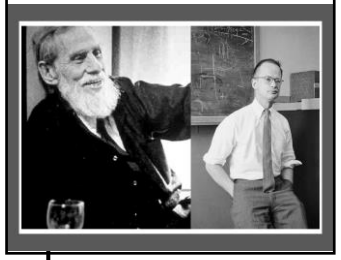


Back
Propagation
(Linnainmaa)



1st
AI Winter

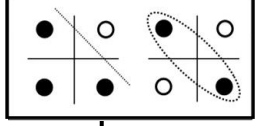
McCulloch Pitts
Non learned
Neuron



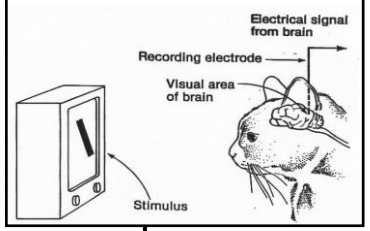
XOR kills
perceptron



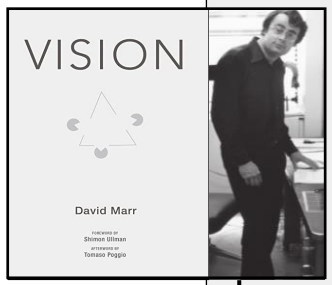
M. Minsky – S. Papert



Hubel & Wiesel



VISION



1957

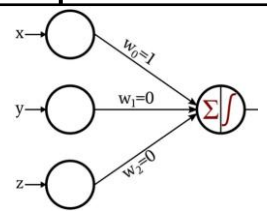
1970

1943

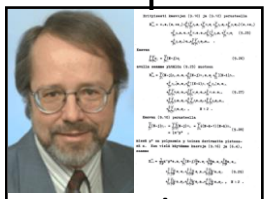
1959

1969

1979



Perceptron
(Rosenblatt)
learned weights



Back
Propagation
(Linnainmaa)



1st
AI Winte

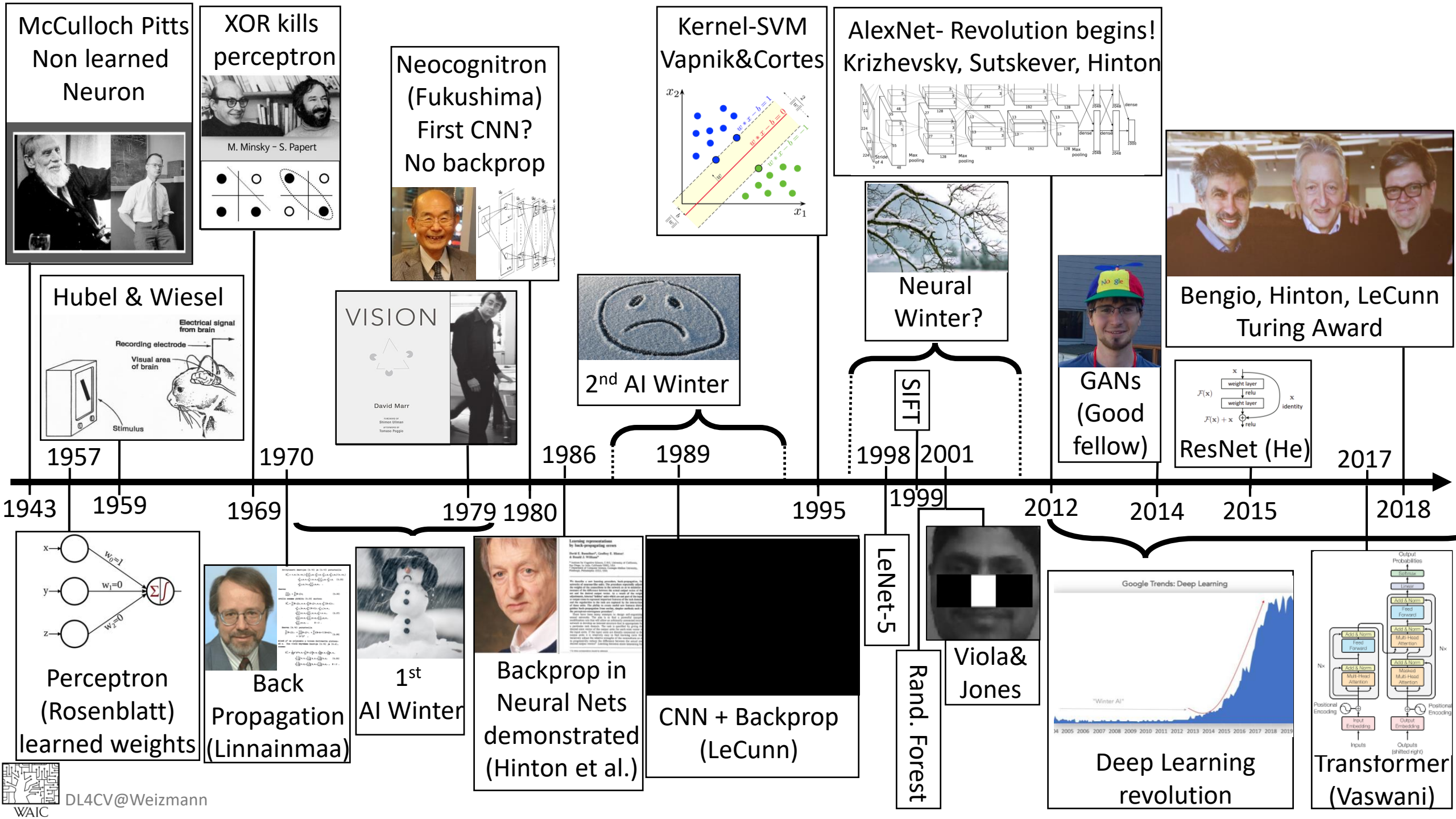


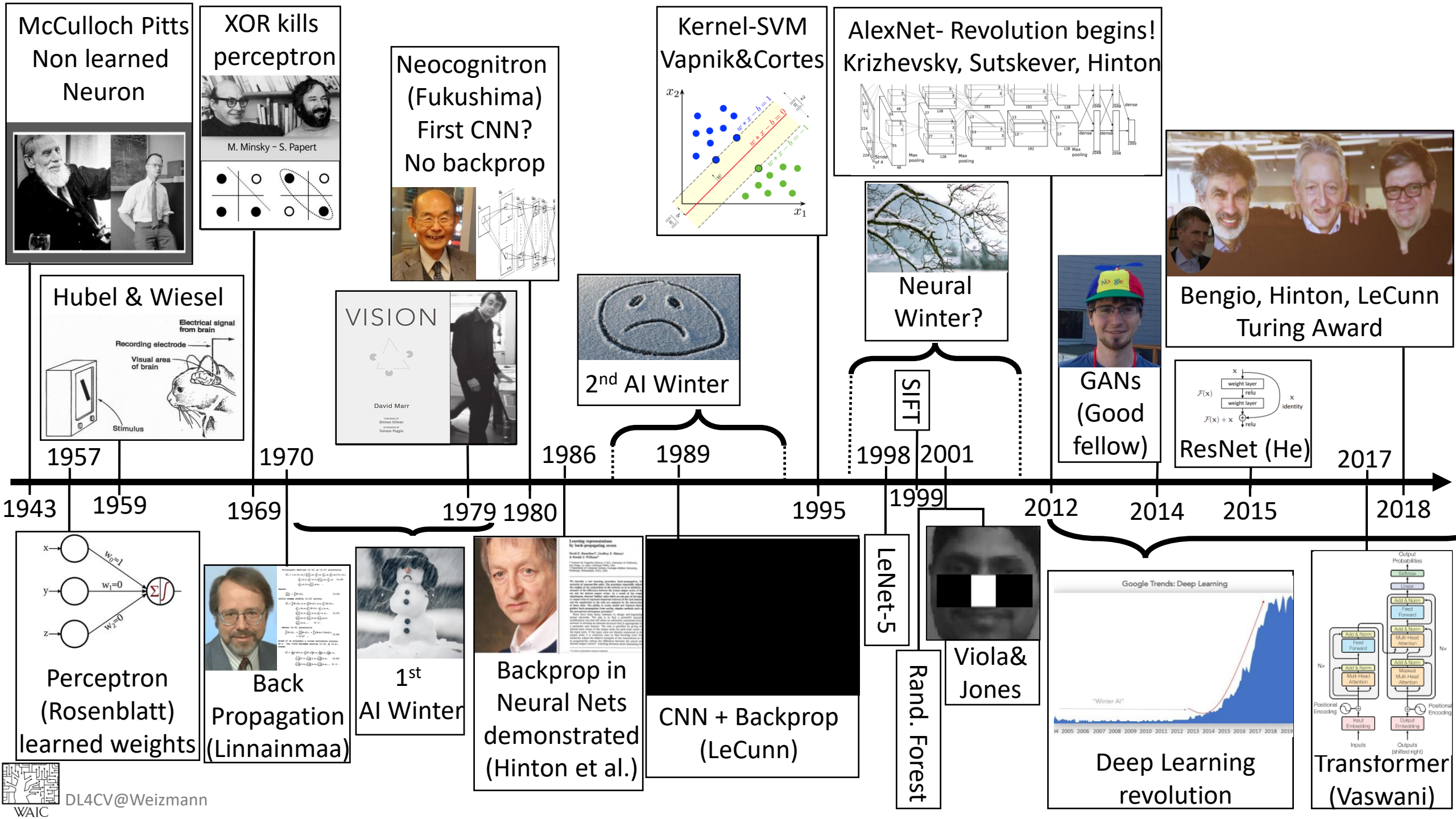
David Marr

FOREWORD BY
Shimon Ullman

AFTERWORD BY
Tomaso Poggio




McCulloch Pitts
Non learned
Neuron
XOR kills
perceptron
M. Minsky – S. Papert
Neocognitron
(Fukushima)
First CNN?
No backprop
Kernel-SVM
Vapnik&Cortes
AlexNet- Revolution begins!
Krizhevsky, Sutskever, Hinton
Hubel & Wiesel
Electrical signal from brain
Recording electrode
Visual area of brain
Stimulus
VISION
David Marr
2nd AI Winter
Neural
Winter?
Bengio, Hinton, LeCunn
Turing Award
GANs
(Good
fellow)
ResNet (He)
SIFT
1957
1970
1986
1989
1998
2001
2017
1943
1959
1969
1979
1980
1995
1999
2012
2014
2015
2018
Perceptron
(Rosenblatt)
learned weights
Back
Propagation
(Linnainmaa)
1st
AI Winter
Backprop in
Neural Nets
demonstrated
(Hinton et al.)
CNN + Backprop
(LeCunn)
LeNet-5
Rand. Forest
Viola&
Jones
Google Trends: Deep Learning
"Winter AI"
Deep Learning
revolution
Transformer
(Vaswani)


McCulloch Pitts
Non learned
Neuron
XOR kills
perceptron
M. Minsky – S. Papert
Neocognitron
(Fukushima)
First CNN?
No backprop
Kernel-SVM
Vapnik&Cortes
AlexNet- Revolution begins!
Krizhevsky, Sutskever, Hinton
Hubel & Wiesel
Electrical signal from brain
Recording electrode
Visual area of brain
Stimulus
VISION
David Marr
2nd AI Winter
Neural
Winter?
SIFT
Bengio, Hinton, LeCunn
Turing Award
GANs
(Good
fellow)
ResNet (He)
1957
1970
1986
1989
1998
2001
2017
1943
1959
1969
1979
1980
1995
1999
2012
2014
2015
2018
Perceptron
(Rosenblatt)
learned weights
Back
Propagation
(Linnainmaa)
1st
AI Winter
Backprop in
Neural Nets
demonstrated
(Hinton et al.)
CNN + Backprop
(LeCunn)
LeNet-5
Rand. Forest
Viola&
Jones
Google Trends: Deep Learning
"Winter AI"
Deep Learning
revolution
Transformer
(Vaswani)

Supervised Learning

Features

Labels

Instances

Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Numeric Nominal Ordinal

Supervised Learning

Features

Labels

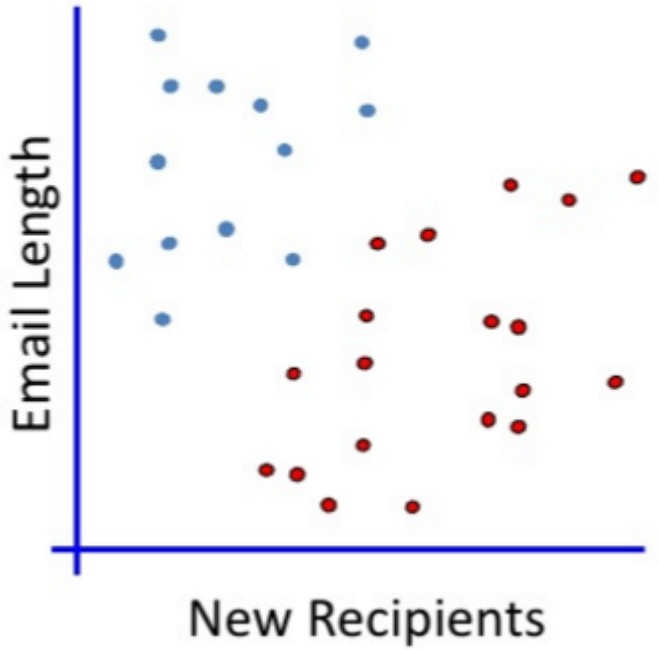
Instances

Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Numeric

Nominal

Ordinal



Supervised Learning

Features

Labels

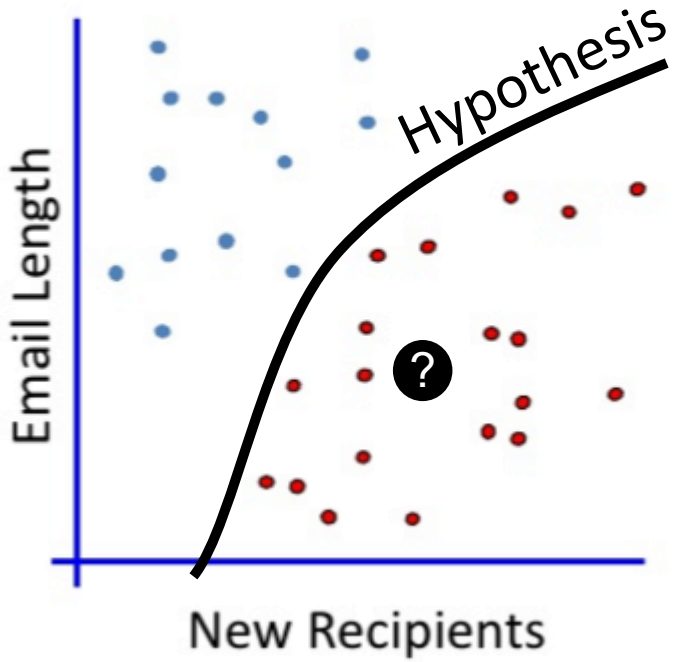
Instances

Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Numeric

Nominal

Ordinal



Supervised Learning

Features

Labels

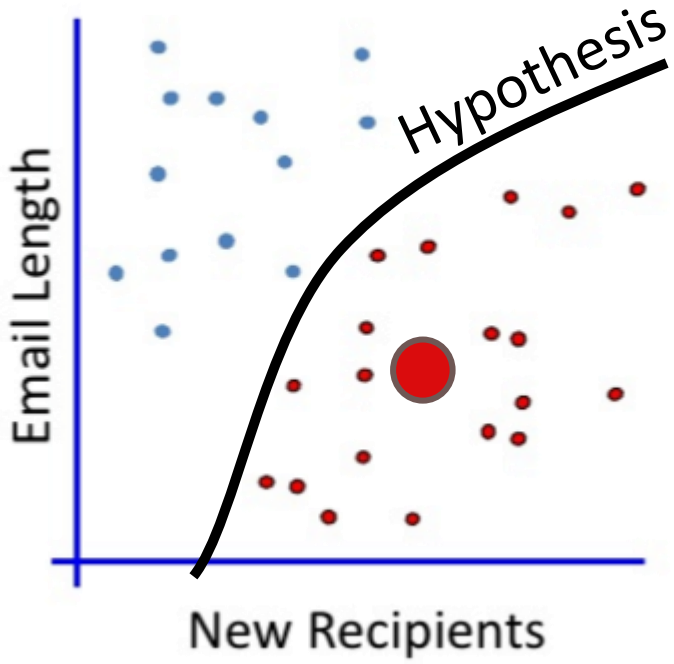
Instances

Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Numeric

Nominal

Ordinal



Supervised Learning

Features

Labels

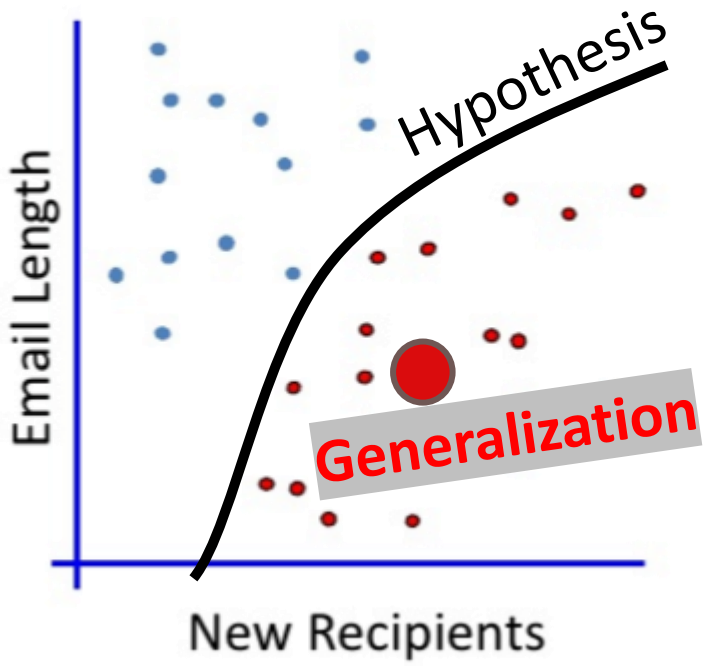
Instances

Number of new Recipients	Email Length (K)	Country (IP)	Customer Type	Email Type
0	2	Germany	Gold	Ham
1	4	Germany	Silver	Ham
5	2	Nigeria	Bronze	Spam
2	4	Russia	Bronze	Spam
3	4	Germany	Bronze	Ham
0	1	USA	Silver	Ham
4	2	USA	Silver	Spam

Numeric

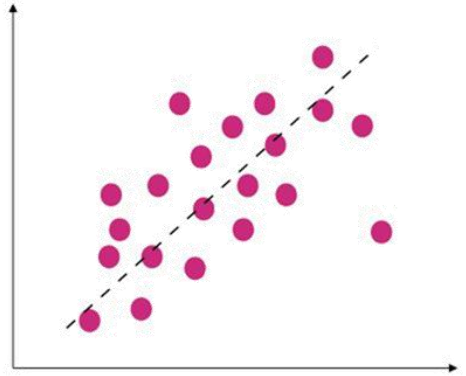
Nominal

Ordinal

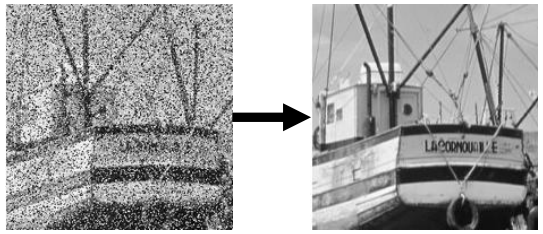


Supervised Learning

Regression



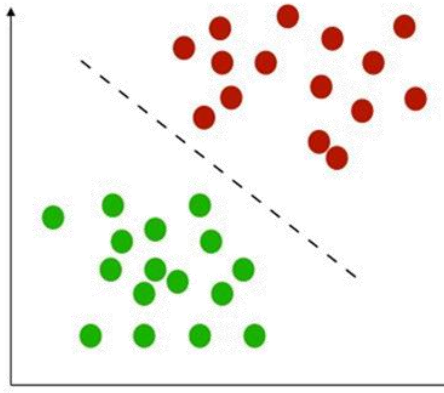
E.g. Image denoising



E.g. Object localization



Classification



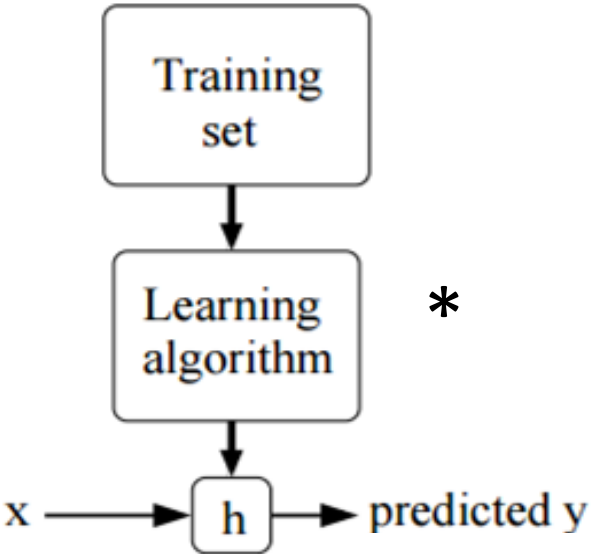
E.g. Image classification





Supervised Learning

$$A(S) = h$$



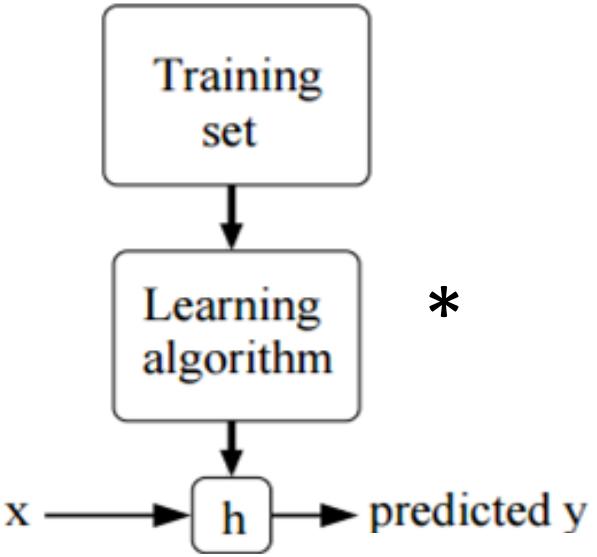
*

* Diagram by Andrew NG

Supervised Learning

Learning Algorithm — Training set — Hypothesis

$$A(S) = h$$



*

Hypothesis class

$\mathcal{H} = \{h_1, h_2 \ldots\}$

Inputs
$\boldsymbol{X}$

Labels
$\boldsymbol{Y}$

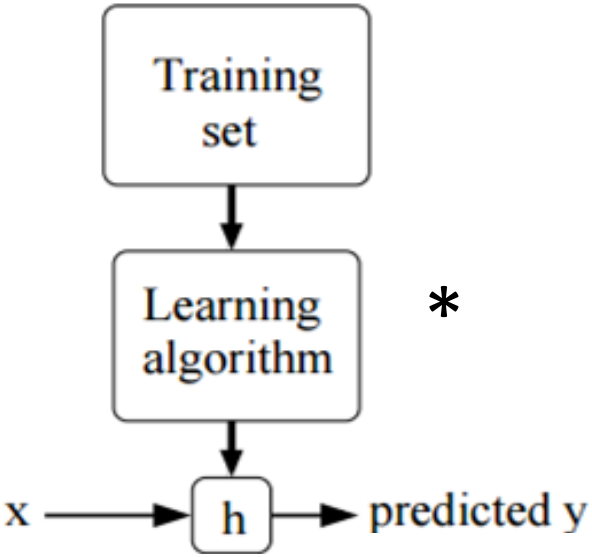
$$\begin{bmatrix} \text{---} \ \mathbf{x_1}^T \ \text{---} \\ \text{---} \ \mathbf{x_2}^T \ \text{---} \\ \vdots \\ \text{---} \ \mathbf{x_M}^T \ \text{---} \end{bmatrix} , \quad \begin{bmatrix} \text{---} \ \mathbf{y_1}^T \ \text{---} \\ \text{---} \ \mathbf{y_2}^T \ \text{---} \\ \vdots \\ \text{---} \ \mathbf{y_M}^T \ \text{---} \end{bmatrix}$$

* Diagram by Andrew NG

Supervised Learning

Learning Algorithm — Training set — Hypothesis

$$A(S) = h$$



*

Hypothesis class

$\mathcal{H} = \{h_1, h_2 \dots\}$

$h(x) \approx y$

Loss

$\mathcal{L}$

Optimization method

Inputs

$\boldsymbol{X}$

$$\begin{bmatrix} \text{---} & \mathbf{x_1}^T & \text{---} \\ \text{---} & \mathbf{x_2}^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x_M}^T & \text{---} \end{bmatrix}$$

Labels

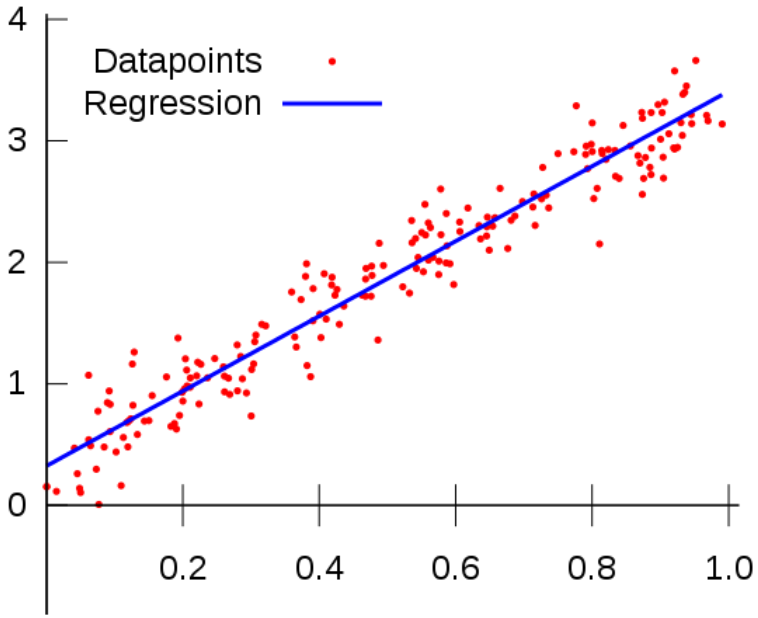
$\boldsymbol{Y}$

$$\begin{bmatrix} \text{---} & \mathbf{y_1}^T & \text{---} \\ \text{---} & \mathbf{y_2}^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{y_M}^T & \text{---} \end{bmatrix}$$

,

* Diagram by Andrew NG

Linear Regression



Linear Regression

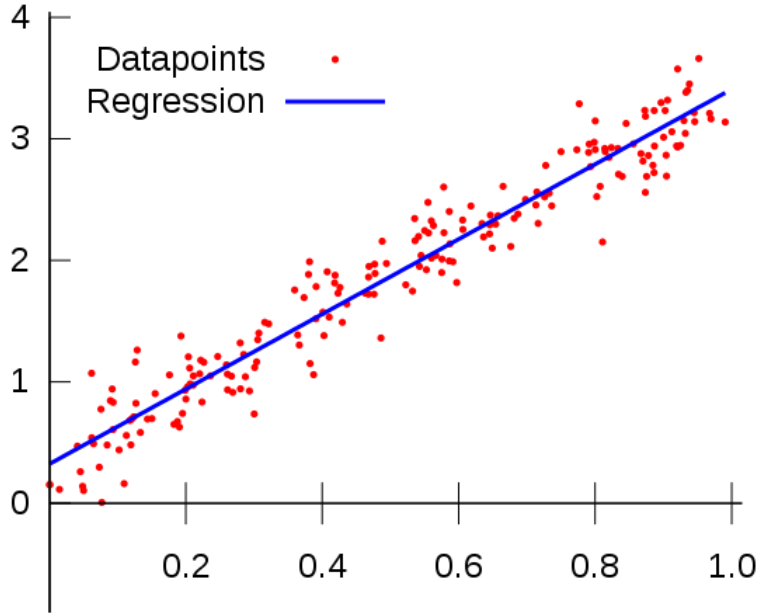
Hypothesis class: Linear →

$$\mathcal{H} = \{h_{\boldsymbol{\theta}} | \, \boldsymbol{\theta} \epsilon \, \mathbb{R}^{N+1}\} \;, \quad h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \theta_0 + \sum_{j=1}^{N} \theta_j x_j = \theta_0 + \tilde{\boldsymbol{\theta}}^T \boldsymbol{x} = \boldsymbol{\theta}^T \begin{pmatrix} 1 \\ | \\ \boldsymbol{x} \\ | \end{pmatrix}$$

*Bias= Just add **1** at top of the input vec!*

Loss: Mean Squared Error →

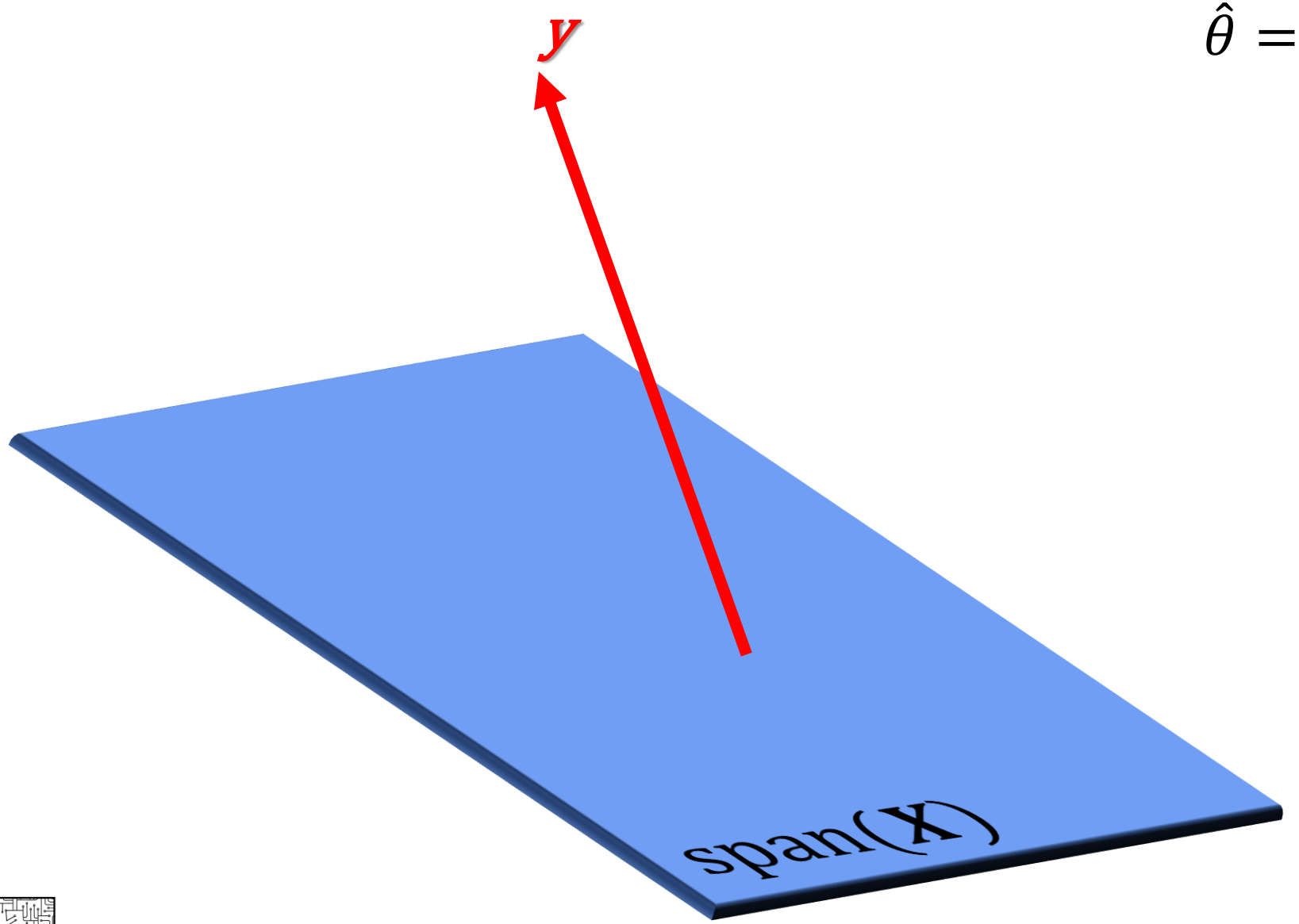
$$\mathcal{L} = \frac{1}{2M} \sum_{i=1}^{M} (h_\theta(\boldsymbol{x}_i) - y_i)^2 = \frac{1}{2M} \|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|^2$$



Optimization method: Normal equations / Gradient Descent

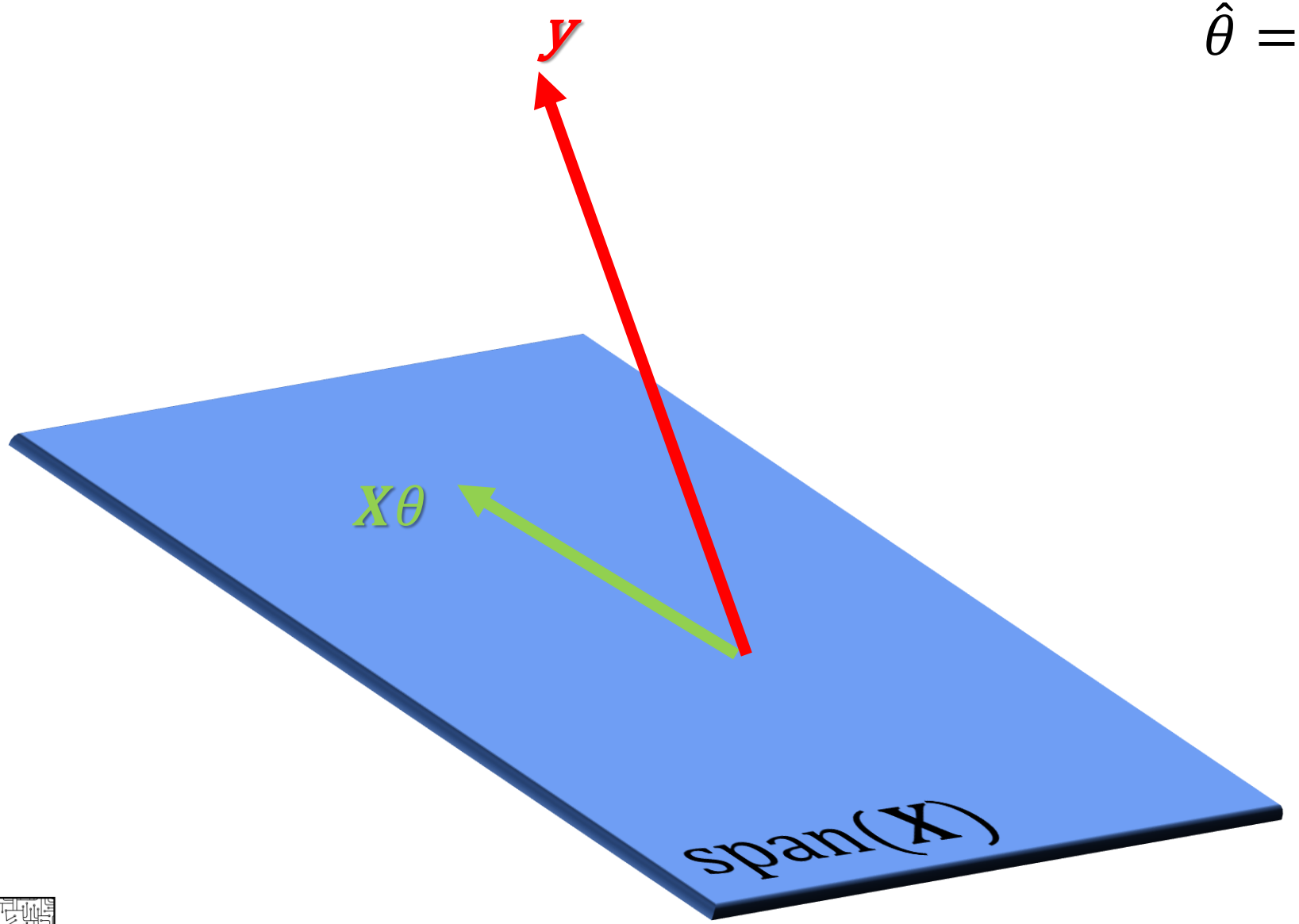
Normal Equations (intuition)

$$\hat{\theta} = argmin_{\theta} \|y - \boldsymbol{X}\theta\|^2$$

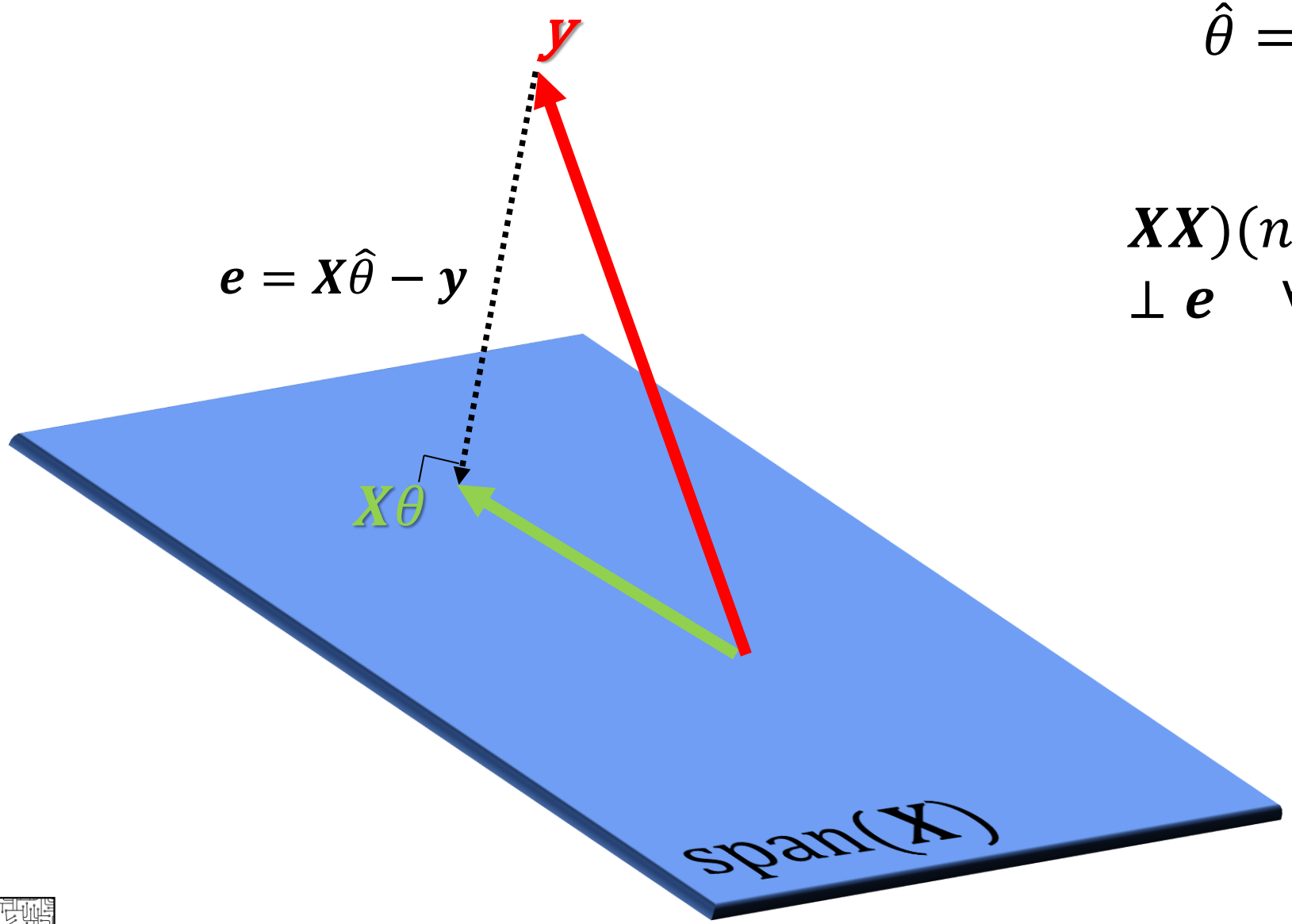


Normal Equations (intuition)

$$\hat{\theta} = argmin_{\theta} \|y - \boldsymbol{X}\theta\|^2$$



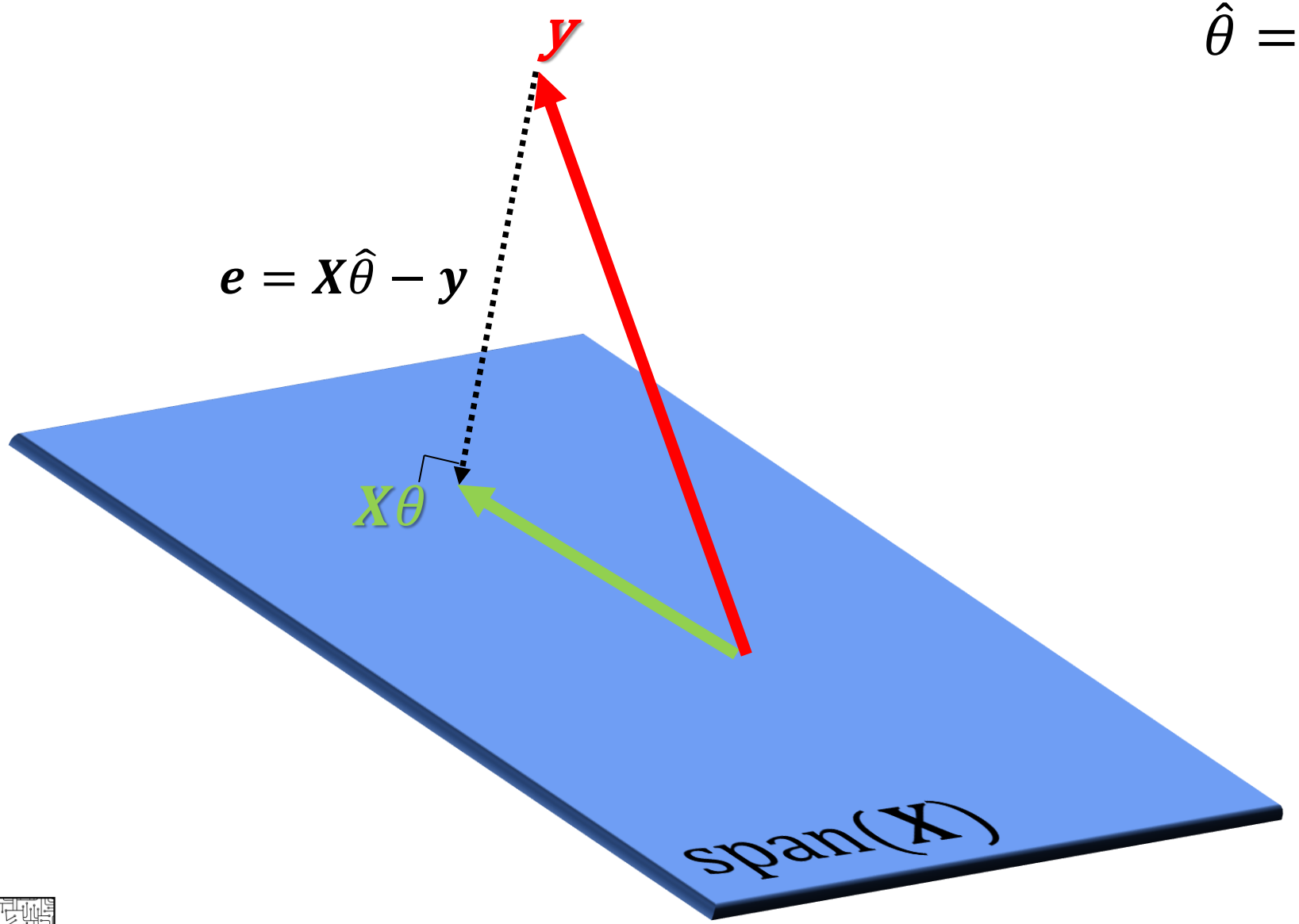
Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$XX)(nnaappssxx \in \forall \quad eex$$
$$\perp e \quad \forall \; x \in span(X)$$

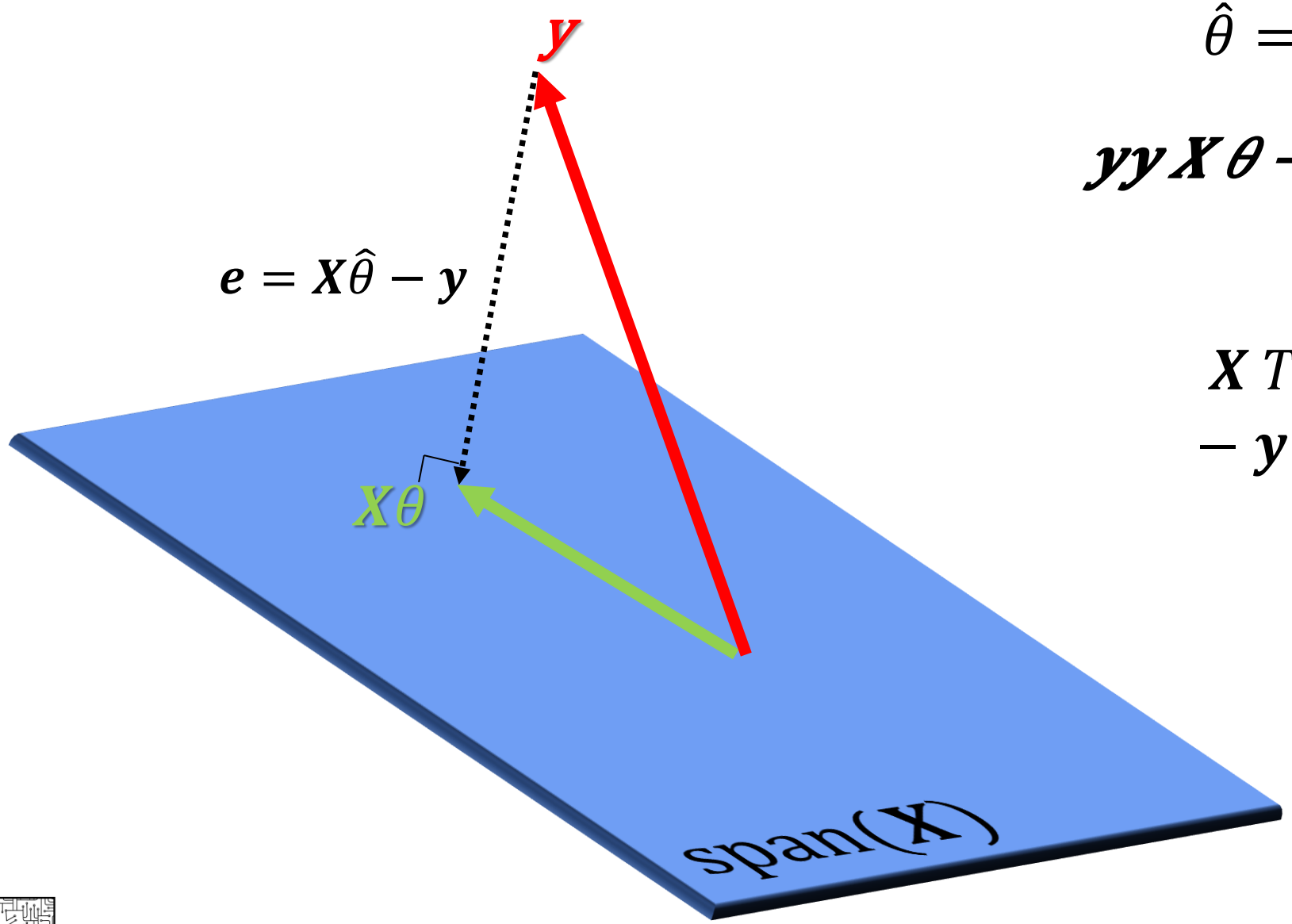
Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$\Downarrow$$

Normal Equations (intuition)

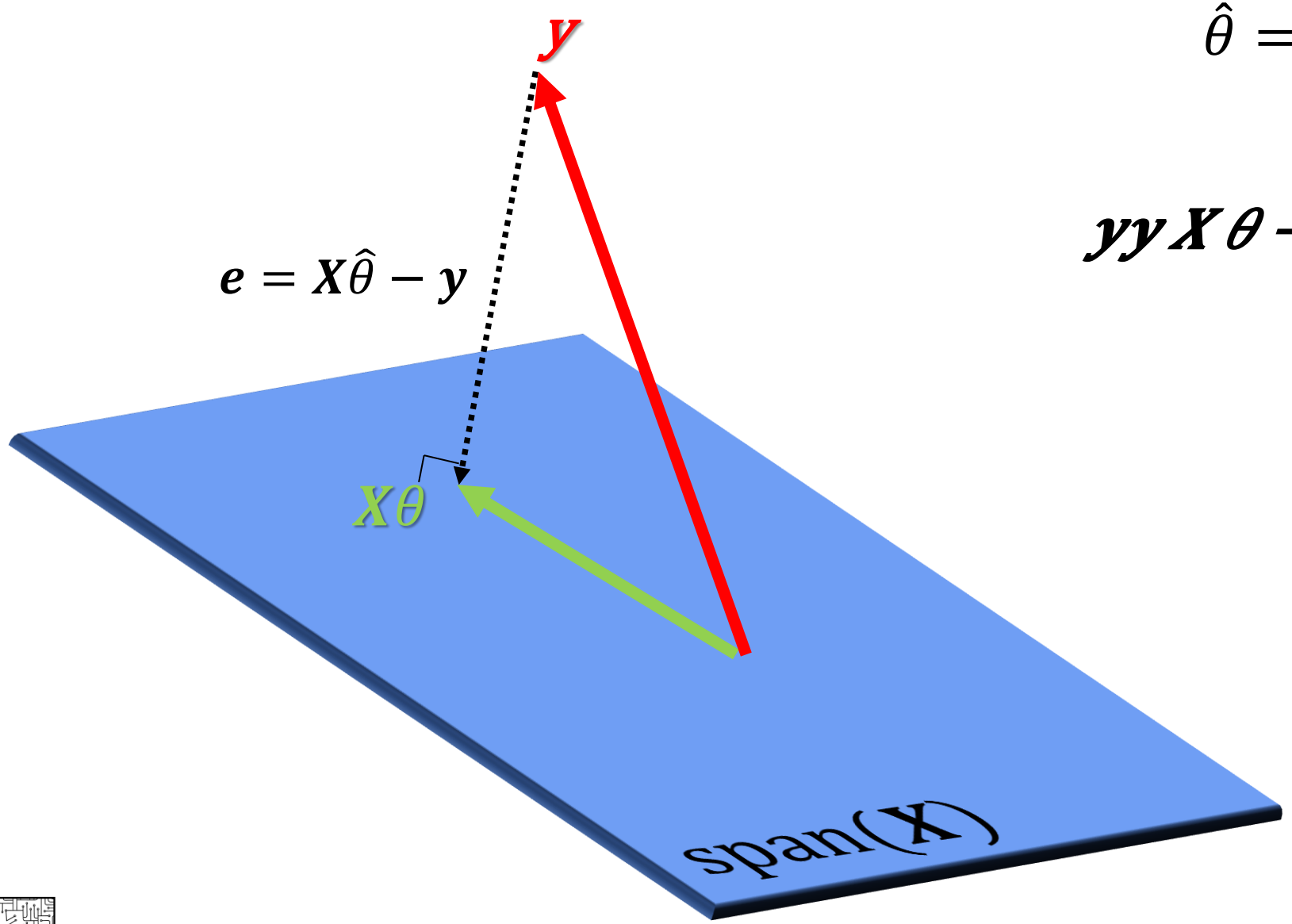


$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$yyX\theta - y = 0$$

$$X\,T\;\; X\,\theta - \;\theta$$
$$-\,y\;\; T\;T\;T\left(X\hat{\theta} - y\right) = 0$$

Normal Equations (intuition)

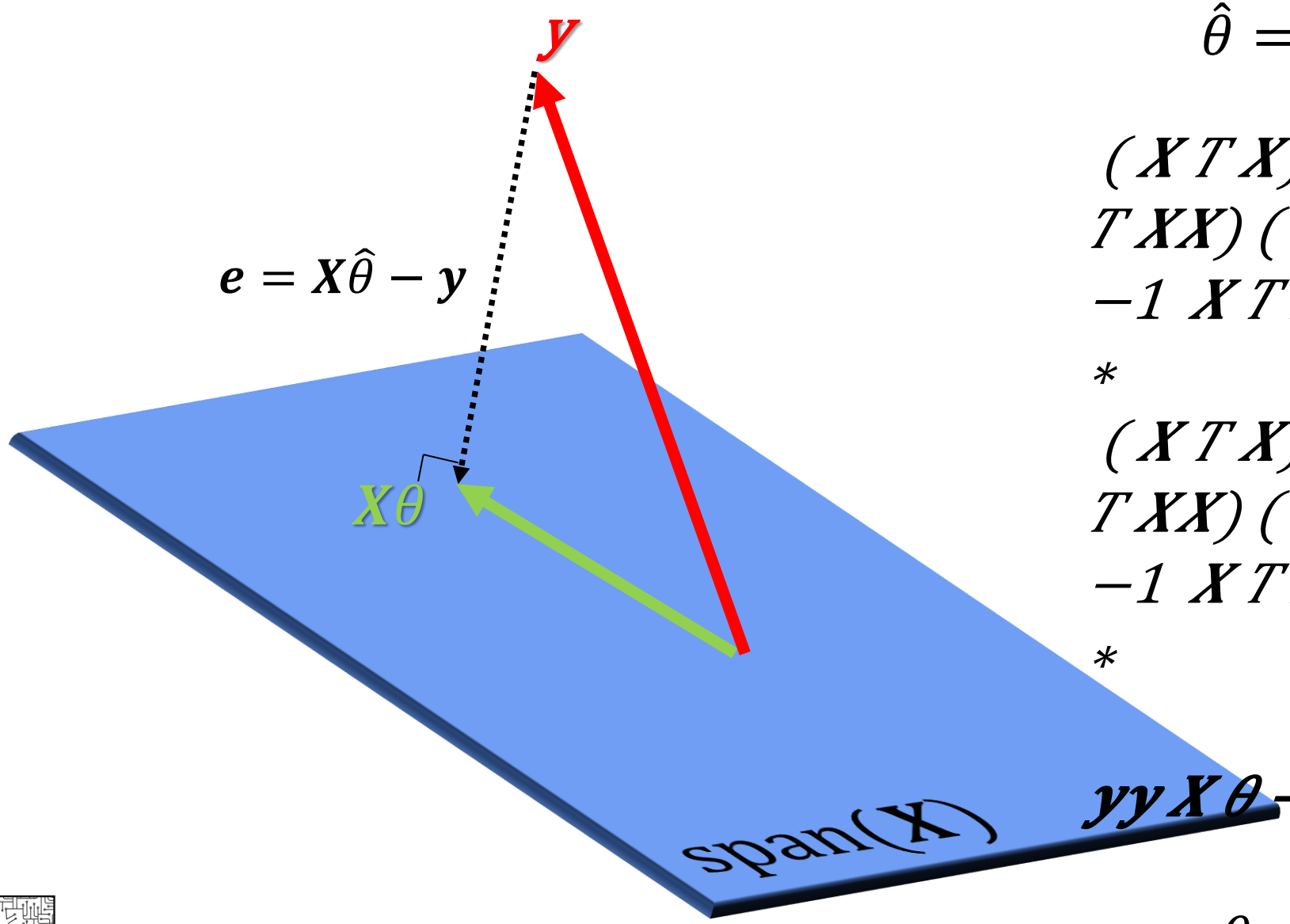


$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$yyX\theta - y = 0$$

⇓

Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$(X^T X)^{-1}(X^T XXX^T TTX^T XX)(X^T X)^{-1-1}(X^T X)^{-1} X^T XXX^T TTX^T yy * * *$$

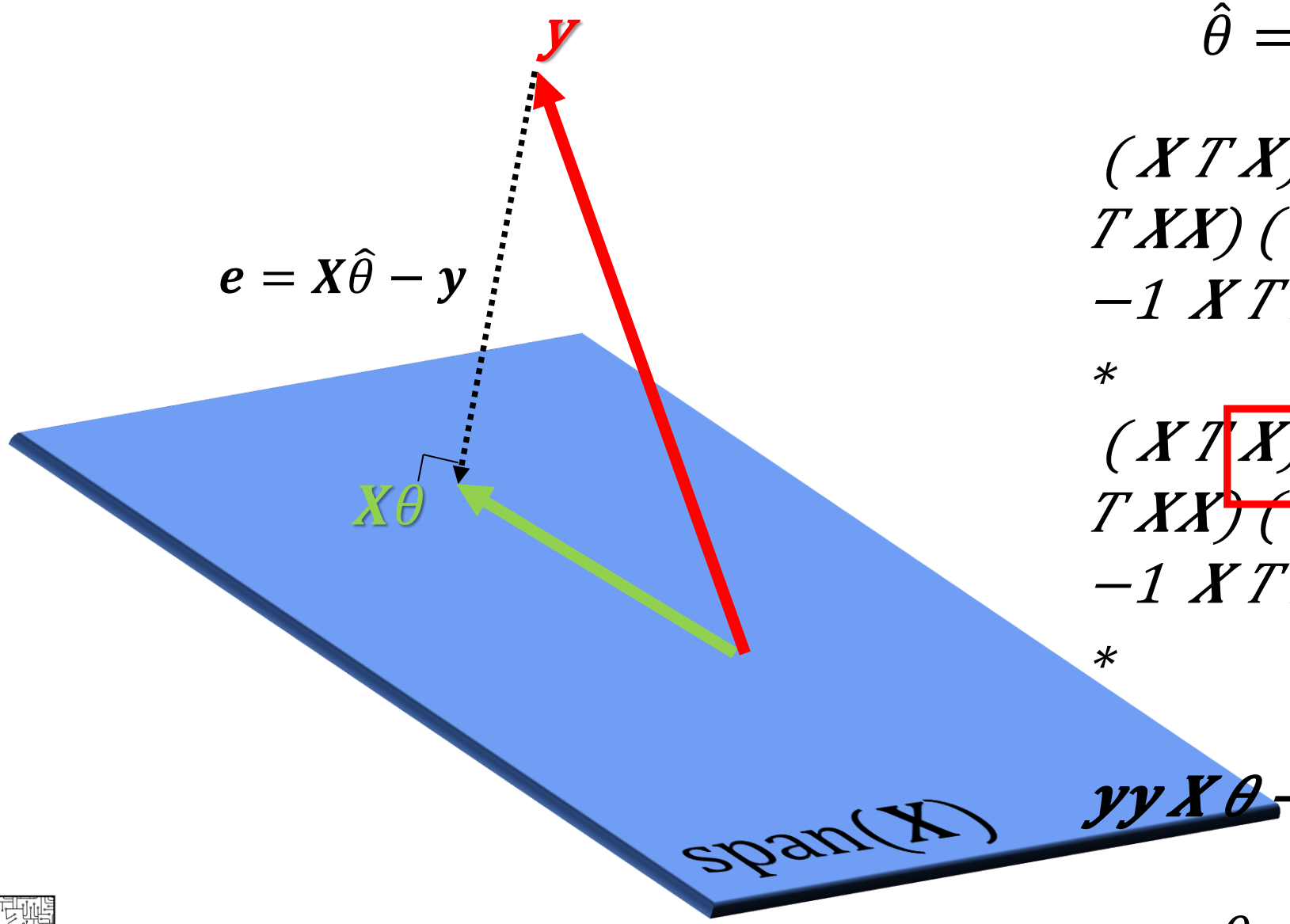
$$(X^T X)^{-1}(X^T XXX^T TTX^T XX)(X^T X)^{-1-1}(X^T X)^{-1} X^T XXX^T TTX^T yy * * *$$

$$yyX\theta - y = 0$$

$$\theta = = (X^T X)^{-1} X^T y$$

invertible

Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

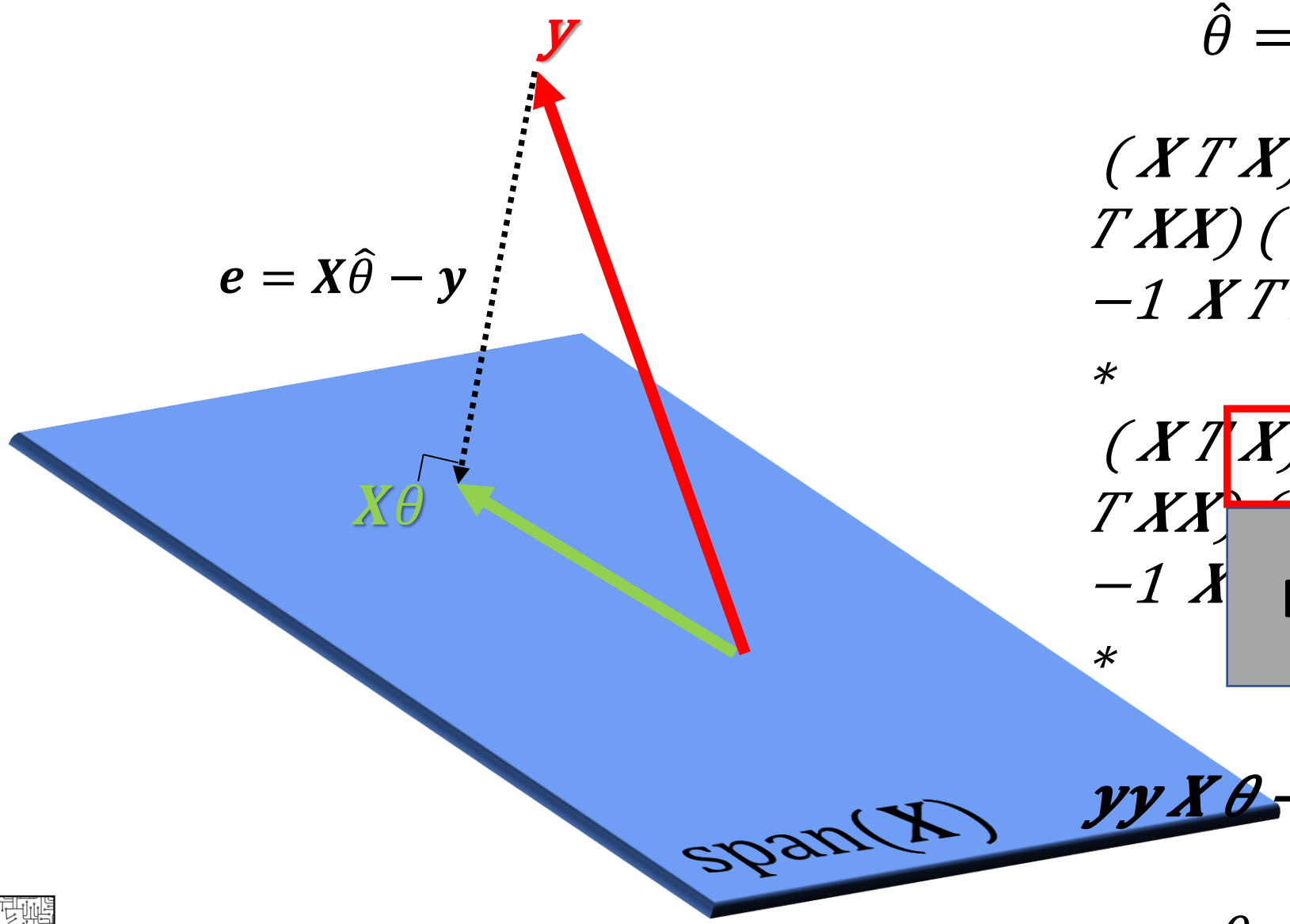
$$(X^T X)^{-1} (X^T X X^T X^T T X^T X X) (X^T X)^{-1-1} (X^T X)^{-1} X^T X X^T X^T T X^T y y * * *$$

$$(X^T X)^{-1} (X^T X X^T X^T T X^T X X) (X^T X)^{-1-1} (X^T X)^{-1} X^T X X^T X^T T X^T y y * * *$$

$$yyX\theta - y = 0$$

$\theta = = (X^T X)^{-1} X^T y$ if $X^T X$ invertible

Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$(X^T X)^{-1} (X^T X X X^T T^T X^T X X) (X^T X)^{-1-1} (X^T X)^{-1} X^T X X X^T T^T X^T y y * * *$$

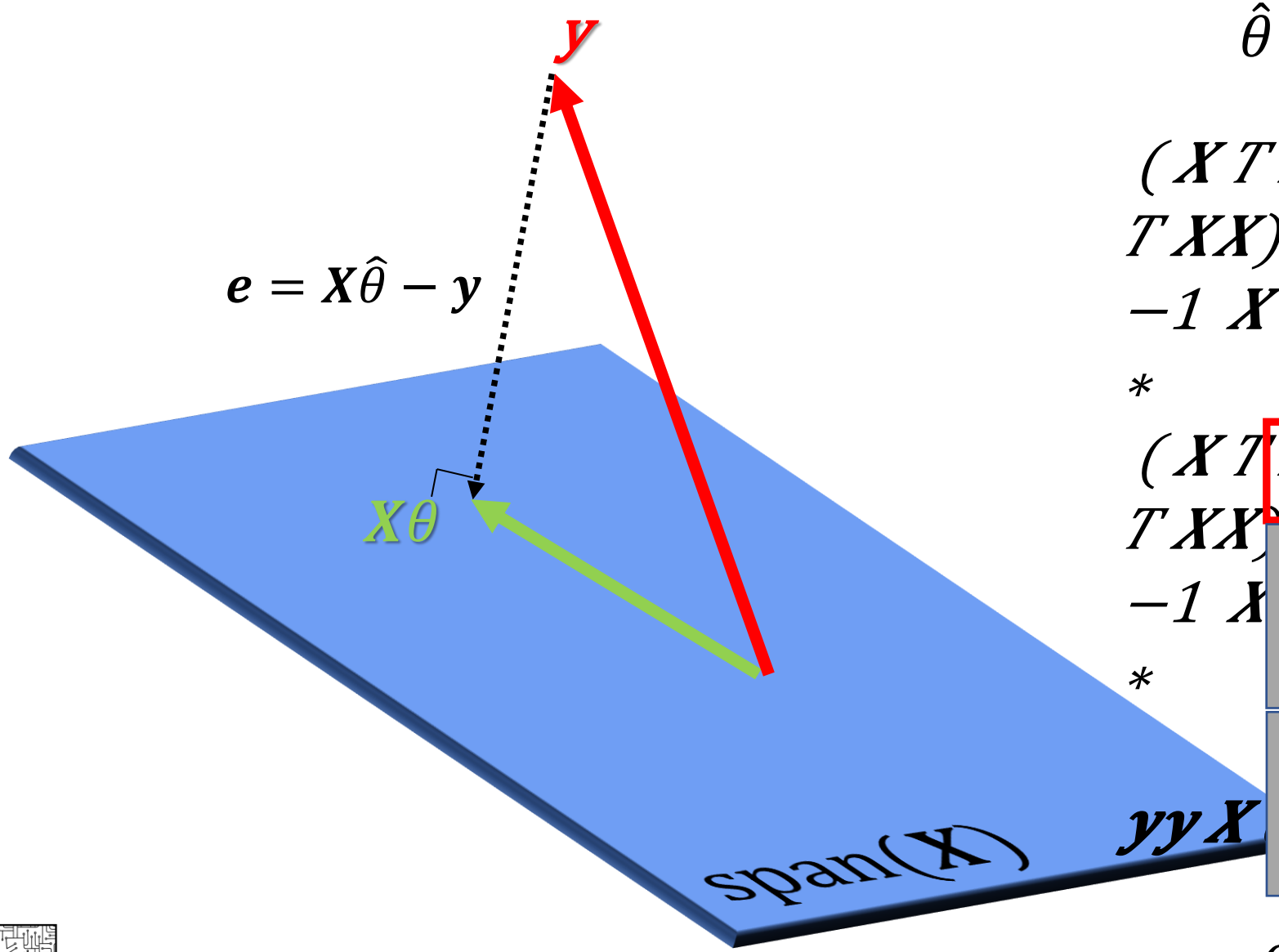
$$(X^T X)^{-1} (X^T X X X^T T^T X^T X X) (X^T X)^{-1-1} (X^T X)^{-1} X^T X X X^T T^T X^T y y * * *$$

Formal proof: HW

$$yyX\theta - y = 0$$

$$\theta = = (X^T X)^{-1} X^T y \quad \text{if } X^T X \text{ invertible}$$

Normal Equations (intuition)



$$\hat{\theta} = argmin_{\theta} \|y - X\theta\|^2$$

$$(XTX)-1(XTXXXTTTX TXX)(XTX)-1-1(XTX)-1XTXXXTTXTyy**$$

$$*(XTX)-1(XTXXXTTTX TXX)(XTX)-1-1(XTX)-1X**$$

$$*$$

Formal proof: HW

Also in HW:
is $X^T X$ invertible?

$$\theta = = (XTfXX^{T}1XTinvertible$$

Normal Equations

Q: Will normal equations always be practical?

$\boldsymbol{X}$ may cost unreasonable memory / time

Normal Equations

Q: Will normal equations always be practical?

A: No;

1. may cost unreasonable memory / time
2. Inverting $\boldsymbol{X}^T\boldsymbol{X}^T\boldsymbol{X}^T\boldsymbol{X}$ may cost unreasonable memory / time



Normal Equations

Q: Will normal equations always be practical?

A: No;

1. may cost unreasonable memory / time
2. Sometimes not applicable: Regularization? Different loss? Non-linearity?



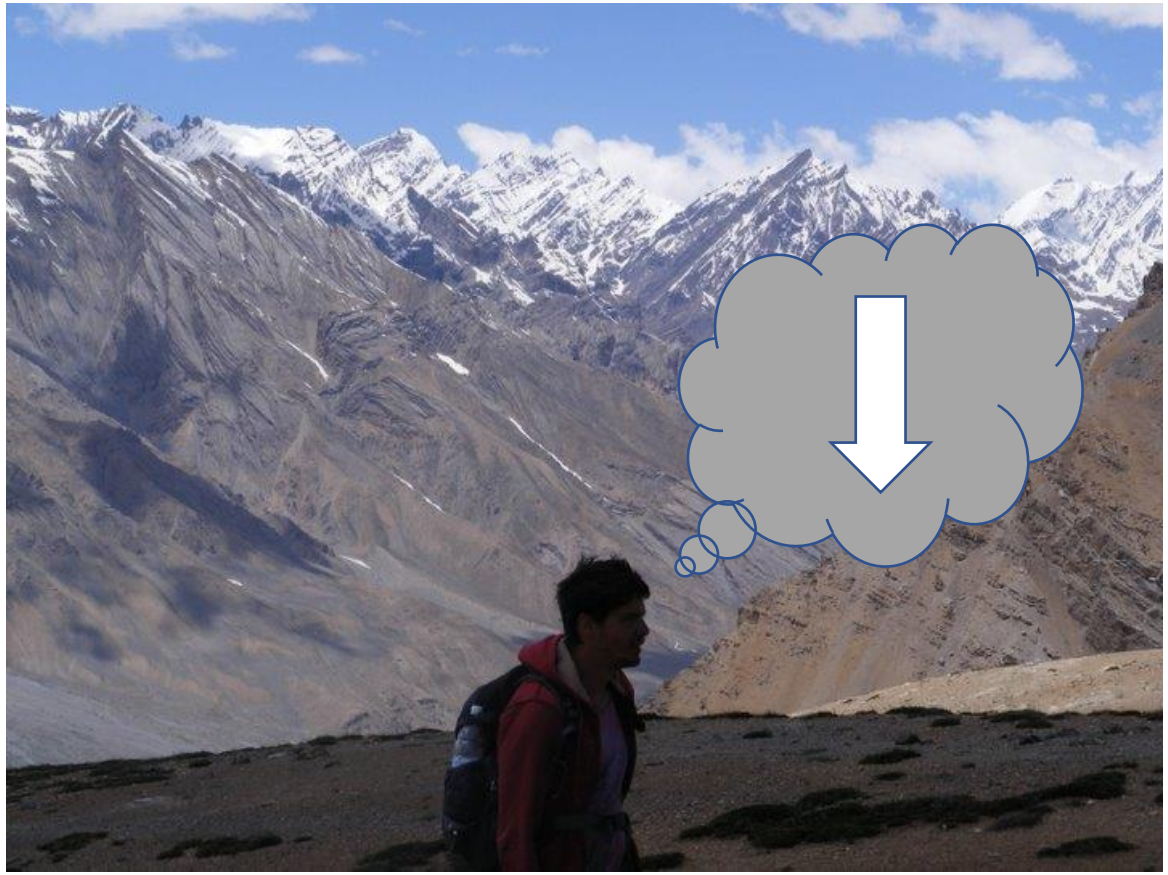
Gradient descent

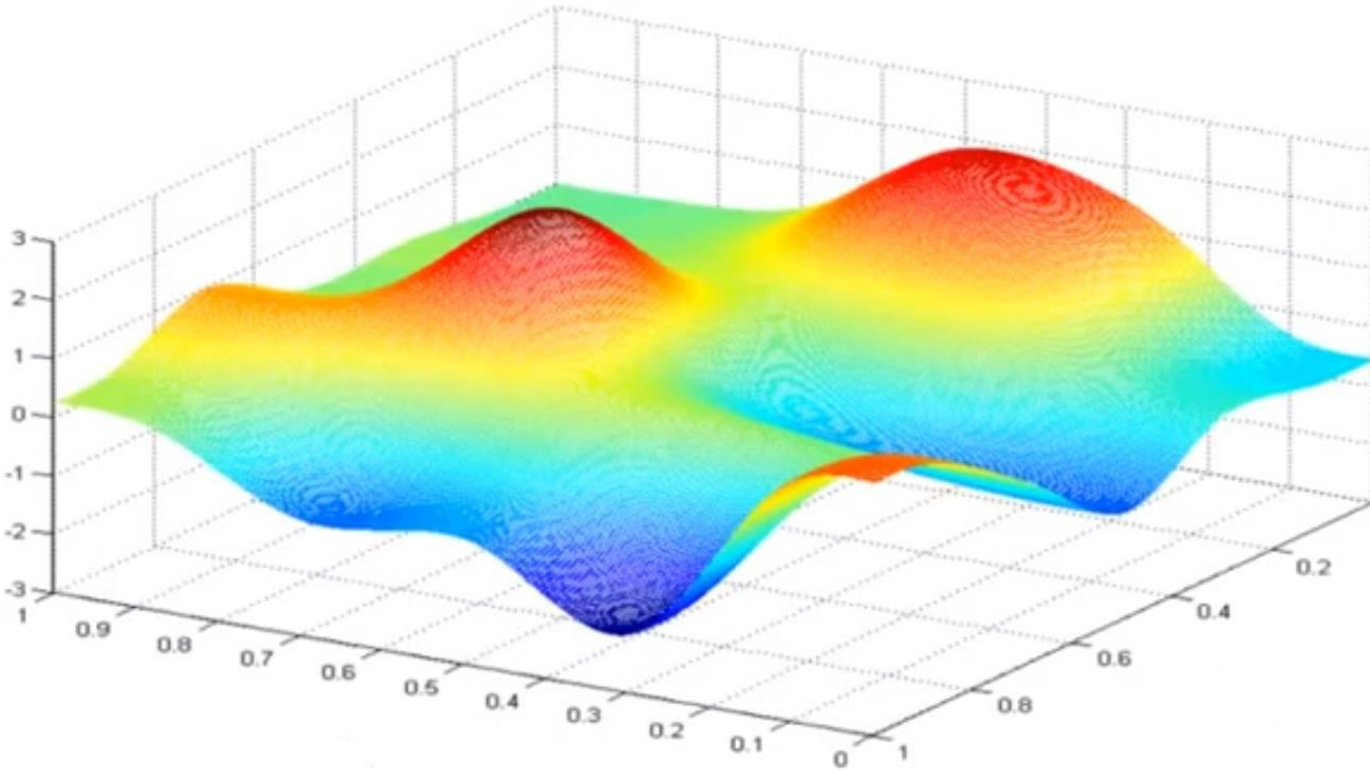
Possible solution: Iteratively reduce loss



Gradient descent

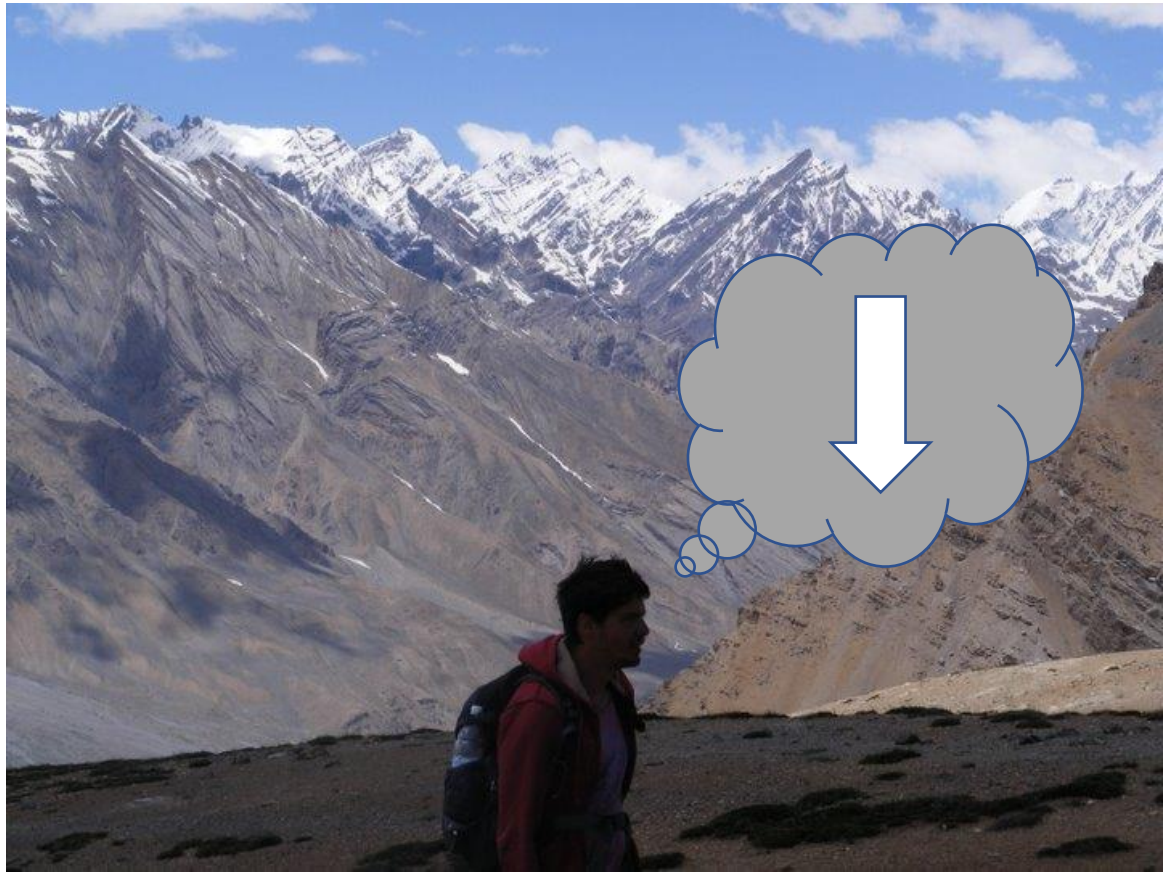
Possible solution: Iteratively reduce loss

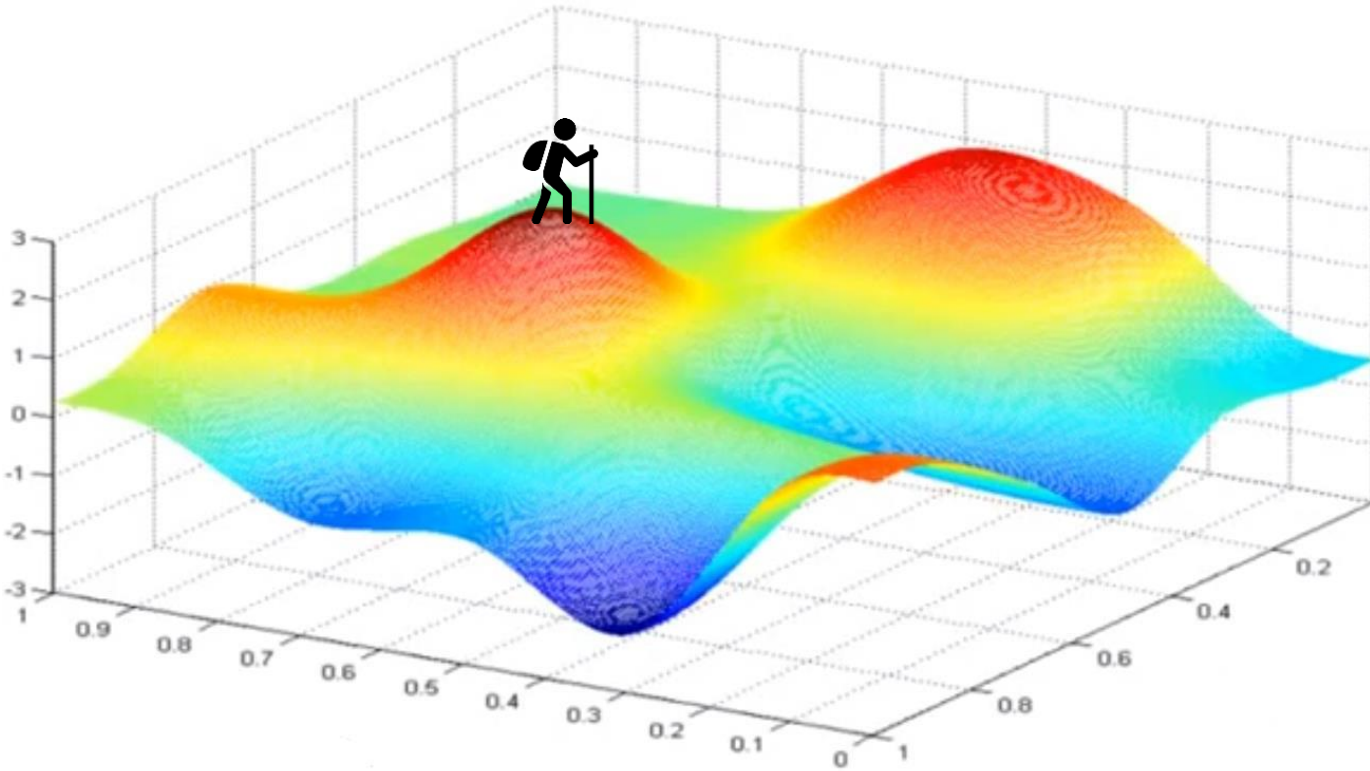




Gradient descent

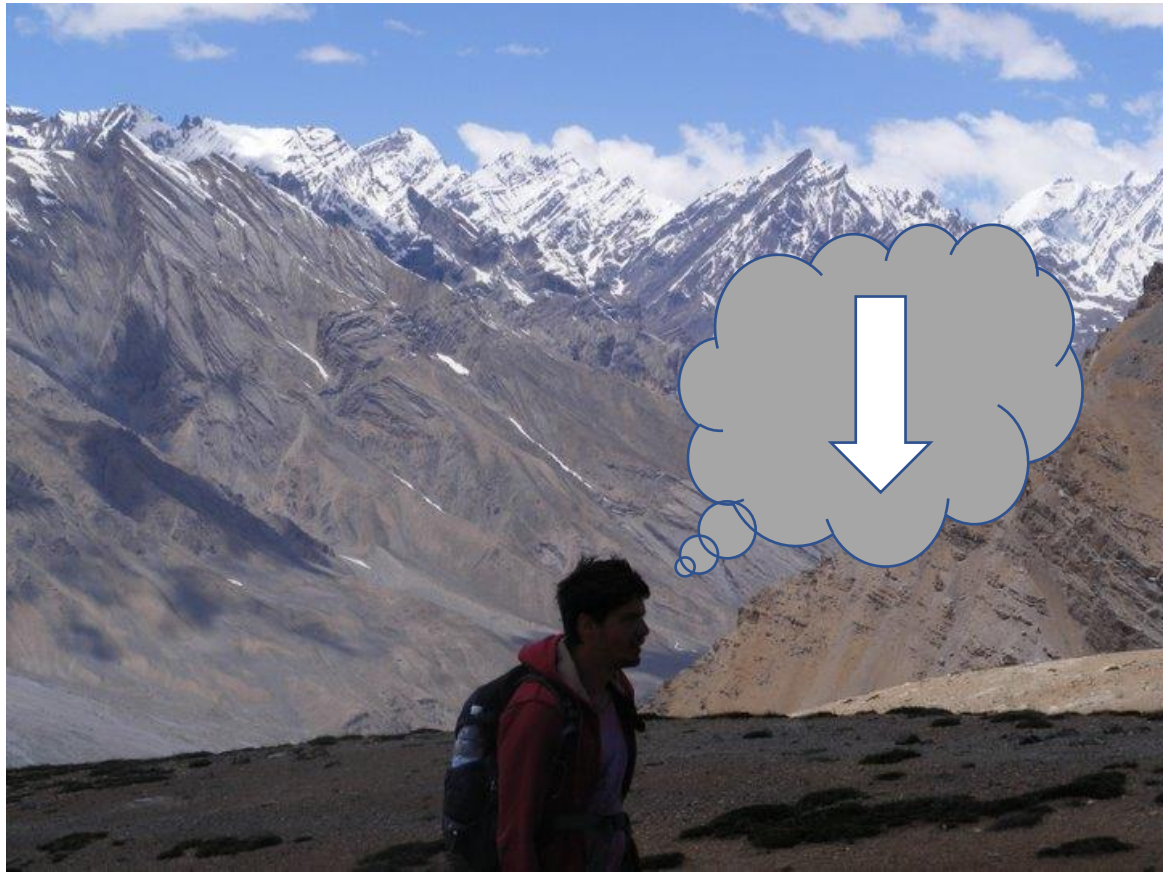
Possible solution: Iteratively reduce loss

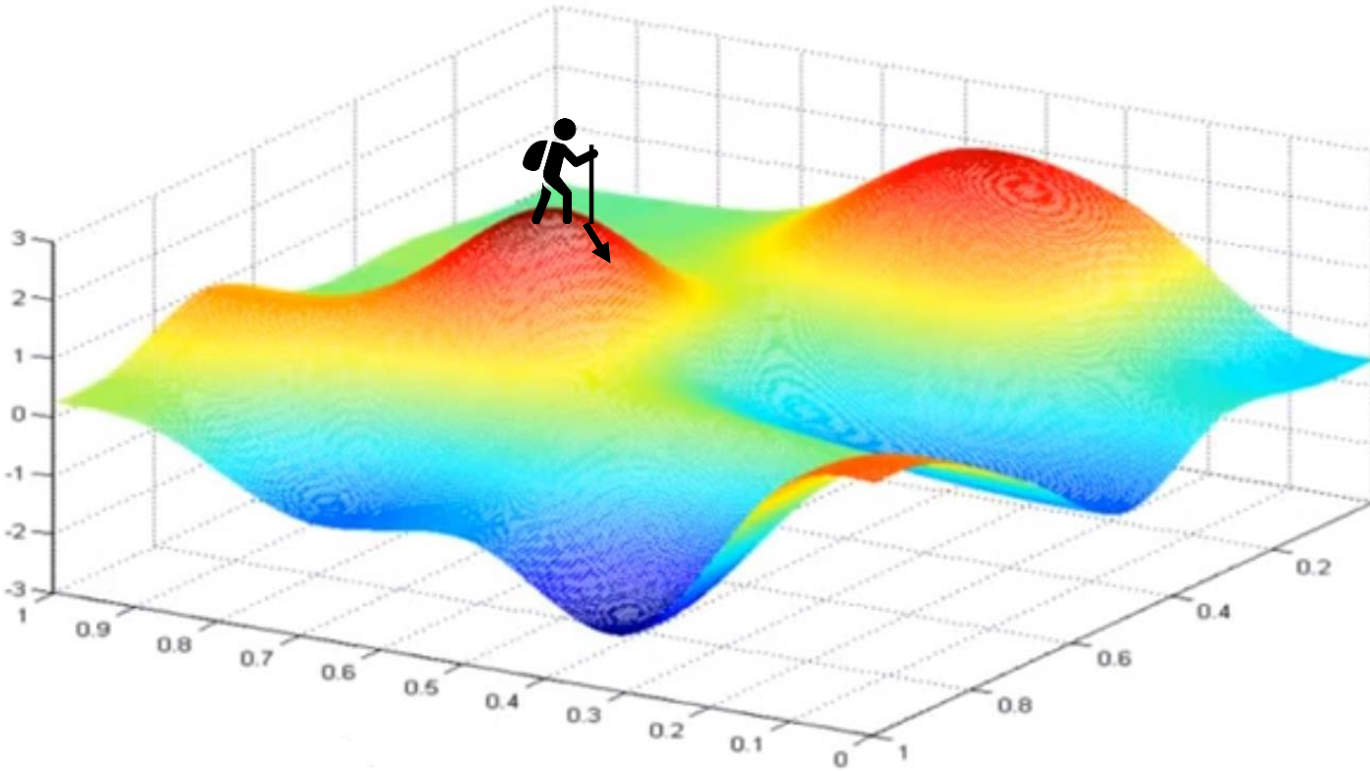




Gradient descent

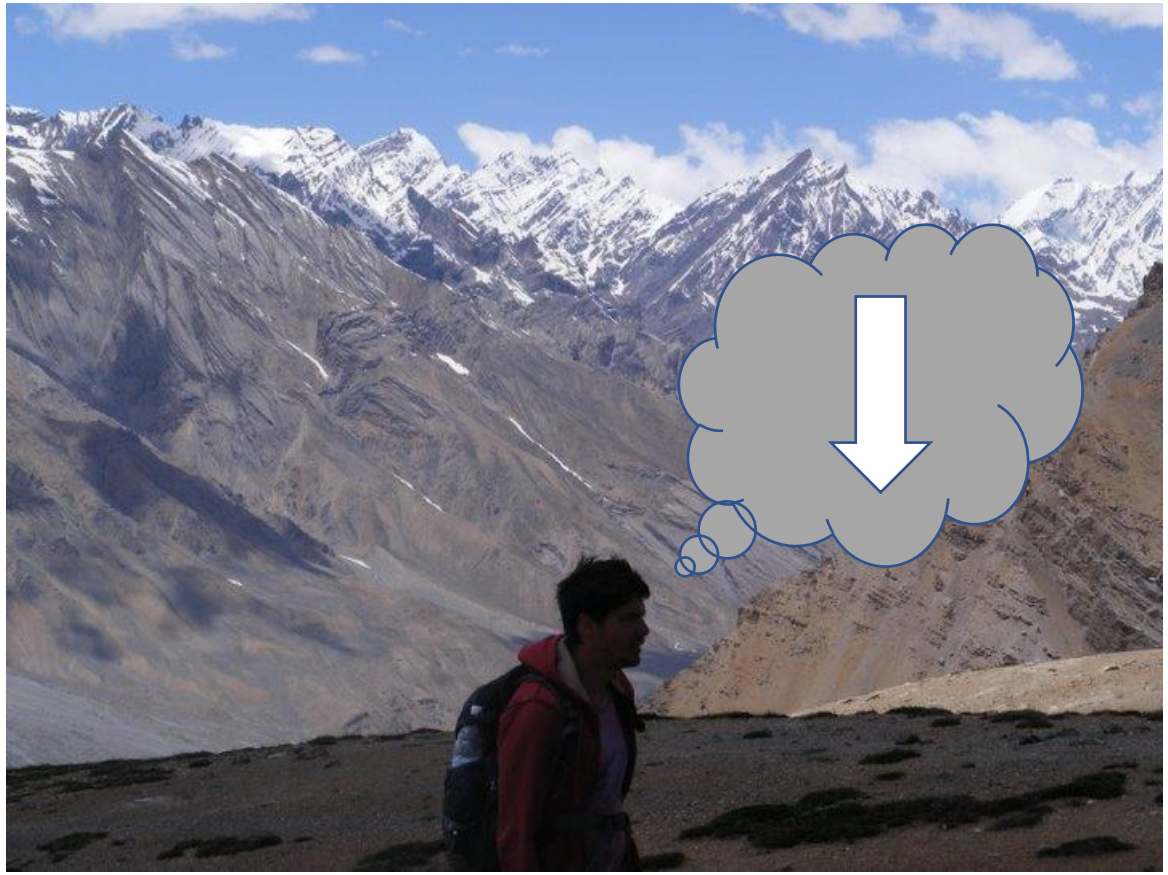
Possible solution: Iteratively reduce loss

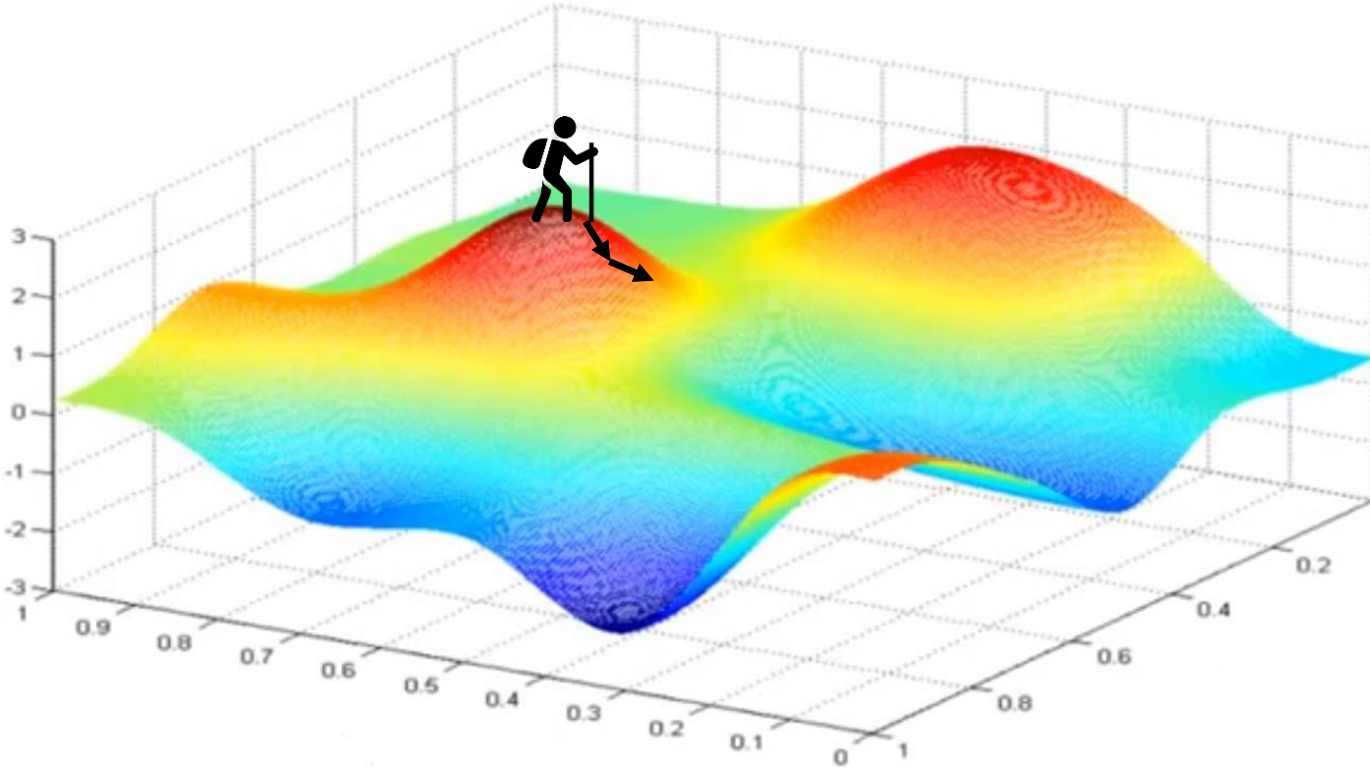




Gradient descent

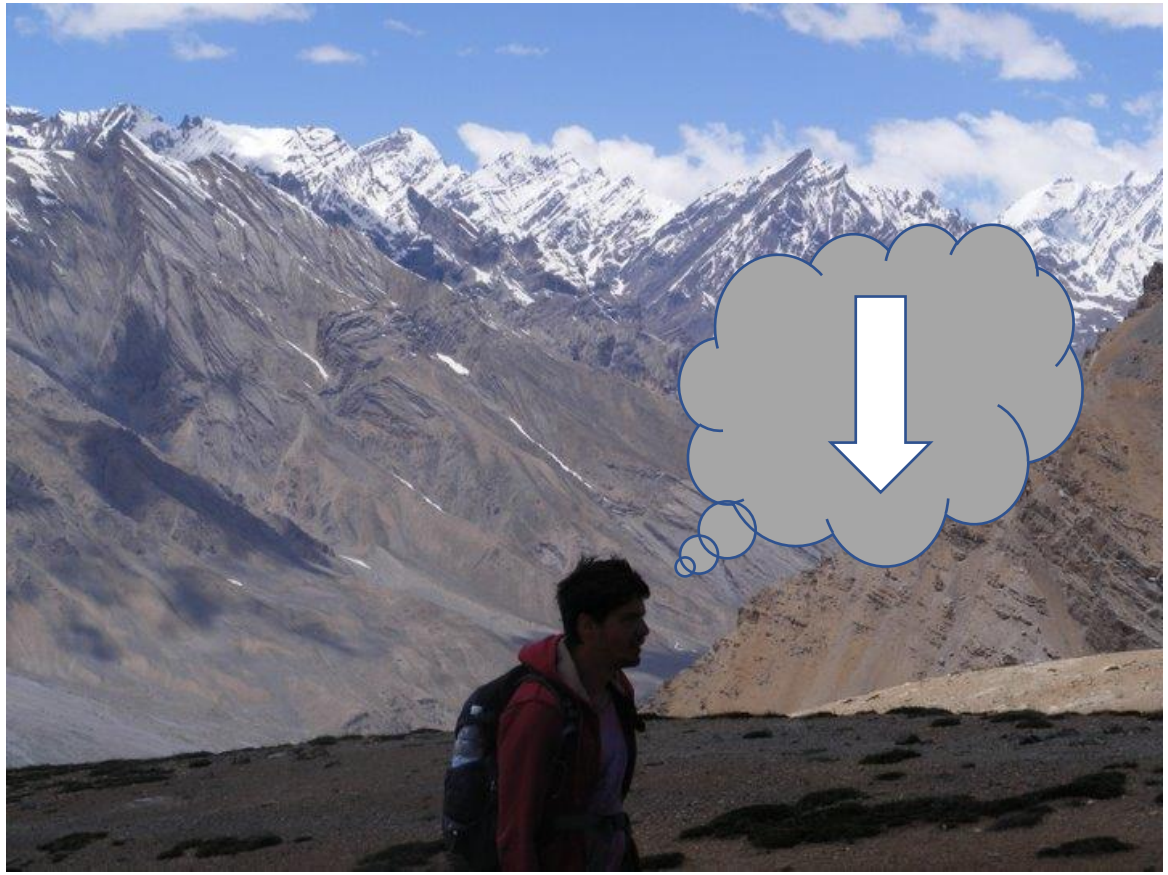
Possible solution: Iteratively reduce loss

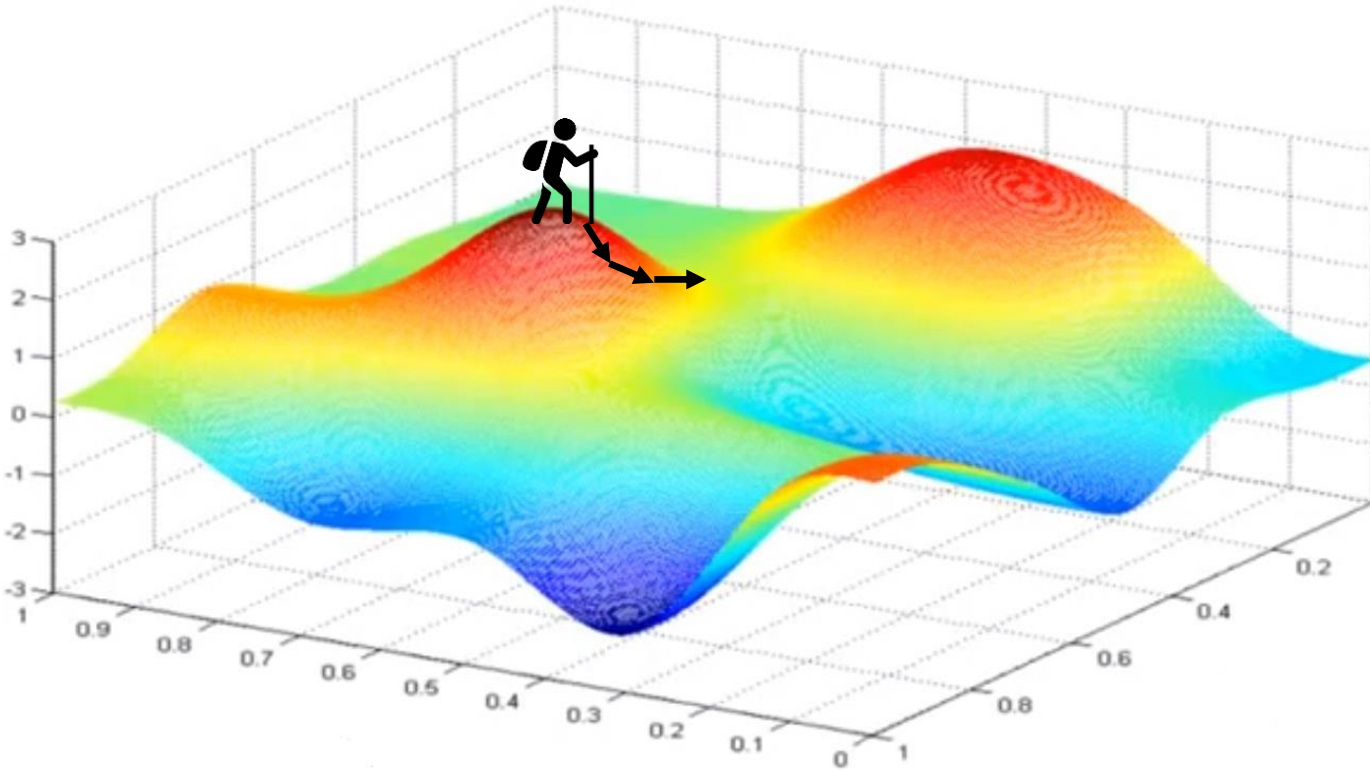




Gradient descent

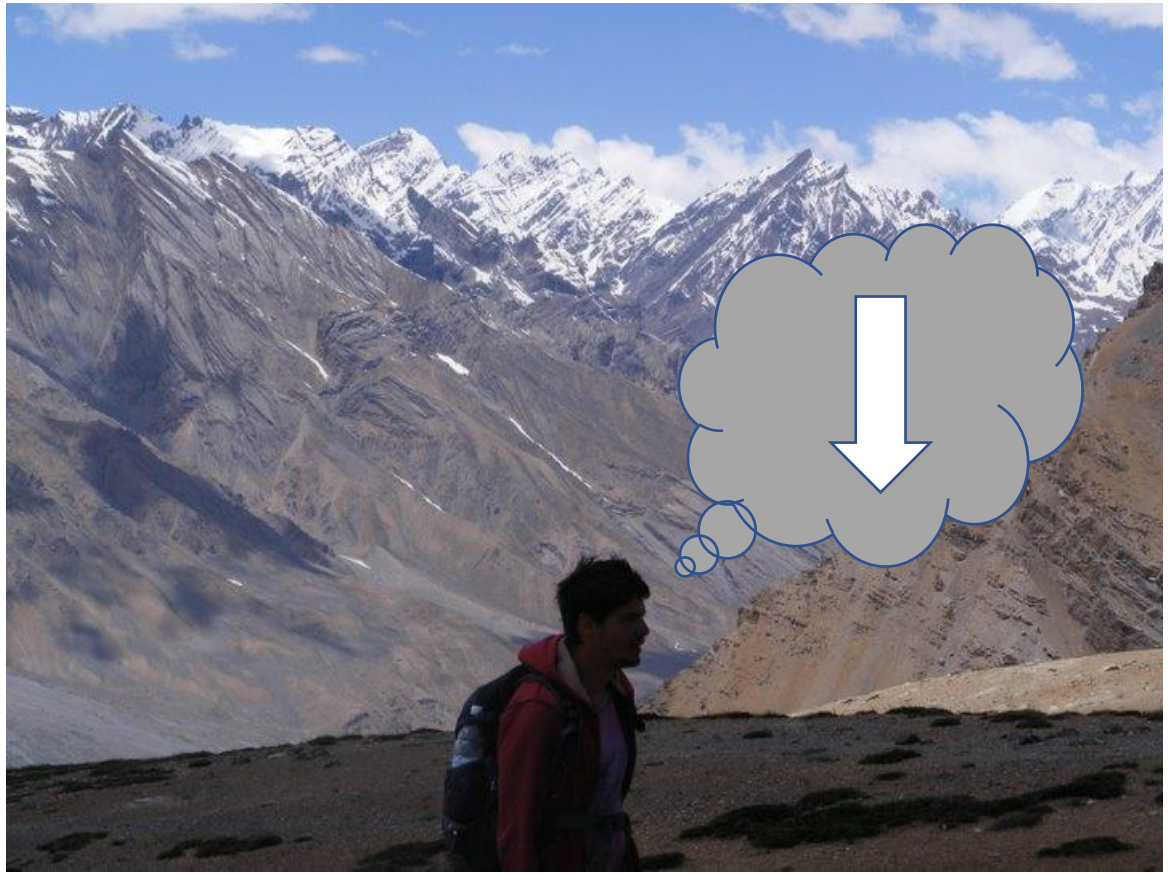
Possible solution: Iteratively reduce loss

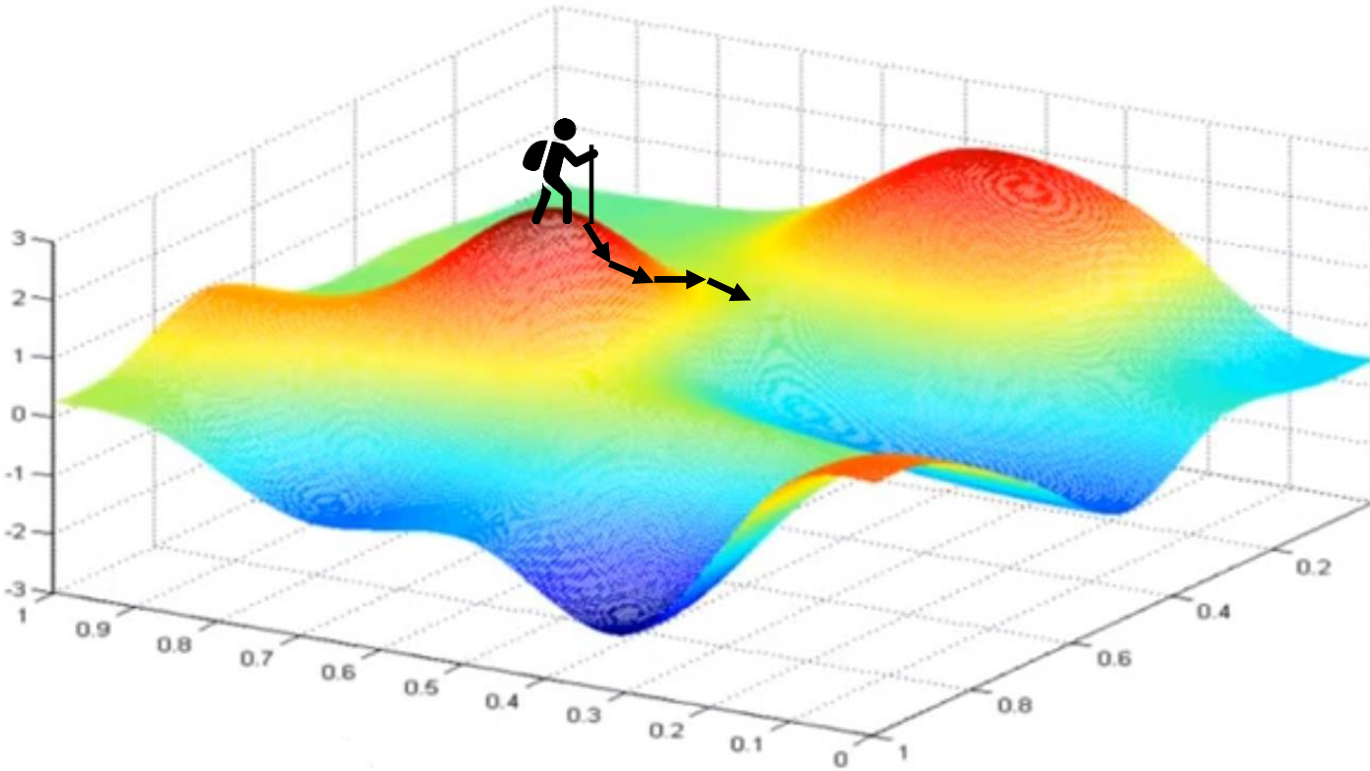




Gradient descent

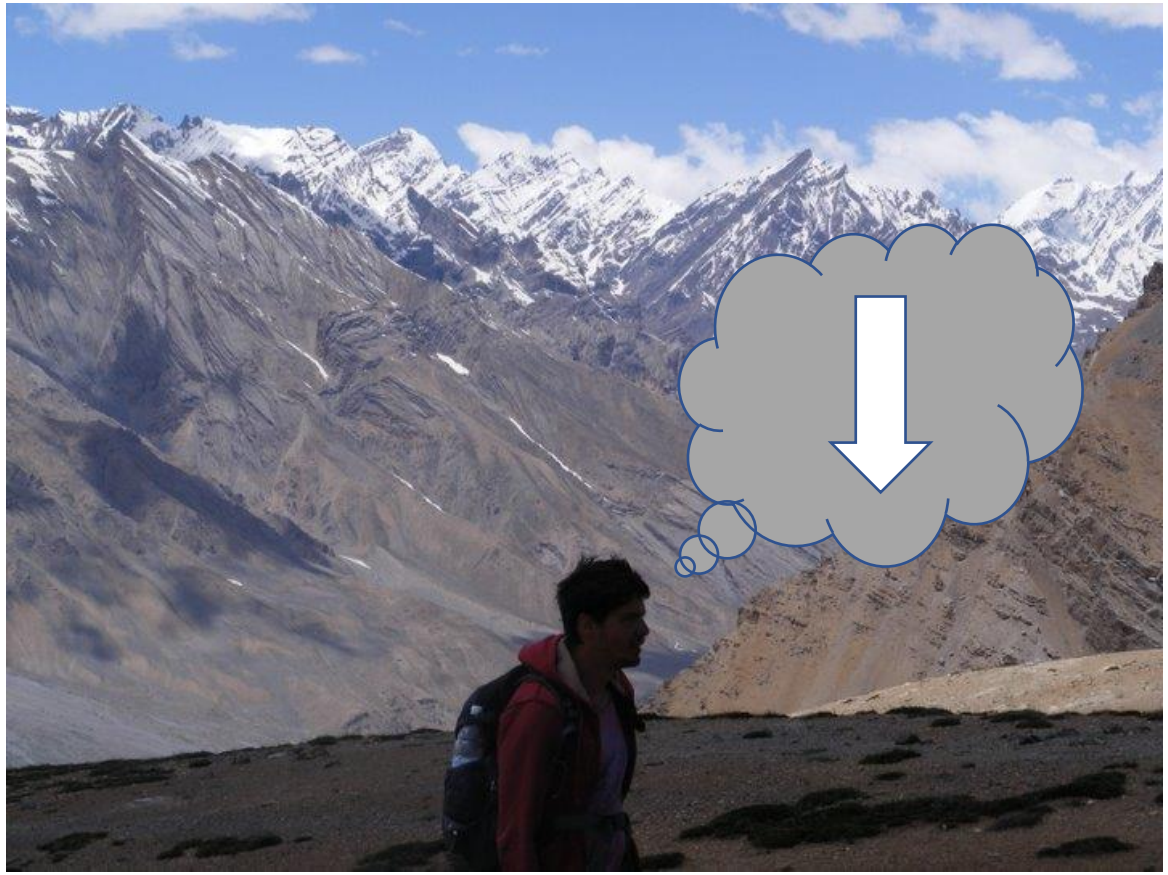
Possible solution: Iteratively reduce loss

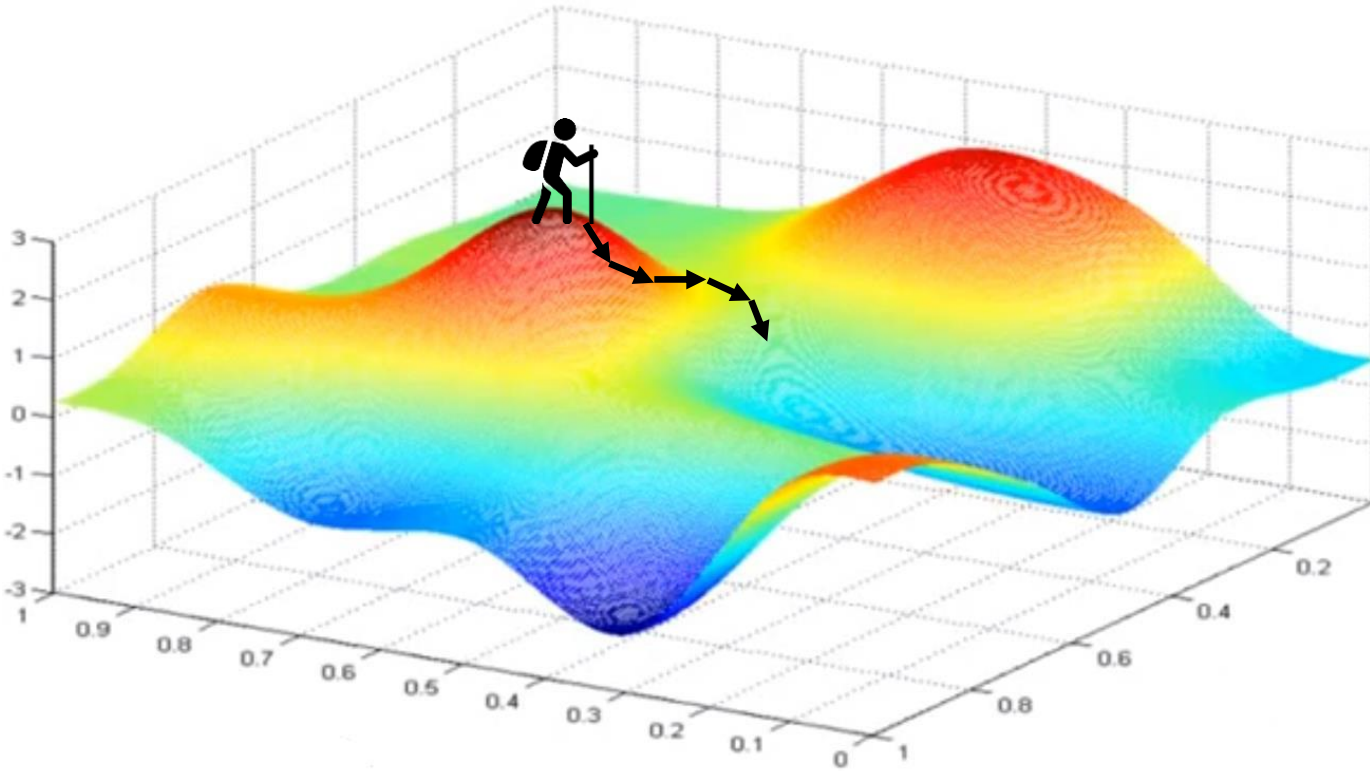




Gradient descent

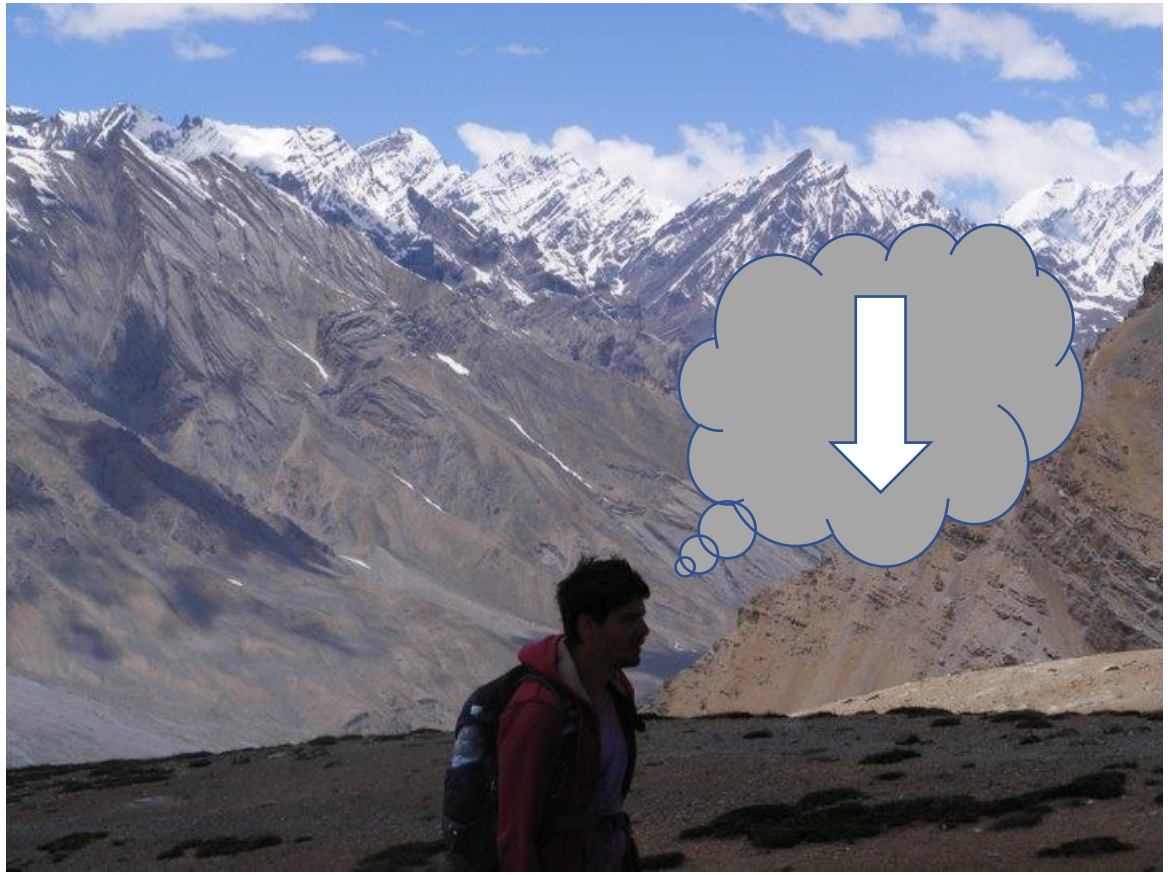
Possible solution: Iteratively reduce loss

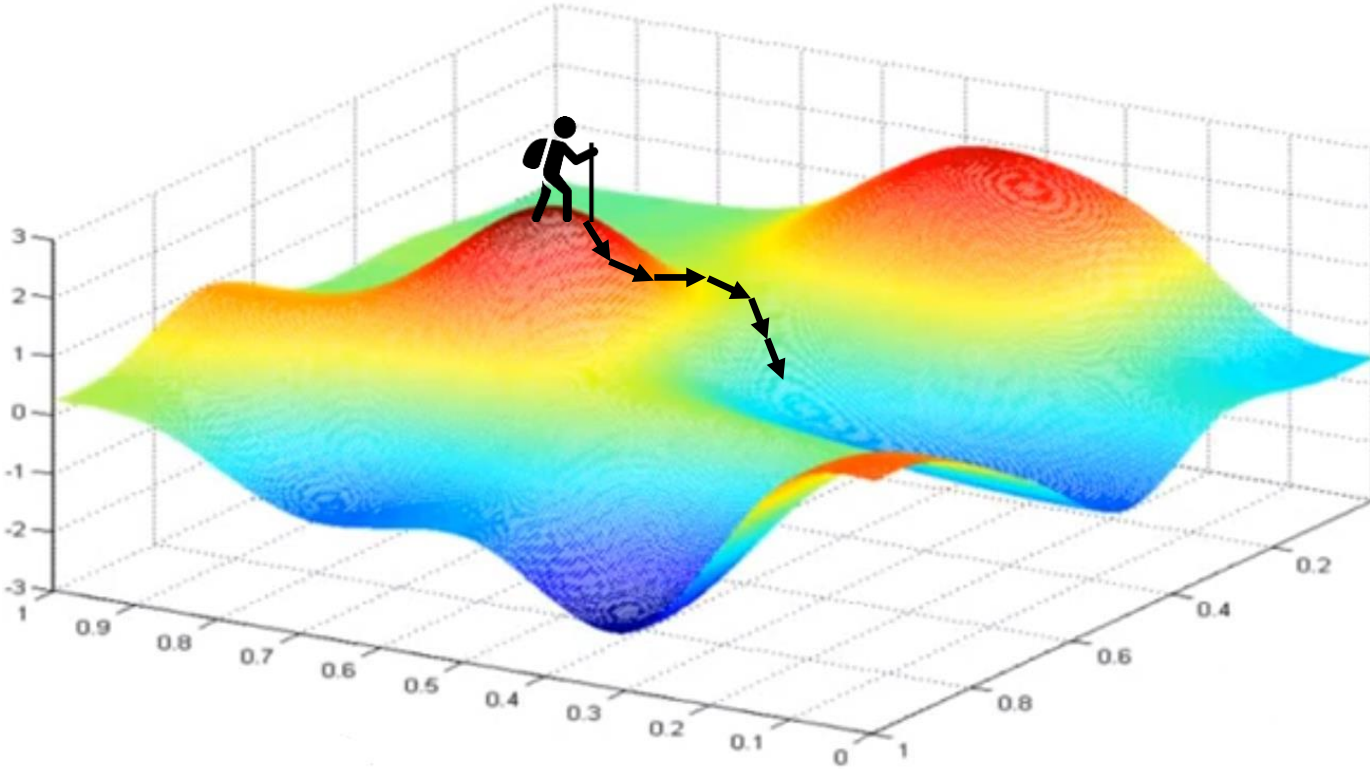




Gradient descent

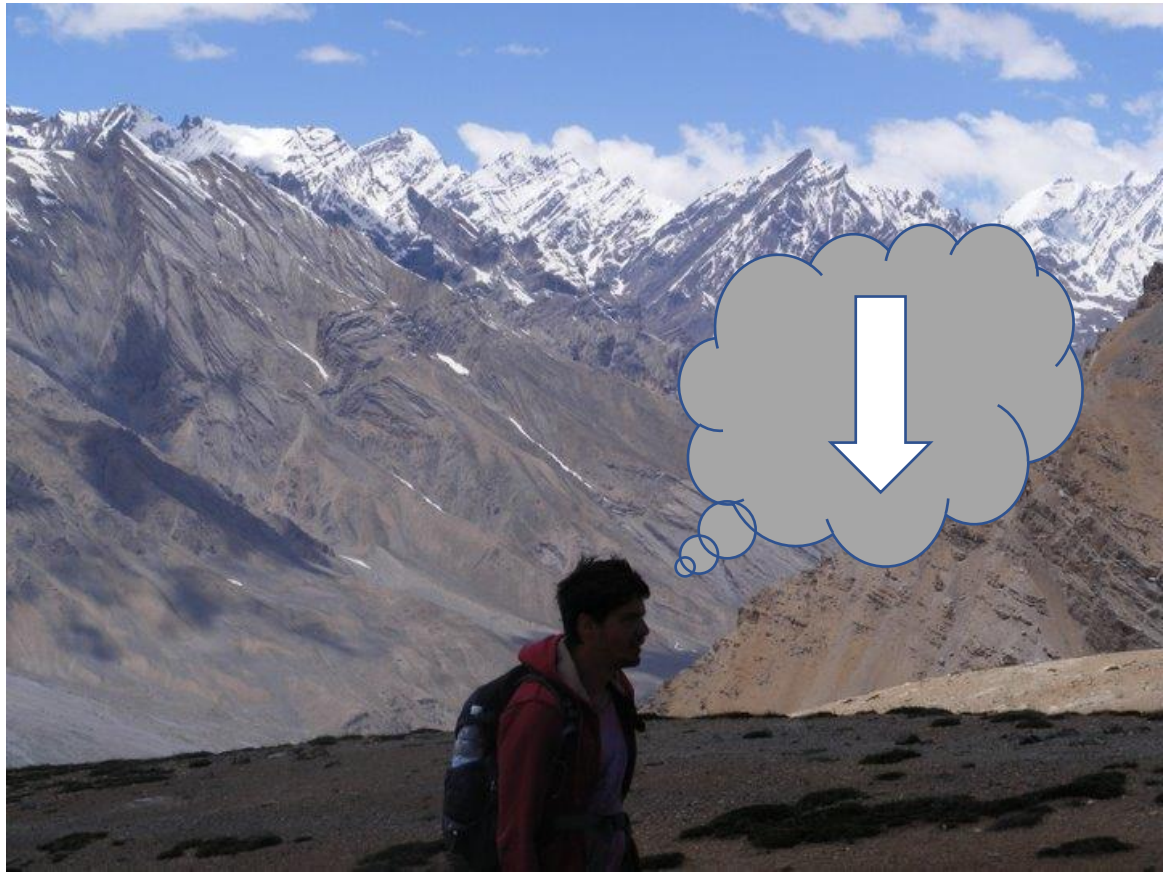
Possible solution: Iteratively reduce loss

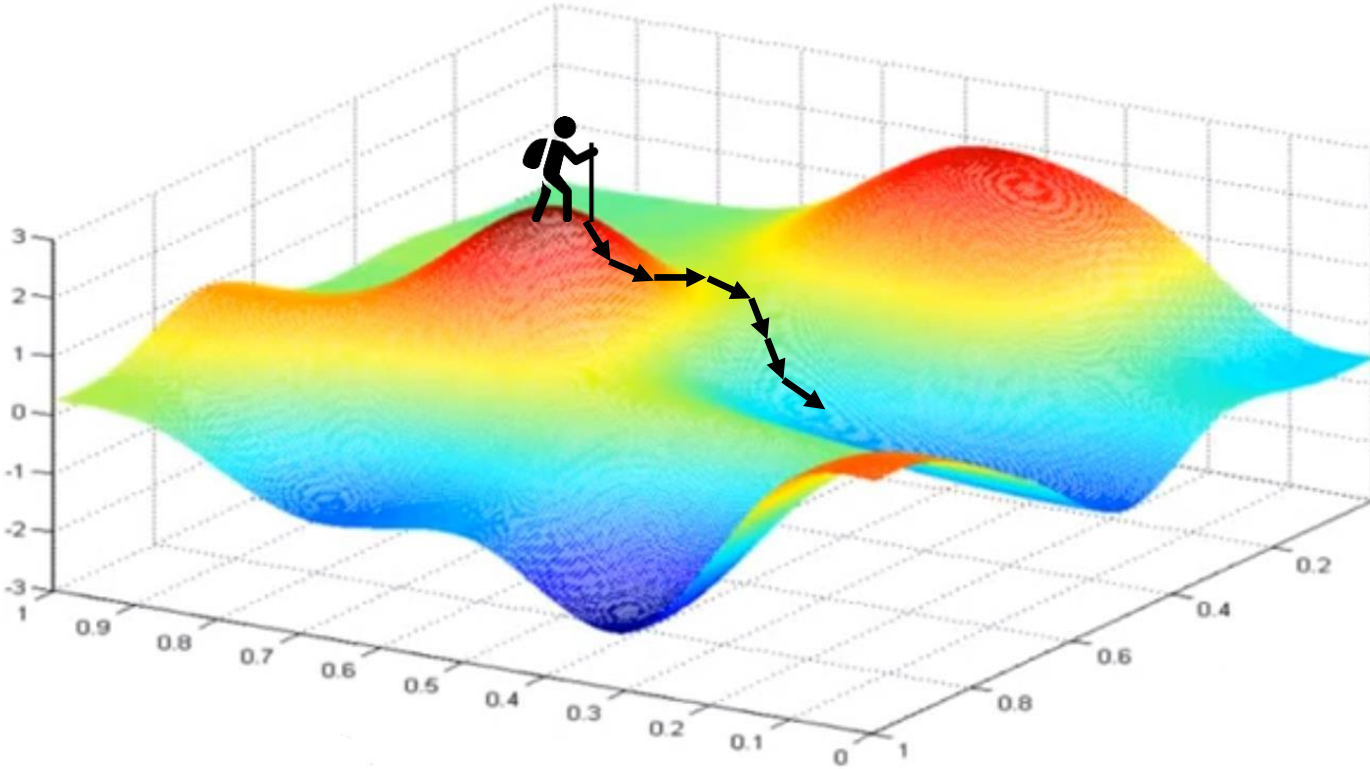




Gradient descent

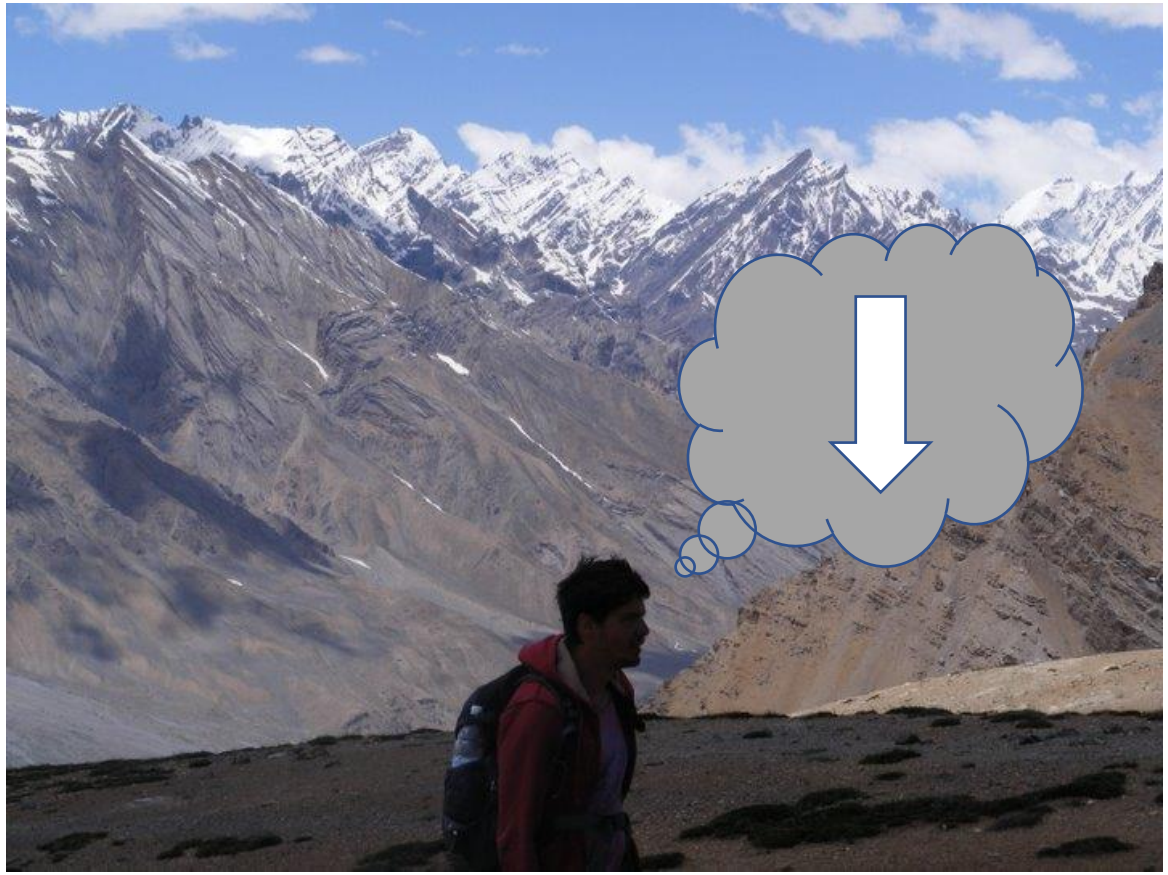
Possible solution: Iteratively reduce loss

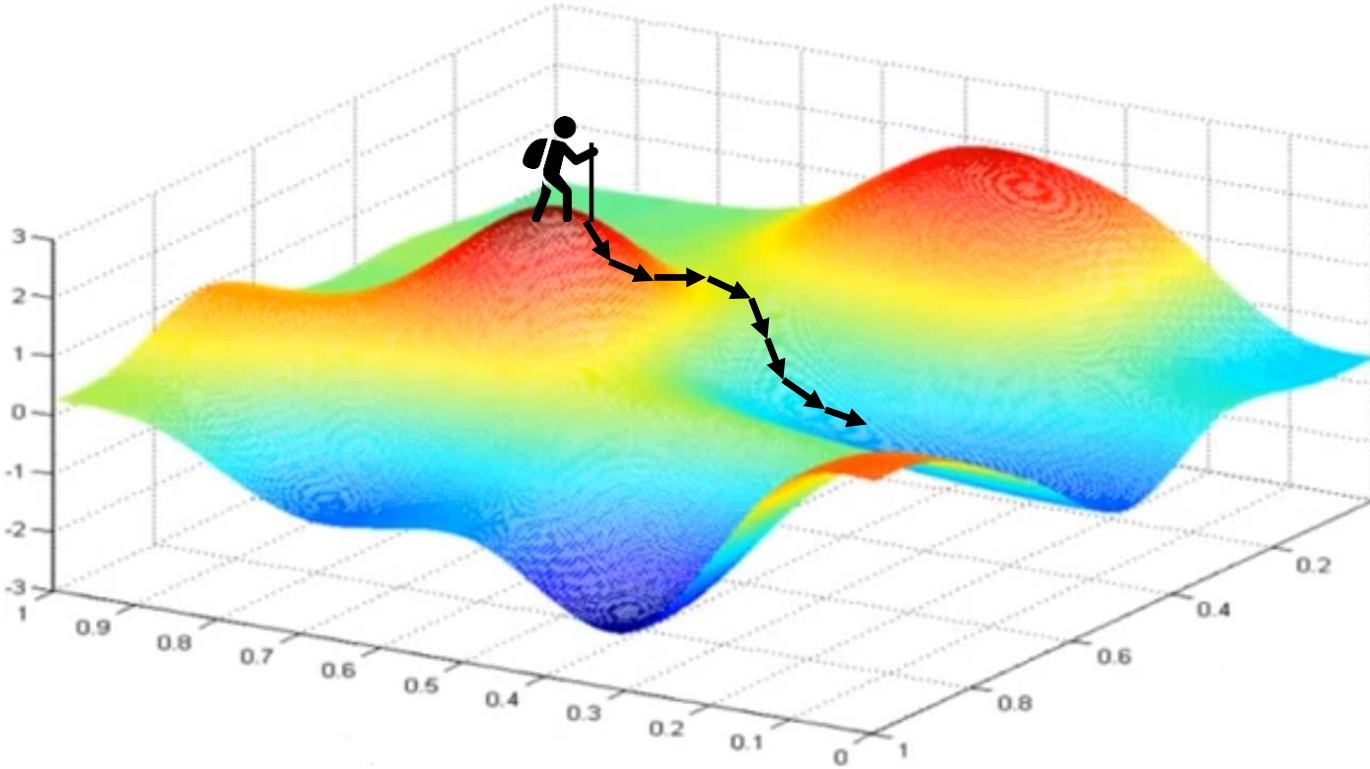




Gradient descent

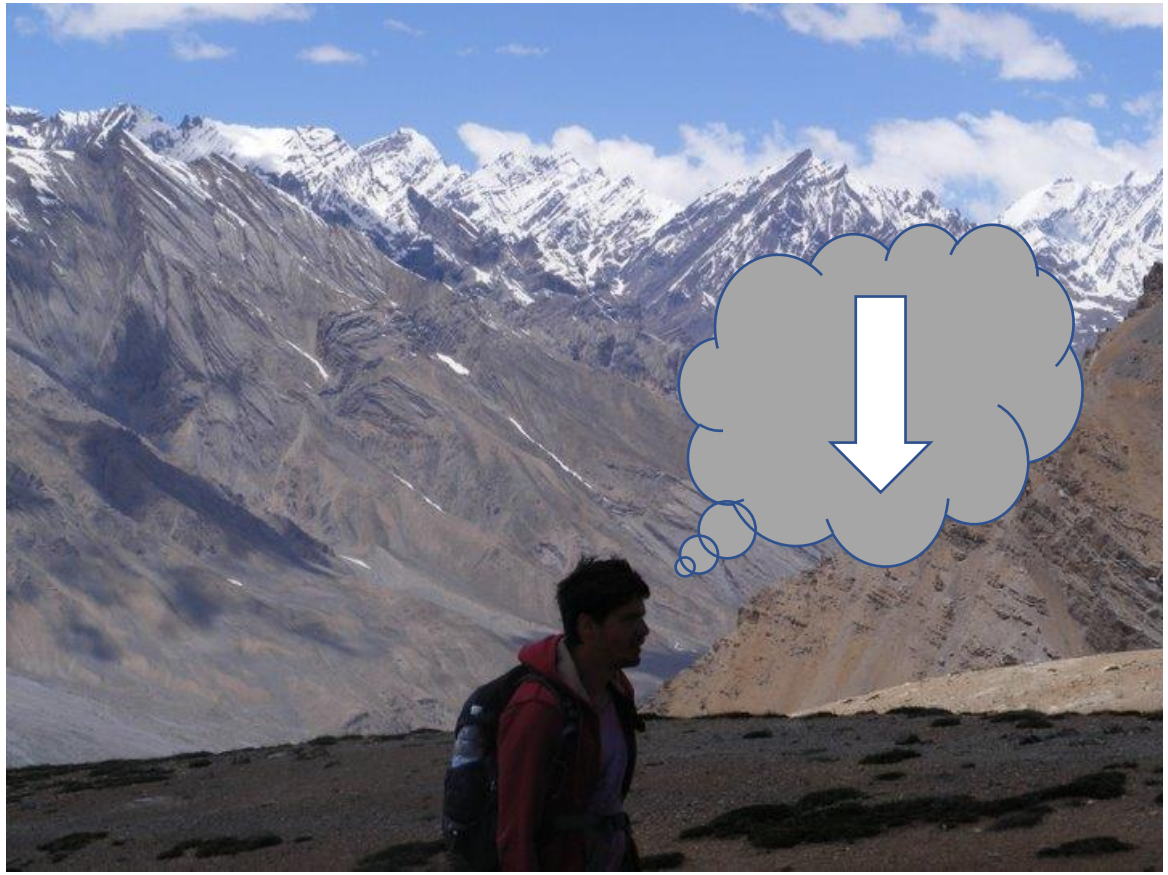
Possible solution: Iteratively reduce loss

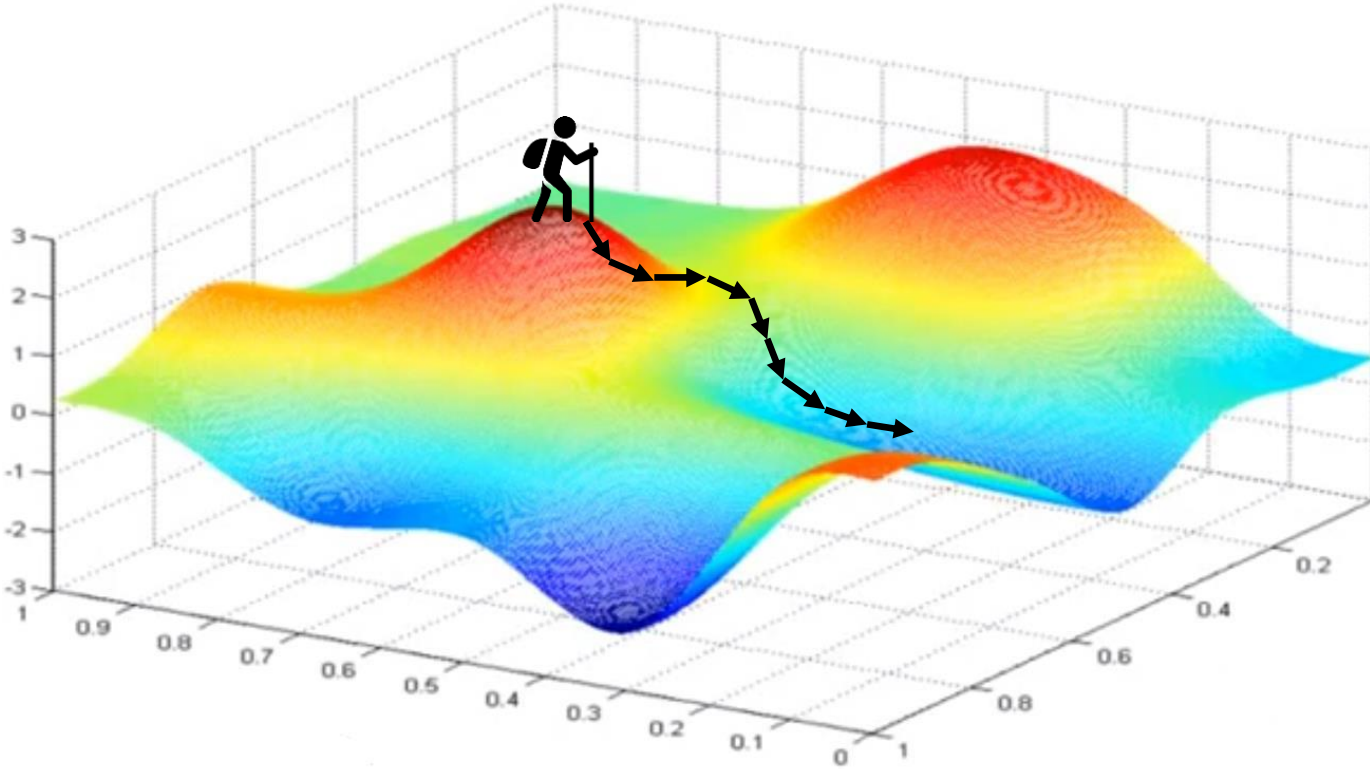




Gradient descent

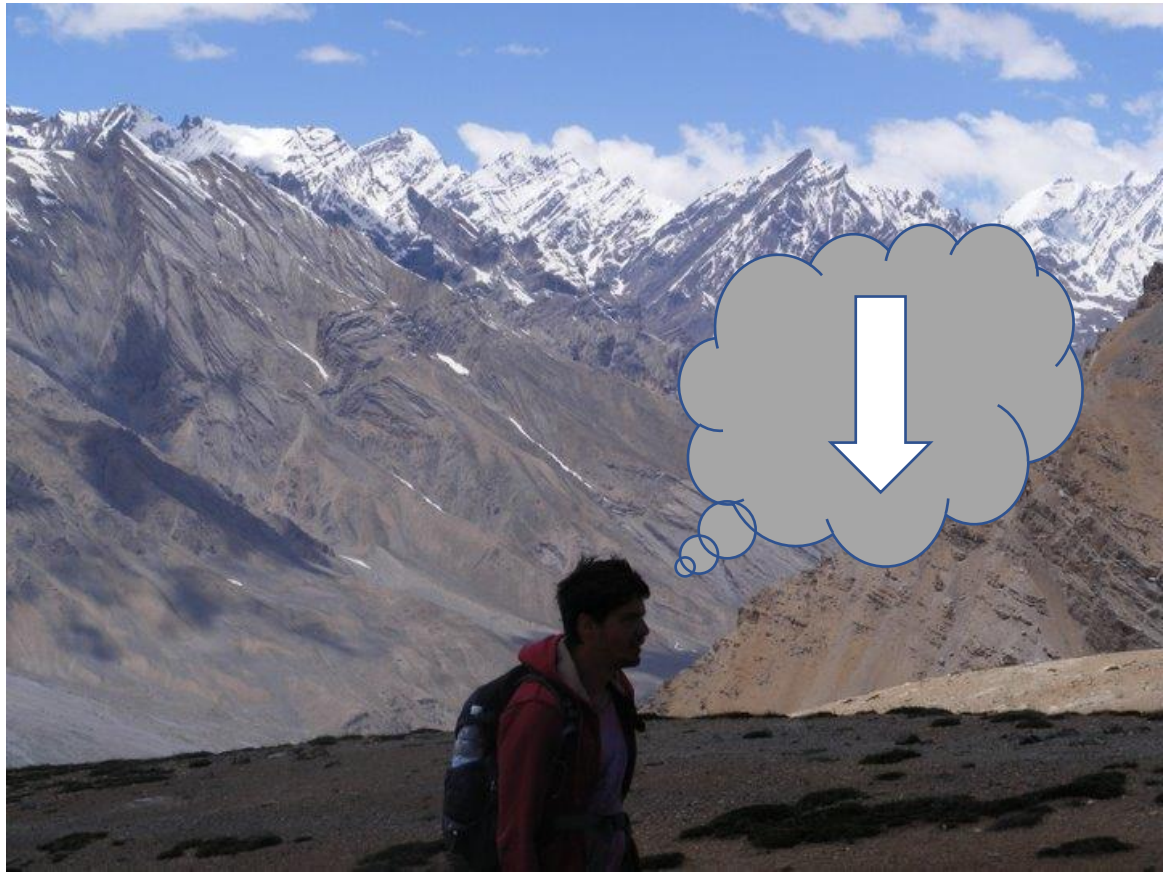
Possible solution: Iteratively reduce loss

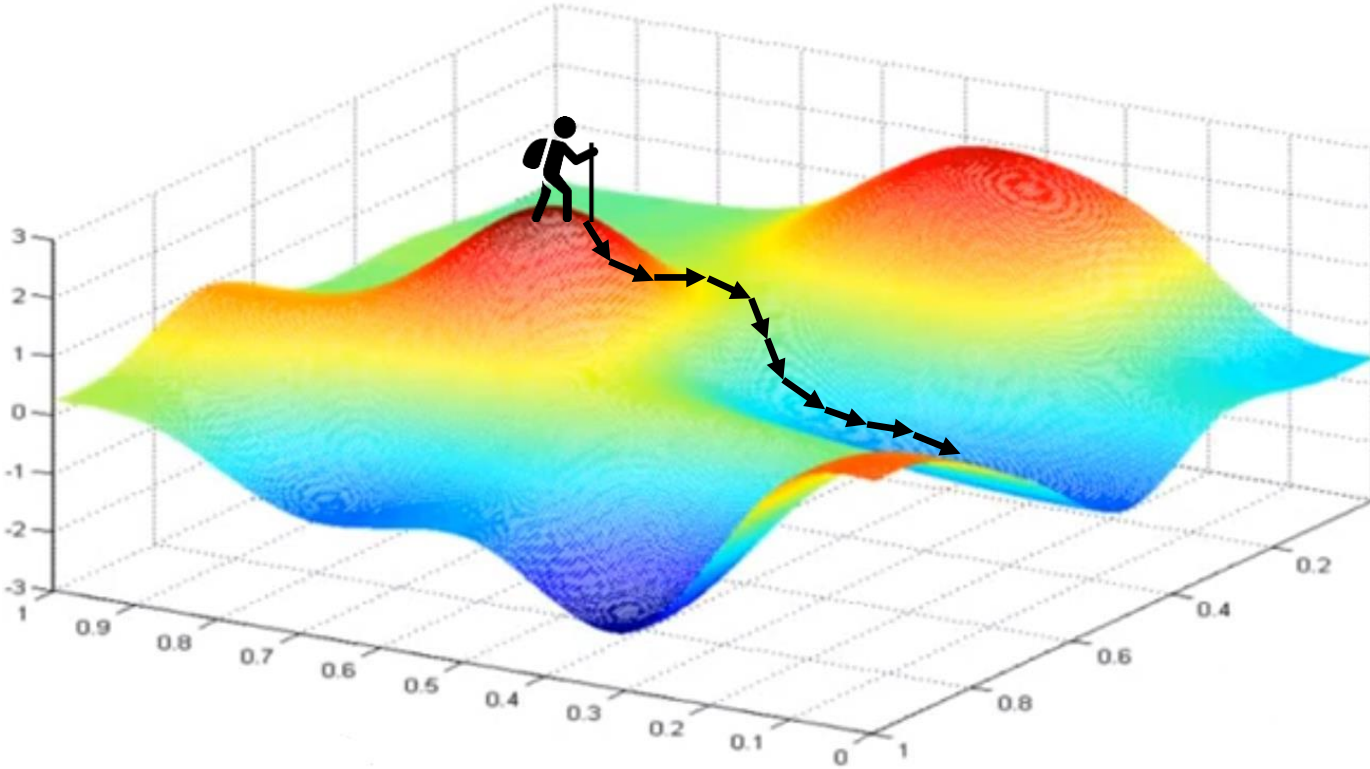




Gradient descent

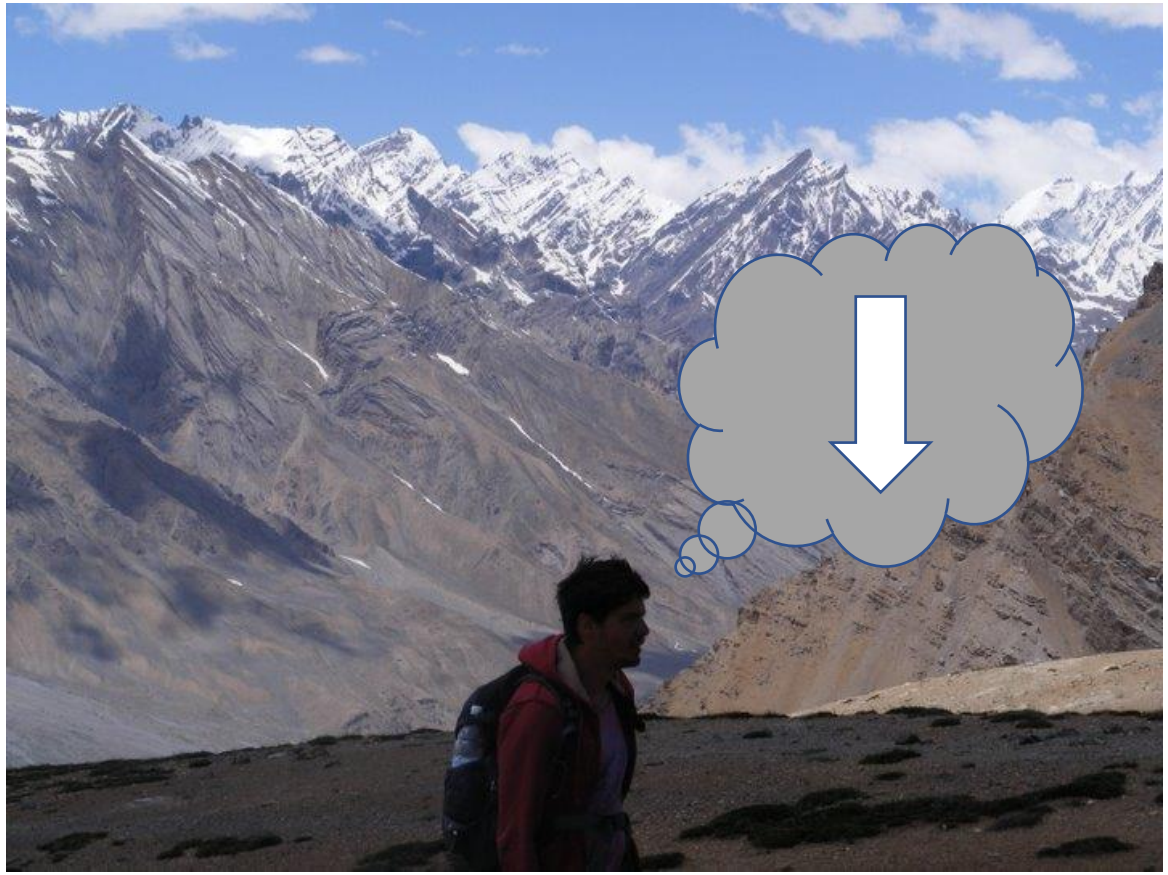
Possible solution: Iteratively reduce loss

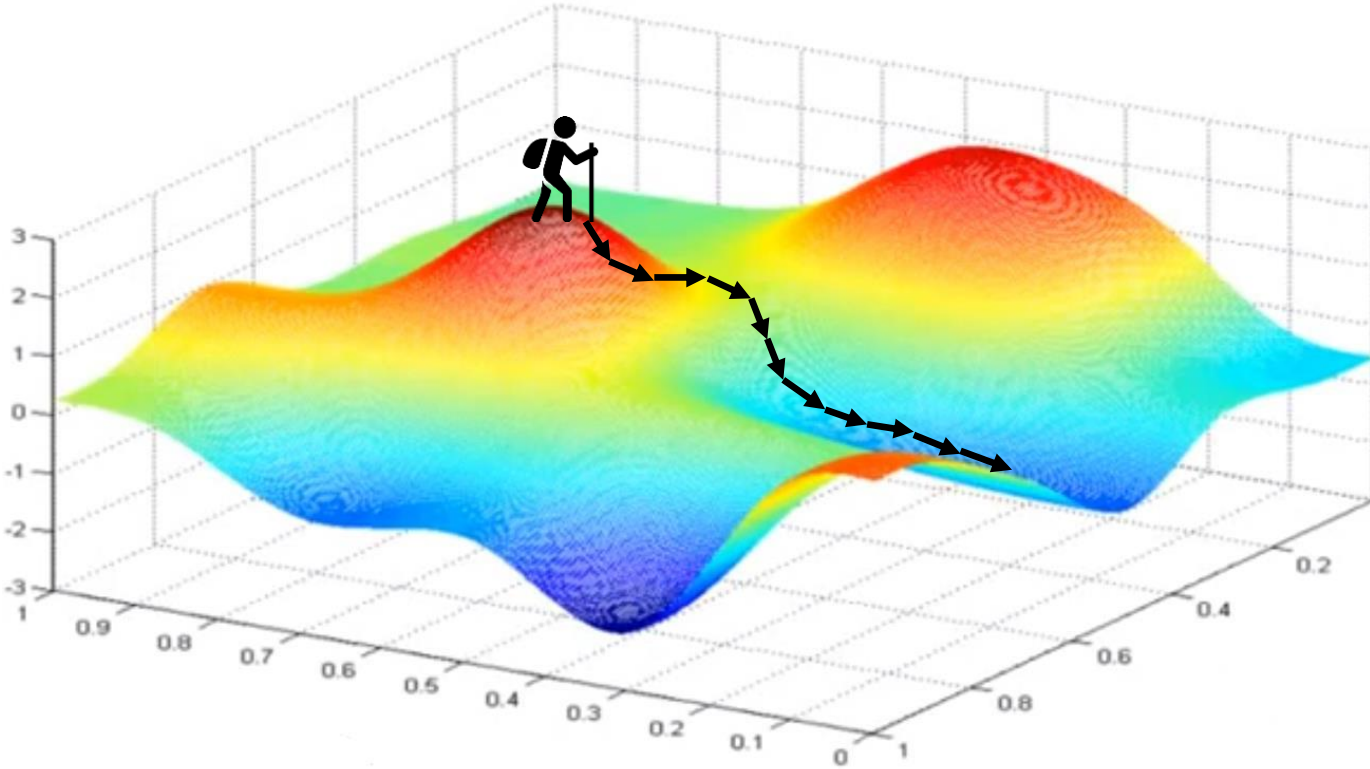




Gradient descent

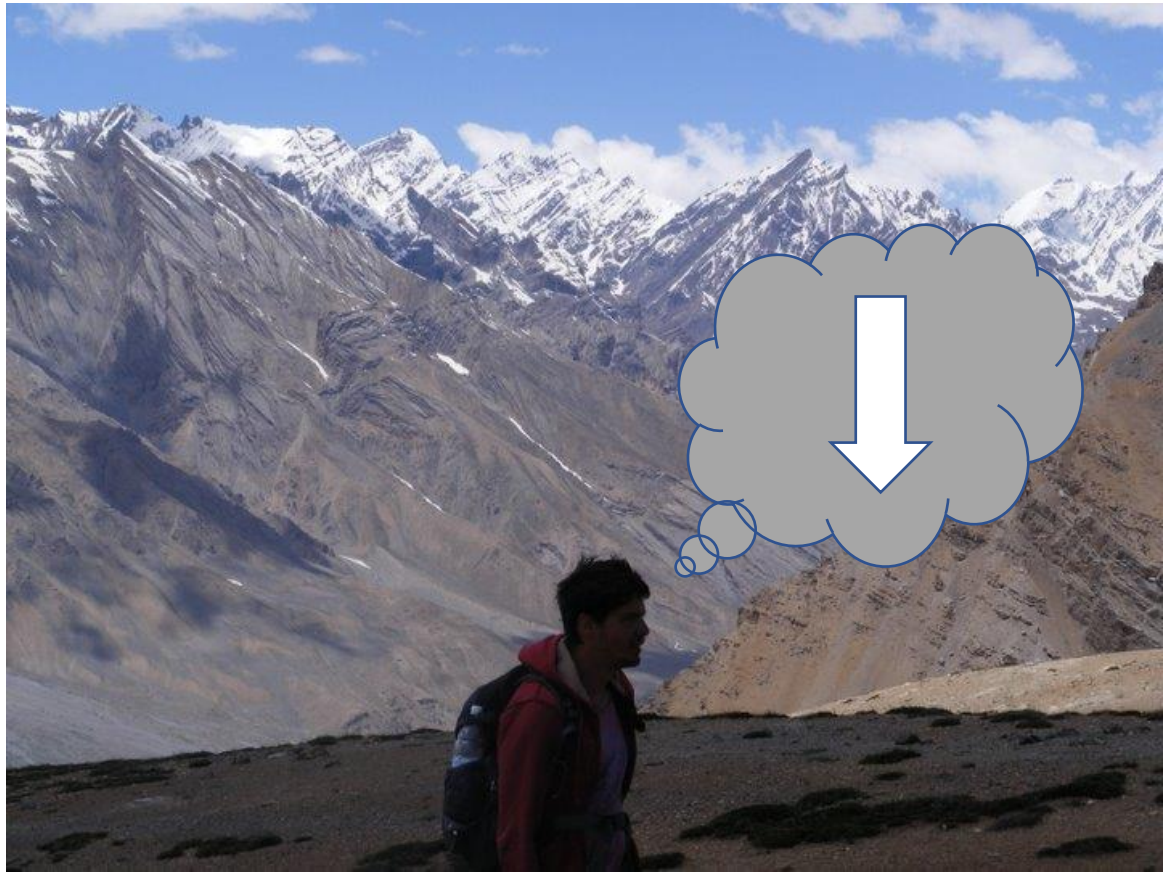
Possible solution: Iteratively reduce loss

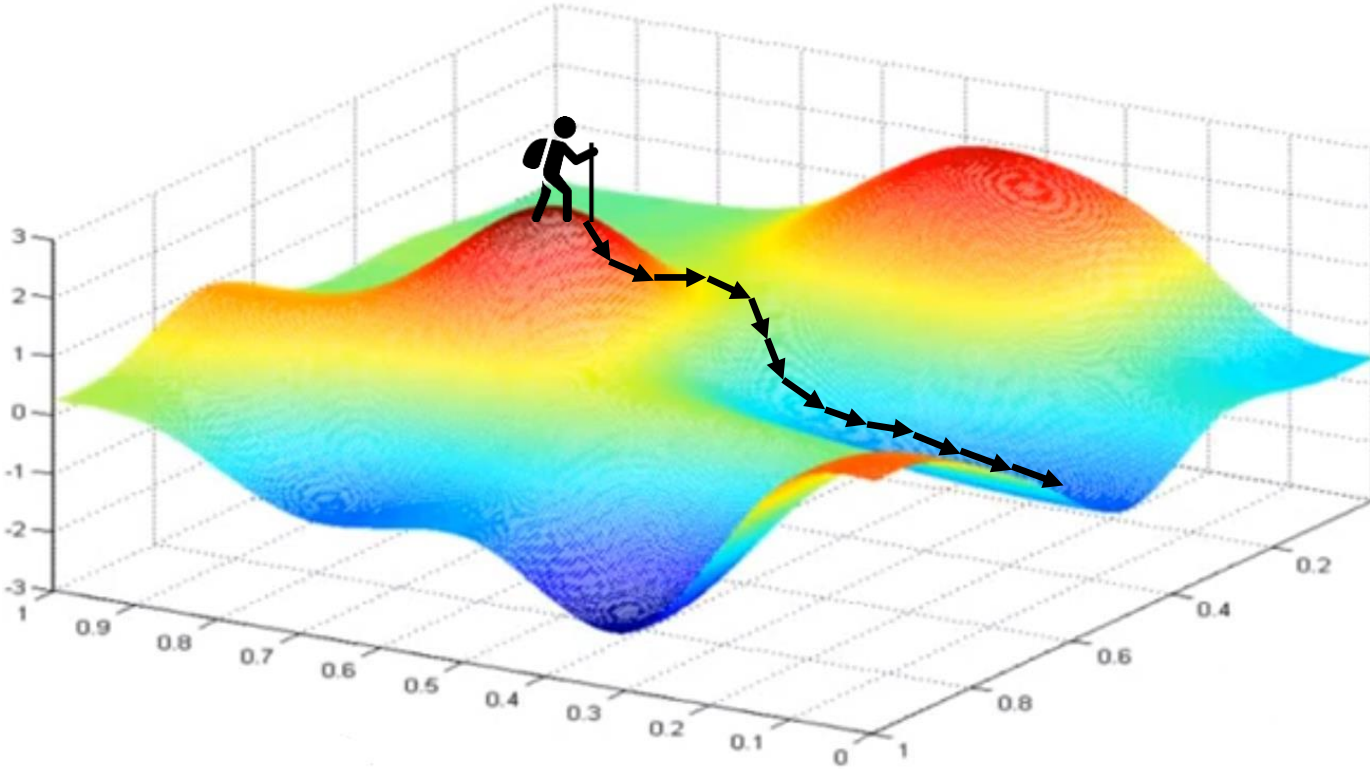




Gradient descent

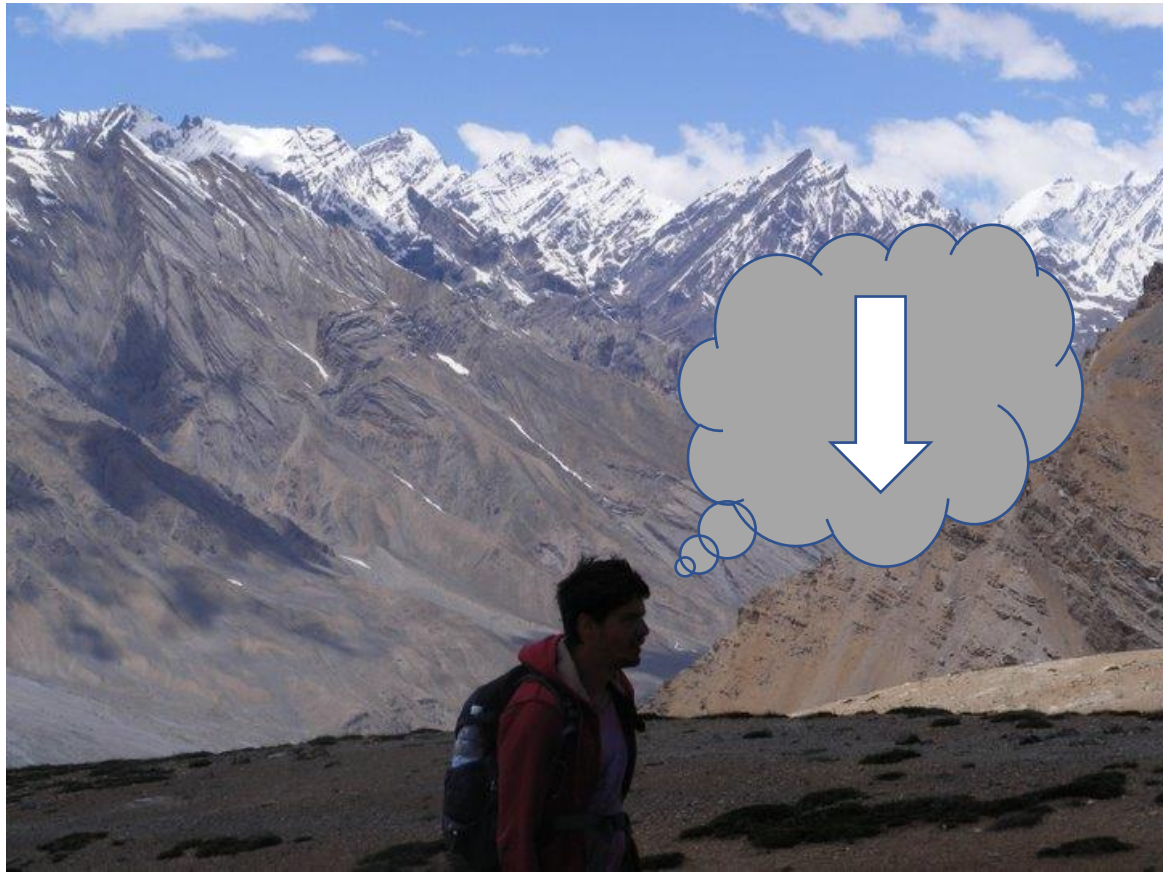
Possible solution: Iteratively reduce loss

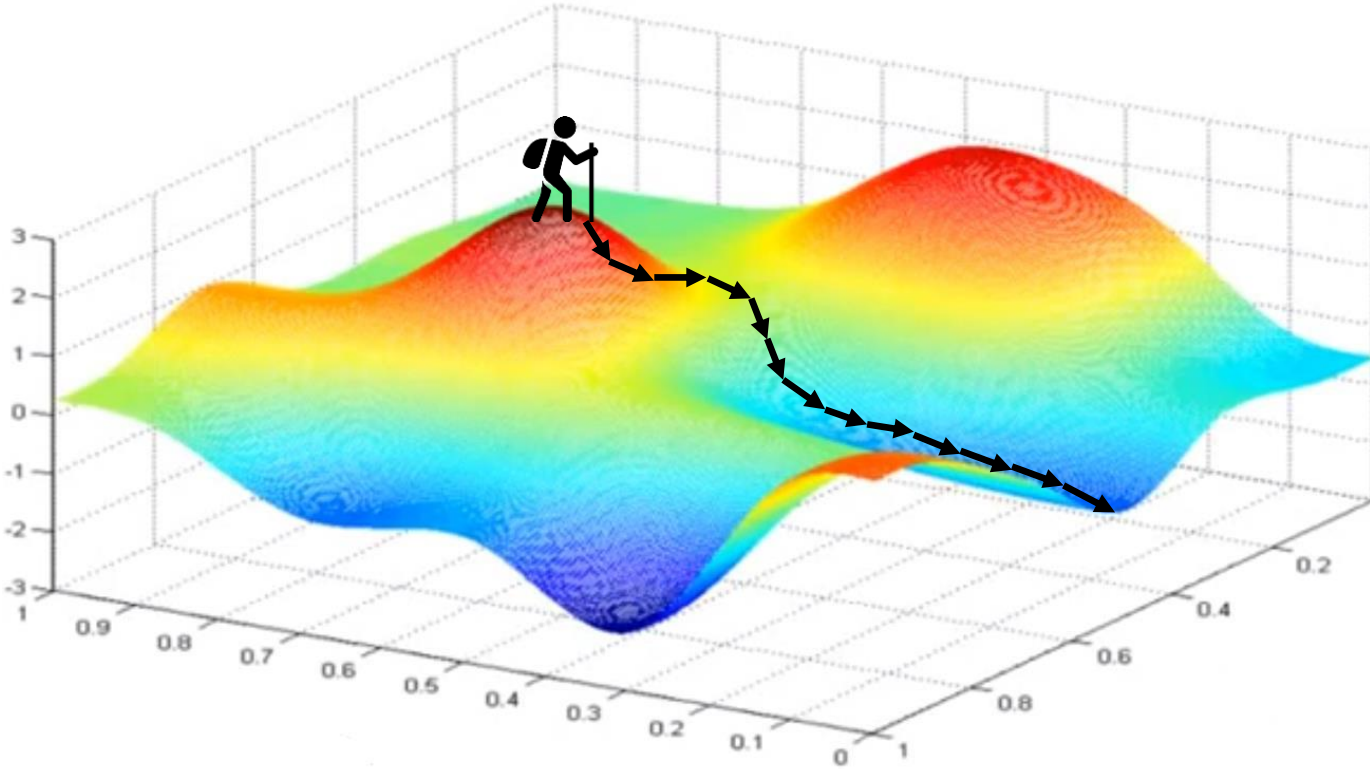




Gradient descent

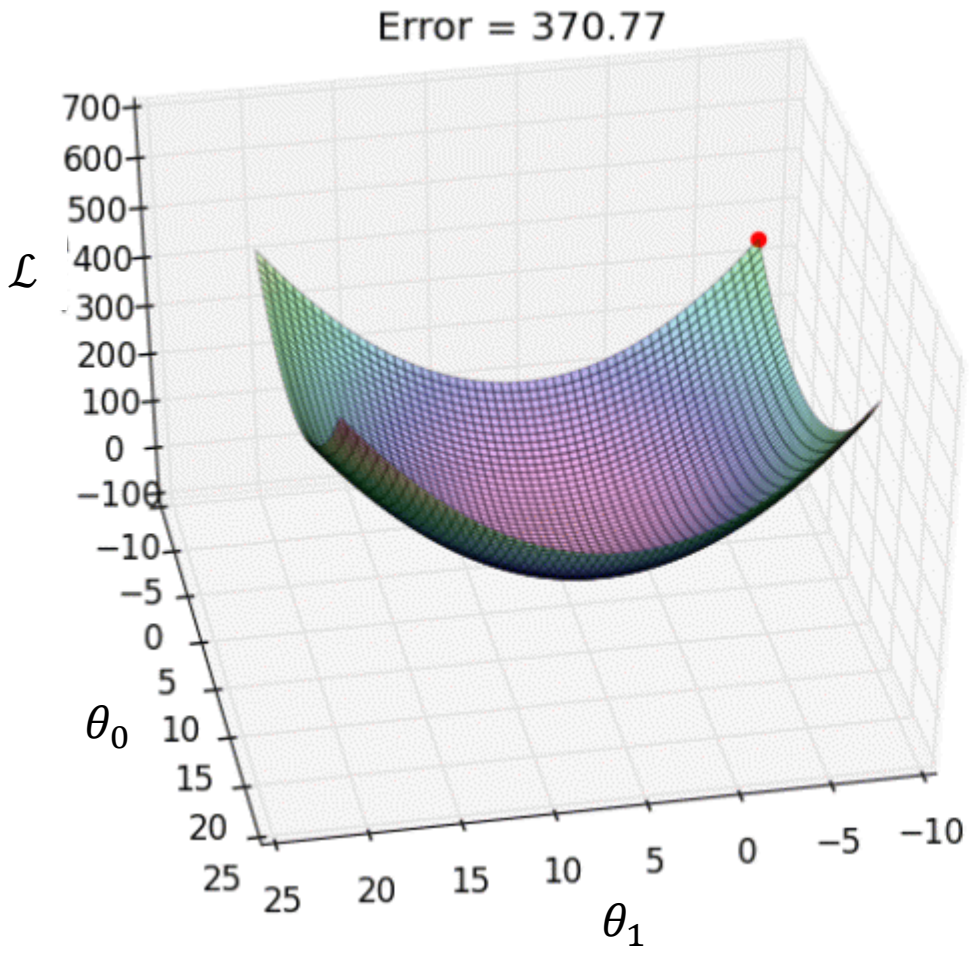
Possible solution: Iteratively reduce loss

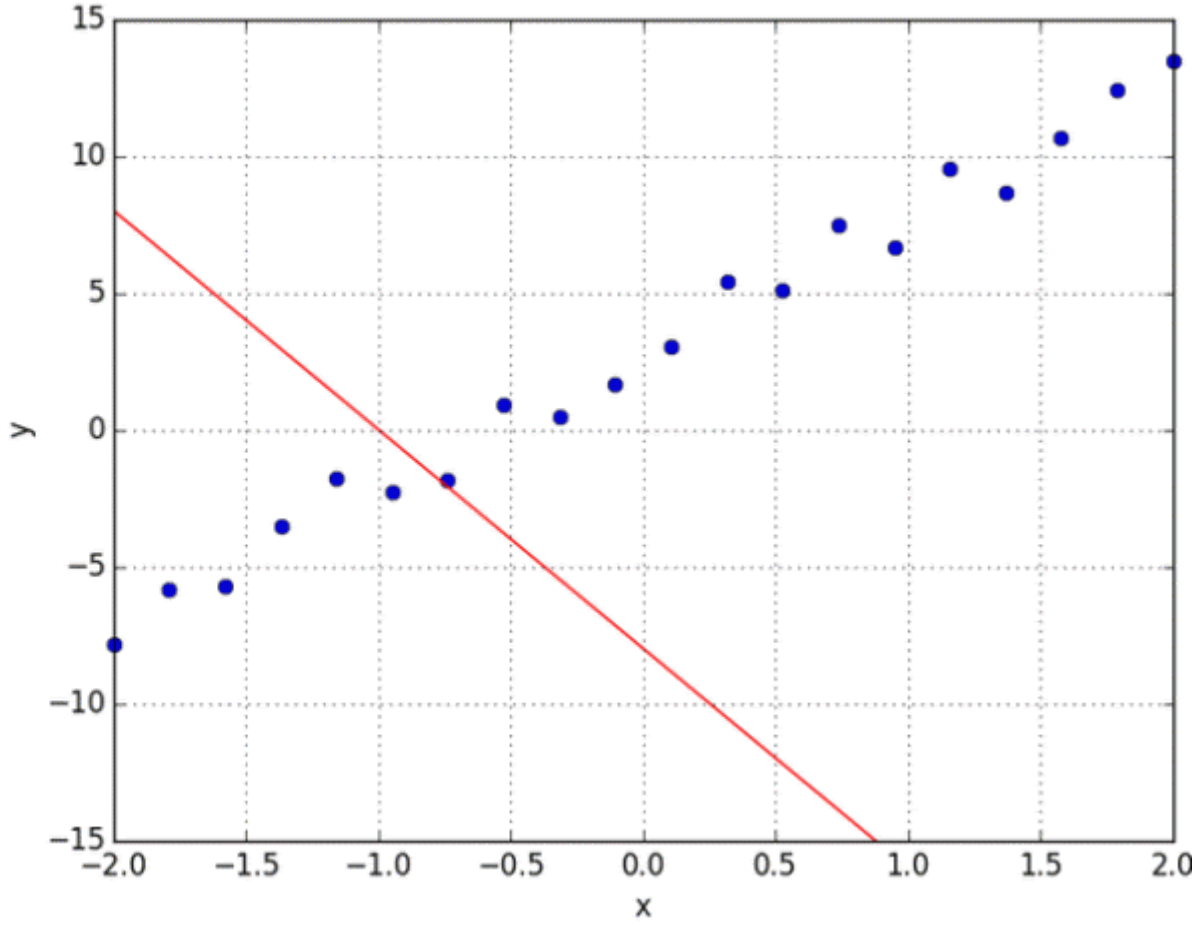




Can we guarantee global min?

Gradient descent

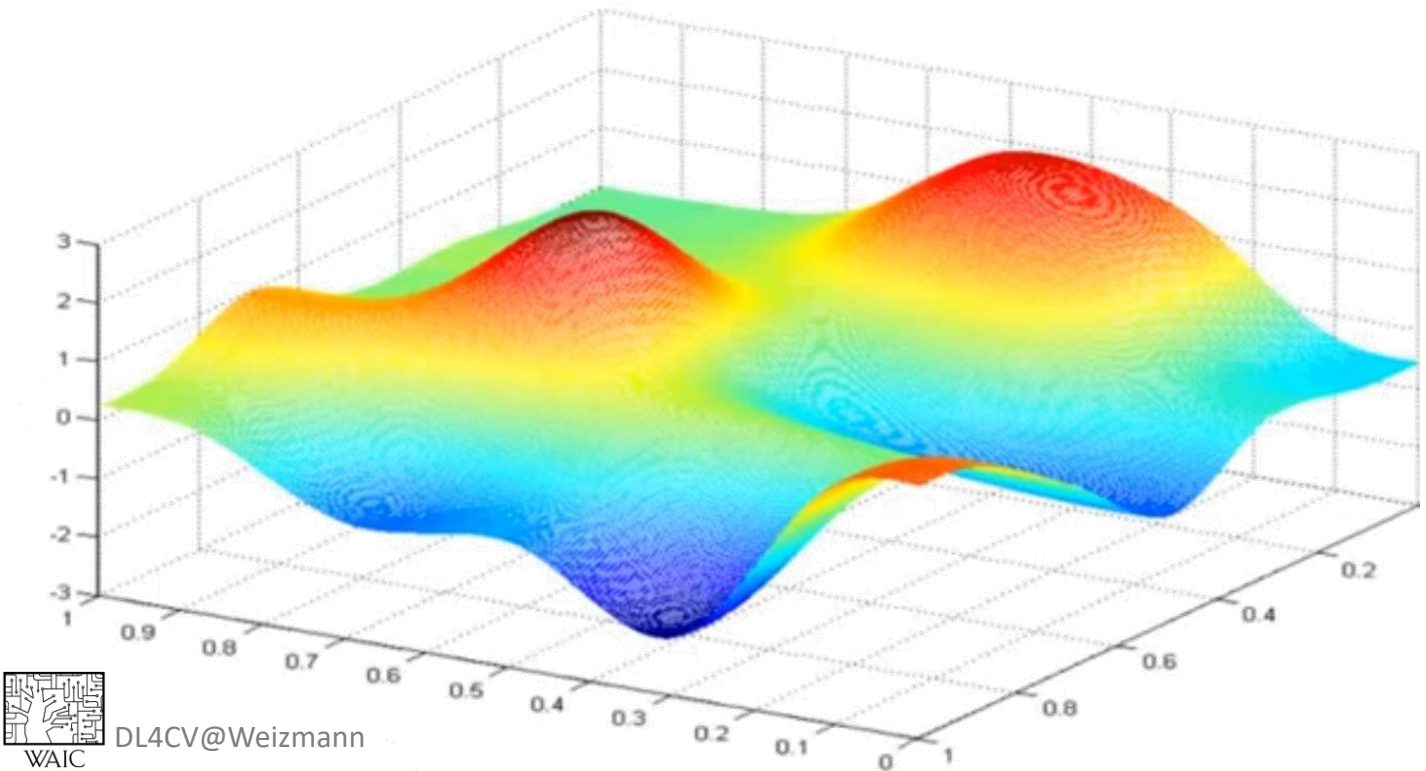




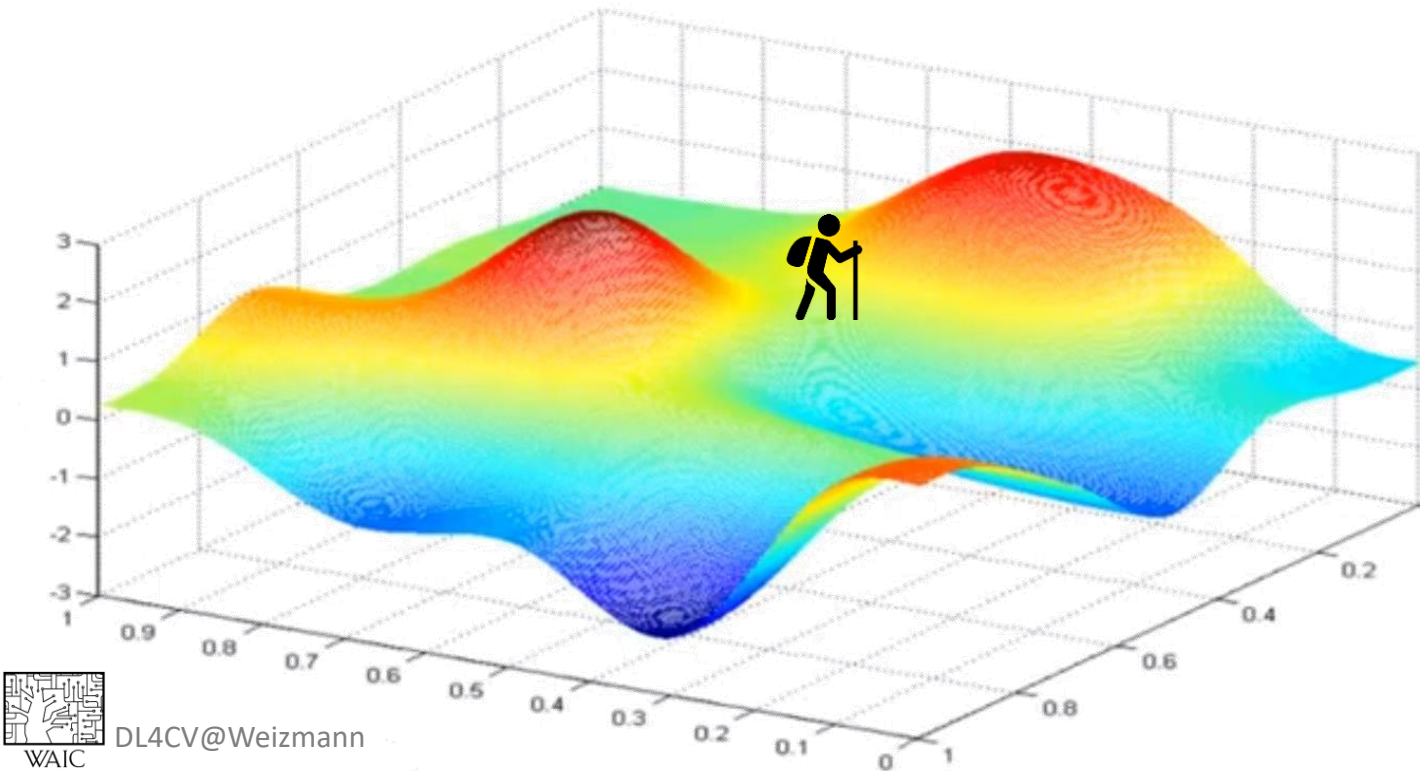
Parameter space: $\mathcal{L}(\boldsymbol{\theta}; S)$

Data space: $h_{\boldsymbol{\theta}}(\boldsymbol{x})$

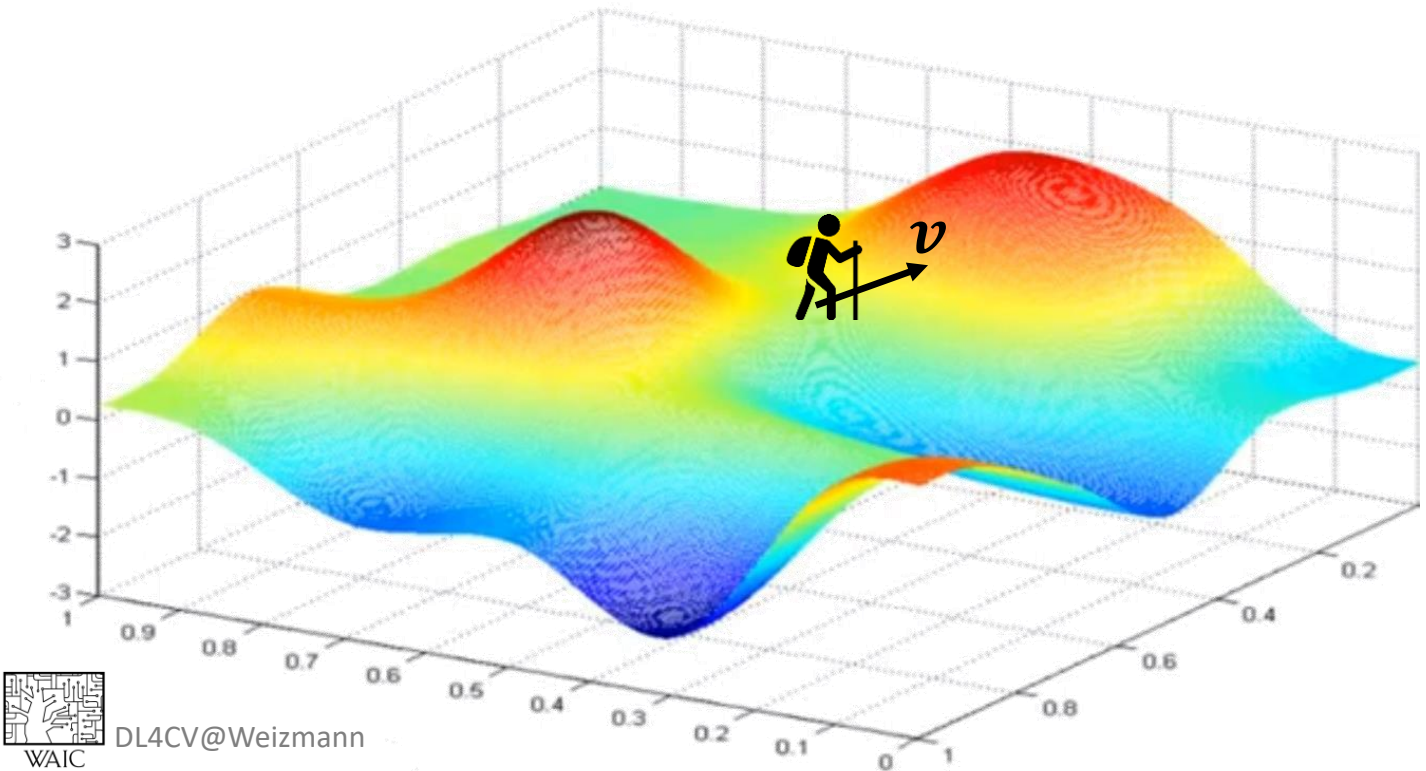
Calculus reminder: Directional derivative



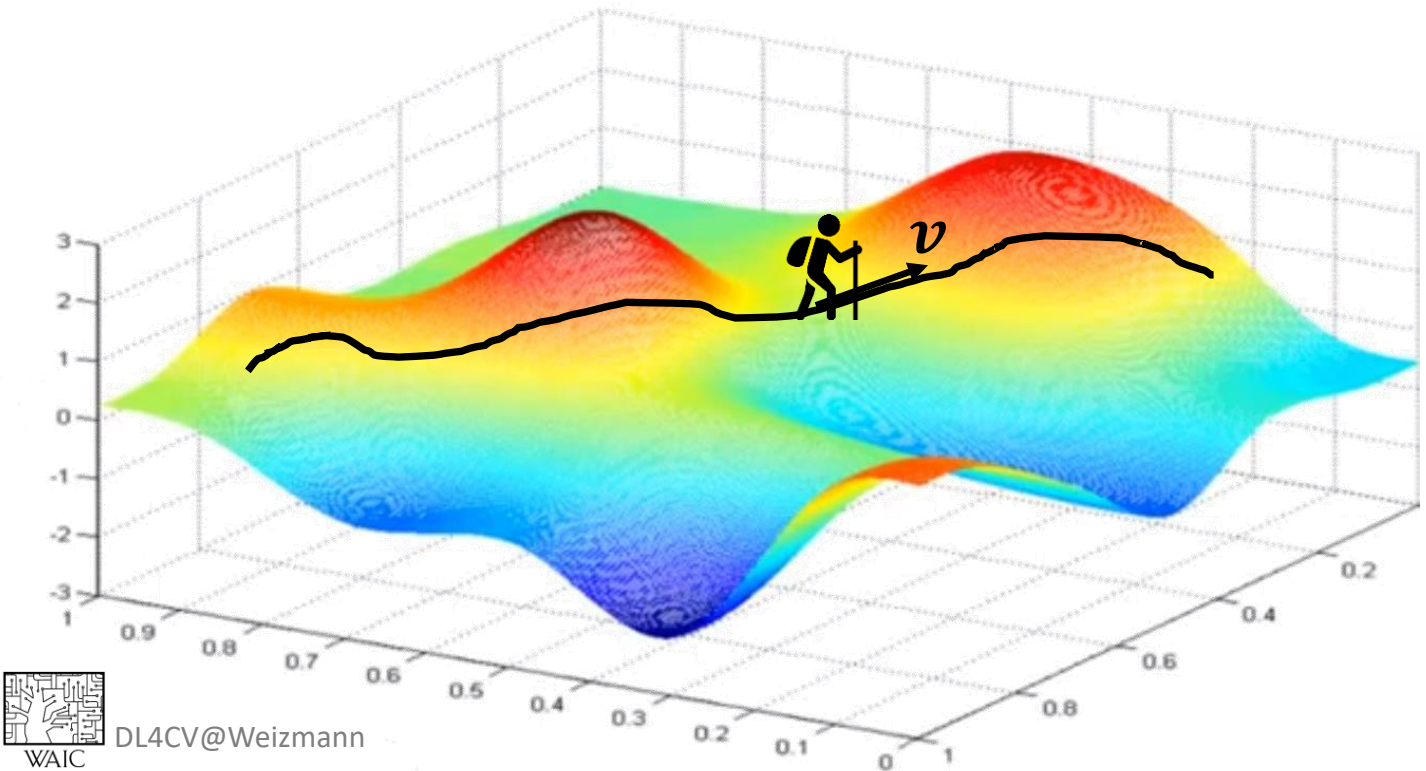
Calculus reminder: Directional derivative



Calculus reminder: Directional derivative



Calculus reminder: Directional derivative

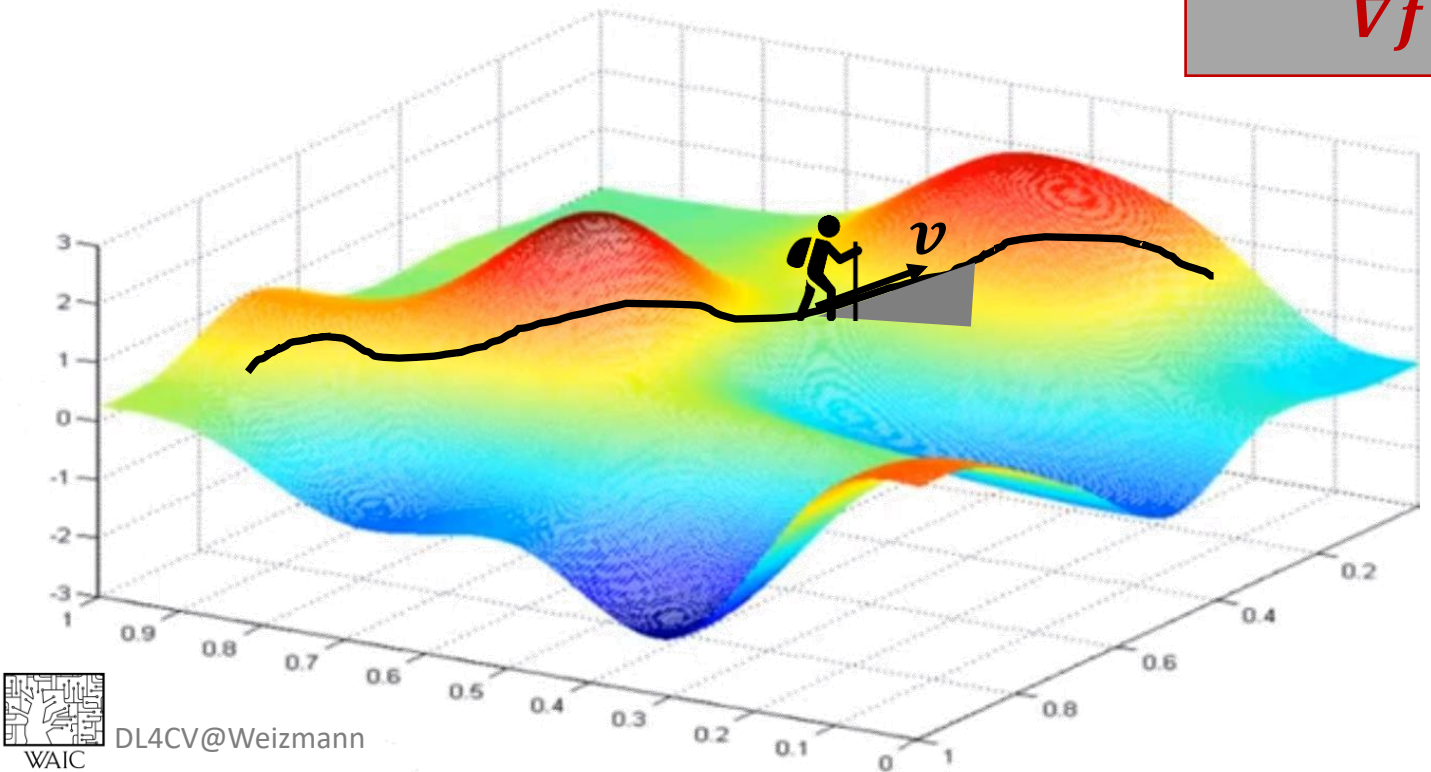


Calculus reminder: Directional derivative

$$\lim_{\varepsilon \to 0} \frac{f(\boldsymbol{x} + \varepsilon \boldsymbol{v}) - f(\boldsymbol{x})}{\varepsilon \|\boldsymbol{v}\|} = \frac{1}{\|\boldsymbol{v}\|} \sum v_i \frac{\partial f}{\partial x_i} = \frac{\boldsymbol{v}^T}{\|\boldsymbol{v}\|} \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{pmatrix} = \frac{1}{\|\boldsymbol{v}\|} \langle \boldsymbol{v}, \nabla f \rangle$$

If differentiable

Gradient!
$\vec{\nabla} f$



According to Cauchy-Schwarz inequality:

- Max value is $\|\nabla f\|$
- Obtained when $\boldsymbol{v}$ is parallel to ∇f

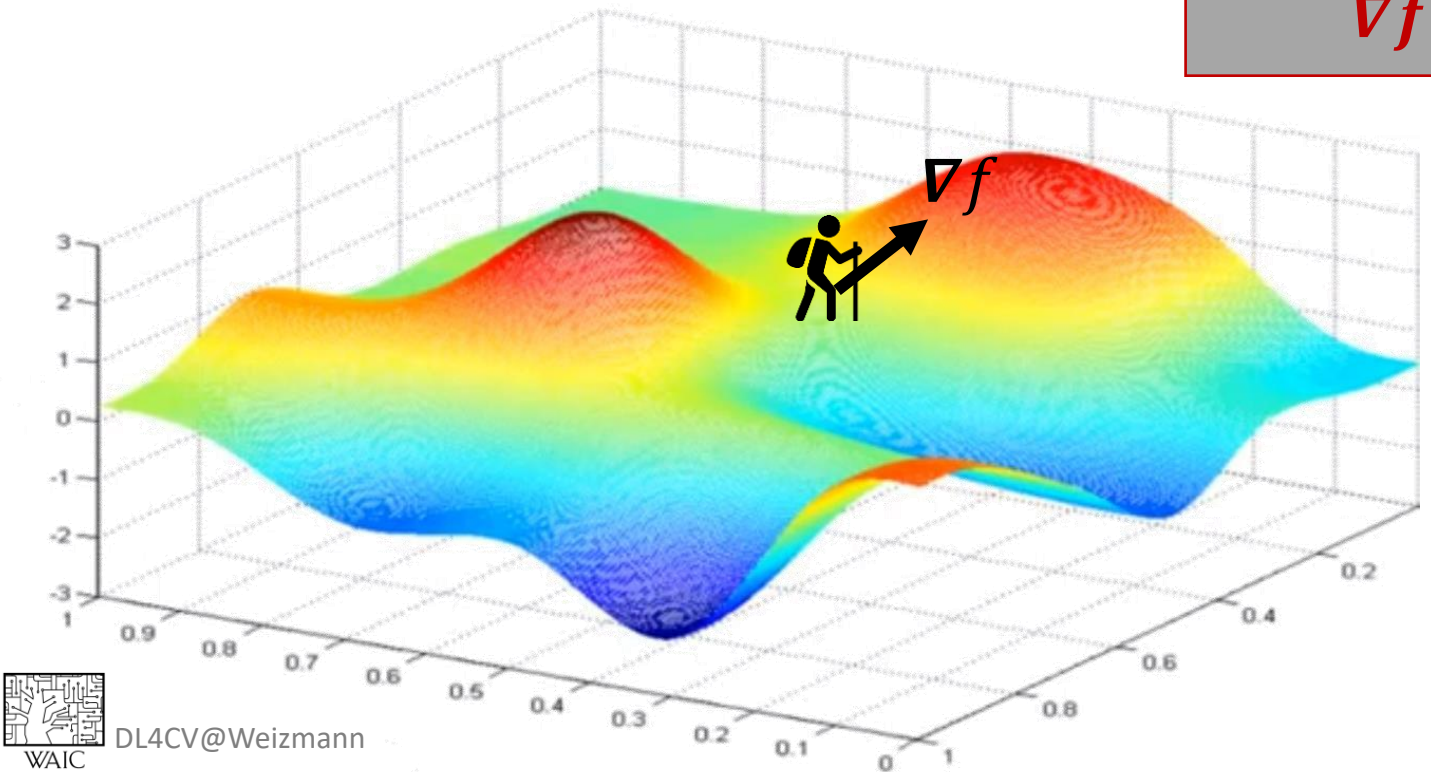
- Gradient directs to steepest ascent.
- It's size is the max steepness.

Calculus reminder: Directional derivative

$$\lim_{\varepsilon \to 0} \frac{f(\boldsymbol{x} + \varepsilon \boldsymbol{v}) - f(\boldsymbol{x})}{\varepsilon \|\boldsymbol{v}\|} = \frac{1}{\|\boldsymbol{v}\|} \sum v_i \frac{\partial f}{\partial x_i} = \frac{\boldsymbol{v}^T}{\|\boldsymbol{v}\|} \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{pmatrix} = \frac{1}{\|\boldsymbol{v}\|} \langle \boldsymbol{v}, \nabla f \rangle$$

If differentiable

Gradient!
$\vec{\nabla} f$



<u>According to Cauchy-Schwarz inequality</u>:

- Max value is $\|\nabla f\|$
- Obtained when $\boldsymbol{v}$ is parallel to ∇f

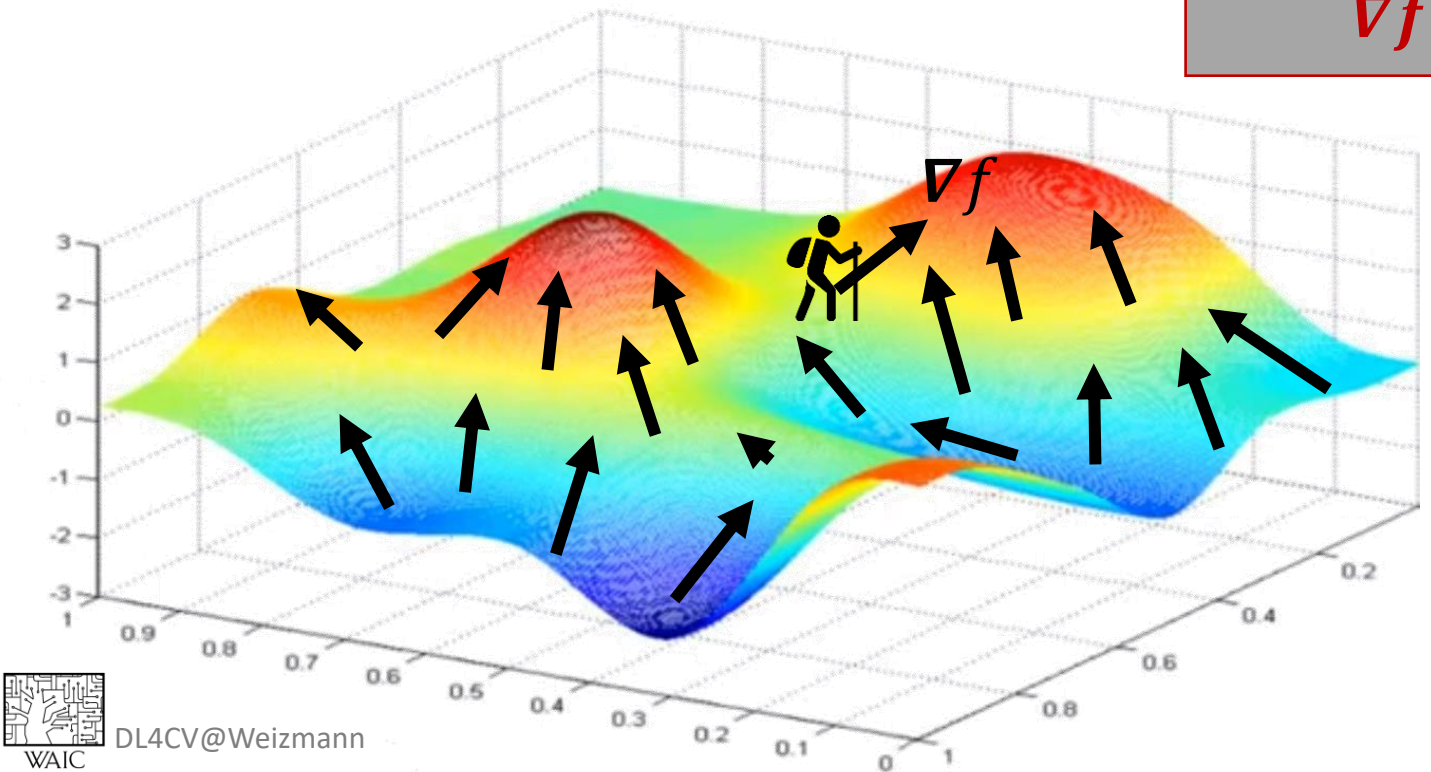
- Gradient directs to steepest ascent.
- It's size is the max steepness.

Calculus reminder: Directional derivative

$$\lim_{\varepsilon \to 0} \frac{f(\boldsymbol{x} + \varepsilon \boldsymbol{v}) - f(\boldsymbol{x})}{\varepsilon \|\boldsymbol{v}\|} = \frac{1}{\|\boldsymbol{v}\|} \sum v_i \frac{\partial f}{\partial x_i} = \frac{\boldsymbol{v}^T}{\|\boldsymbol{v}\|} \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{pmatrix} = \frac{1}{\|\boldsymbol{v}\|} \langle \boldsymbol{v}, \nabla f \rangle$$

If differentiable

Gradient!
$\vec{\nabla} f$



<u>According to Cauchy-Schwarz inequality</u>:

- Max value is $\|\nabla f\|$
- Obtained when $\boldsymbol{v}$ is parallel to ∇f

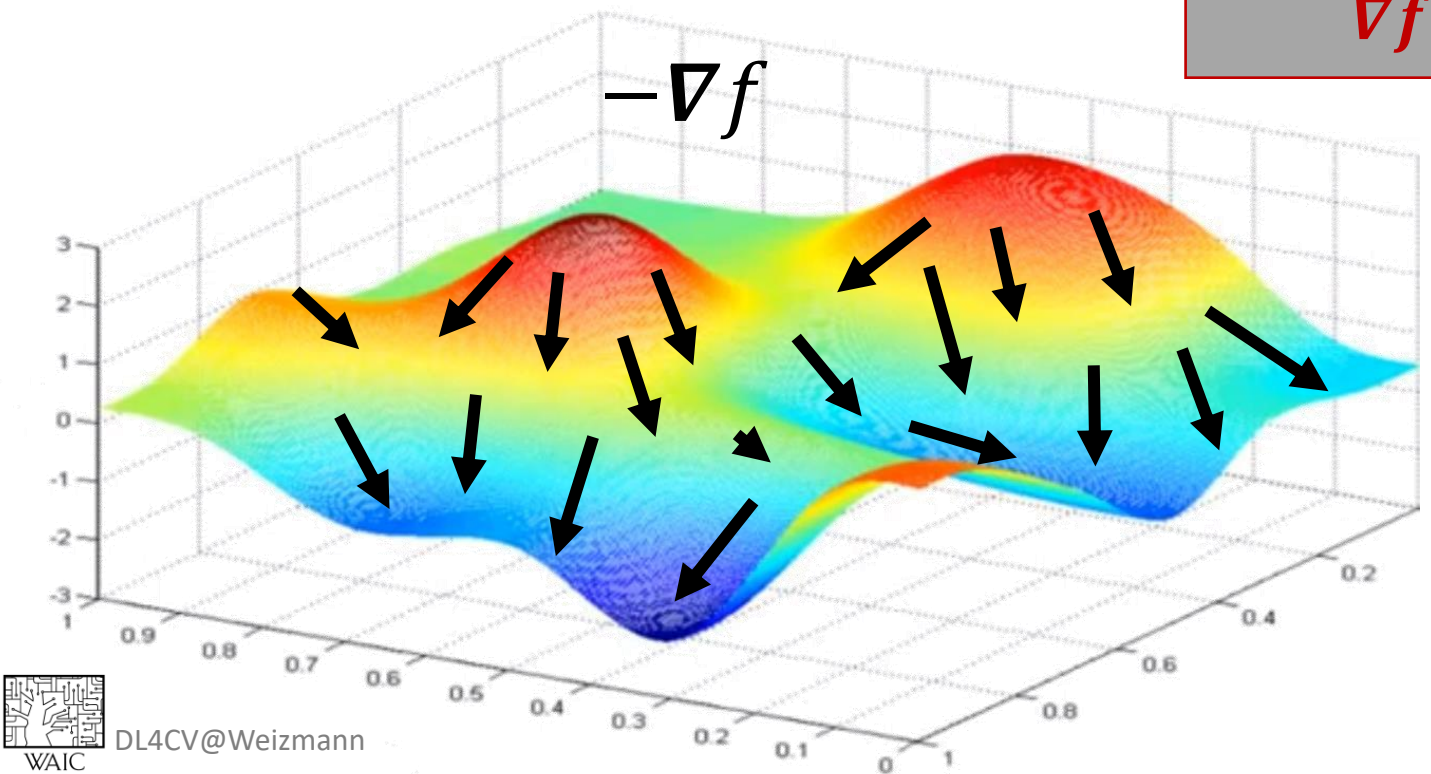
- Gradient directs to steepest ascent.
- It's size is the max steepness.

Calculus reminder: Directional derivative

$$\lim_{\varepsilon \to 0} \frac{f(\boldsymbol{x} + \varepsilon \boldsymbol{v}) - f(\boldsymbol{x})}{\varepsilon \|\boldsymbol{v}\|} = \frac{1}{\|\boldsymbol{v}\|} \sum v_i \frac{\partial f}{\partial x_i} = \frac{\boldsymbol{v}^T}{\|\boldsymbol{v}\|} \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{pmatrix} = \frac{1}{\|\boldsymbol{v}\|} \langle \boldsymbol{v}, \nabla f \rangle$$

If differentiable

Gradient!
$\vec{\nabla} f$



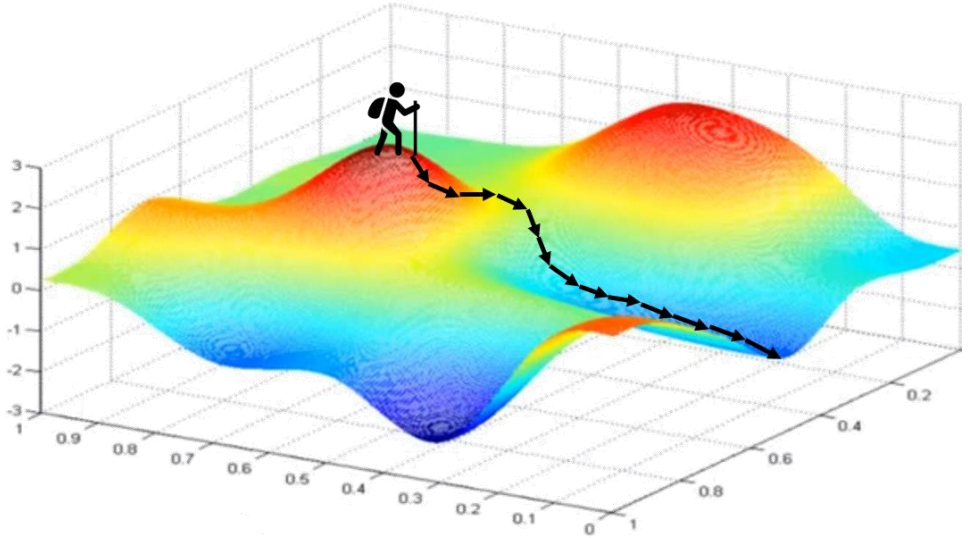
<u>According to Cauchy-Schwarz inequality</u>:

- Max value is $\|\nabla f\|$
- Obtained when $\boldsymbol{v}$ is parallel to ∇f

- Gradient directs to steepest ascent.
- It's size is the max steepness.

Gradient descent

$$\nabla\mathcal{L}(\theta_0, \theta_1 \dots \theta_N) = \begin{pmatrix} \frac{\partial\mathcal{L}}{\partial\theta_0} \\ \frac{\partial\mathcal{L}}{\partial\theta_1} \\ \vdots \\ \frac{\partial\mathcal{L}}{\partial\theta_N} \end{pmatrix}$$





Augustin Louis Cauchy

1. Initialize $\theta \sim$ Random
2. Repeat until convergence:
   {
   $$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha\nabla\mathcal{L}(\boldsymbol{\theta}; S)$$
   }

α: Learning rate

Gradient descent



Full batch Gradient Descent

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \boldsymbol{\nabla} \mathcal{L}(\boldsymbol{\theta}; S)$$

Gradient descent



Full batch Gradient Descent

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \boldsymbol{\nabla} \mathcal{L}(\boldsymbol{\theta}; S)$$

Stochastic Gradient Descent

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \boldsymbol{\nabla} \mathcal{L}(\boldsymbol{\theta};\ x_i, y_i)$$

$$i \sim U[1, M]$$

Stochastic Mini-Batch Gradient Descent

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \boldsymbol{\nabla} \mathcal{L}(\boldsymbol{\theta}; \boldsymbol{B})$$

$$\boldsymbol{B} = \{(x_i, y_i)\ ;\quad \boldsymbol{i} \sim U^m[1, M]\}$$

Gradient descent



Full batch Gradient Descent

Stochastic Gradient Descent

Stochastic Mini-Batch Gradient Descent

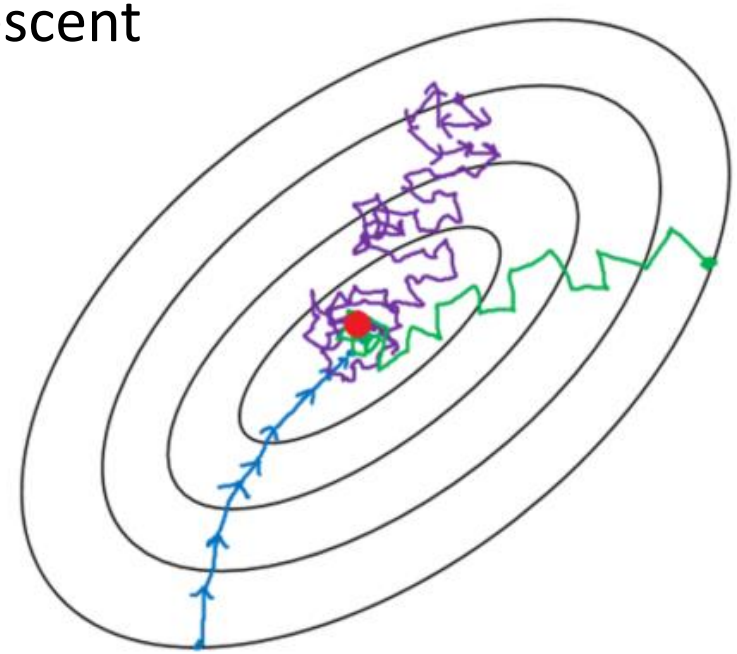


Figure by Z² Little on Medium

Gradient descent for Linear Regression

$$\mathcal{L} = \frac{1}{2m}\sum_{i=1}^{m}(\boldsymbol{\theta}^T \boldsymbol{x}_i - y_i)^2$$

$$\frac{\partial}{\partial \boldsymbol{\theta}}\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{\theta}^T \boldsymbol{x}_i - y_i)\boldsymbol{x}_i = \frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i(\boldsymbol{X\theta} - \boldsymbol{y})_{\boldsymbol{i}} = \boldsymbol{X}^T(\boldsymbol{X\theta} - \boldsymbol{y})$$

$\boldsymbol{e}$

Repeat until convergence:

{

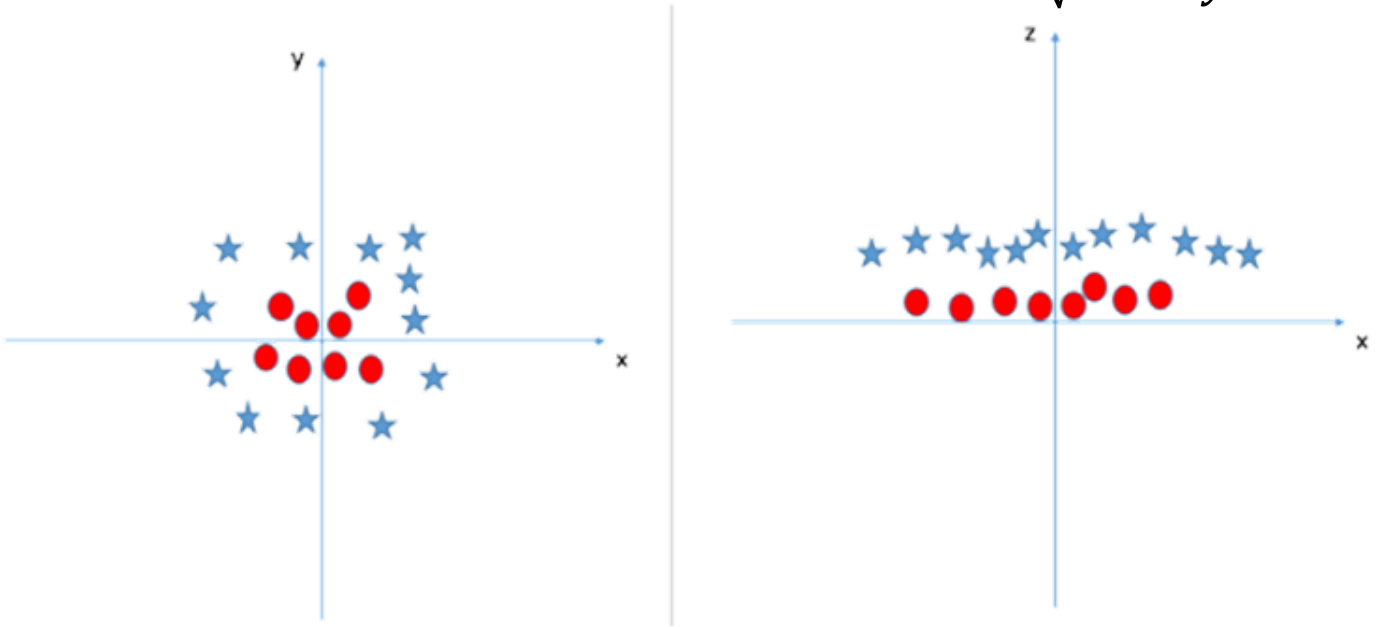
$$\boldsymbol{\theta} := \boldsymbol{\theta} - \frac{\alpha}{m}\boldsymbol{X}^T\boldsymbol{e}$$

}

Q: Find the relation between convergence and Normal Equations

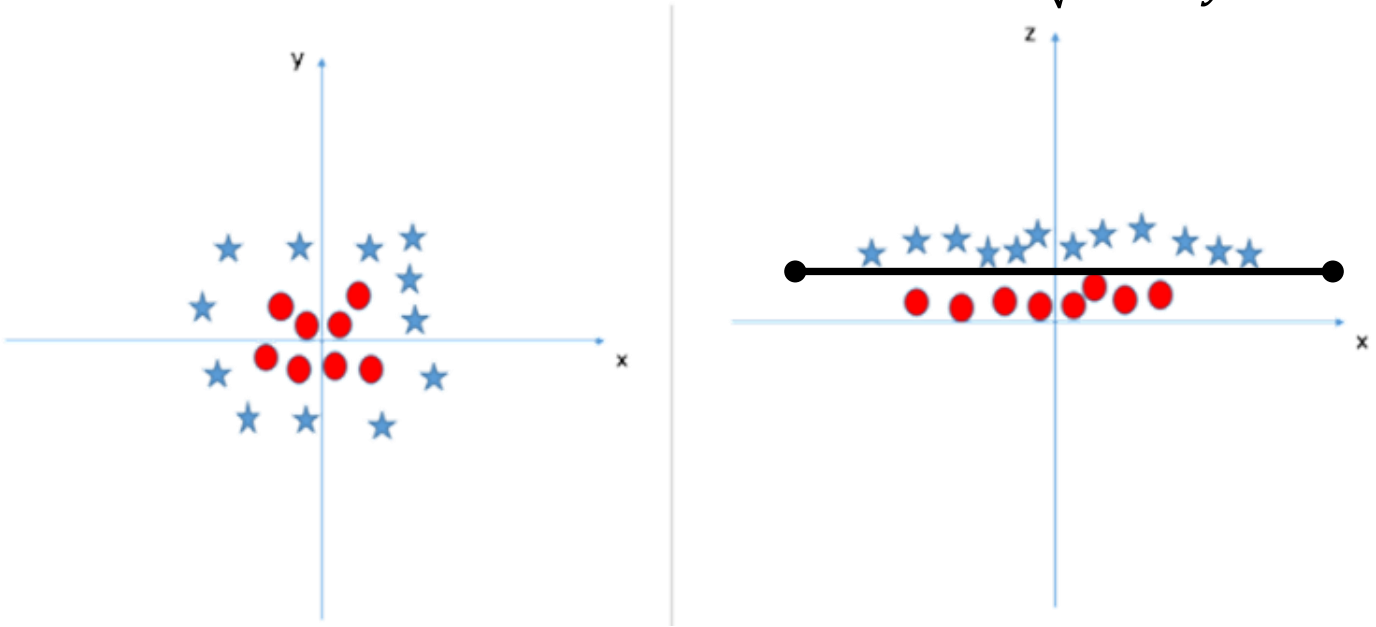
Feature transform

$$z = \sqrt{x^2 + y^2}$$



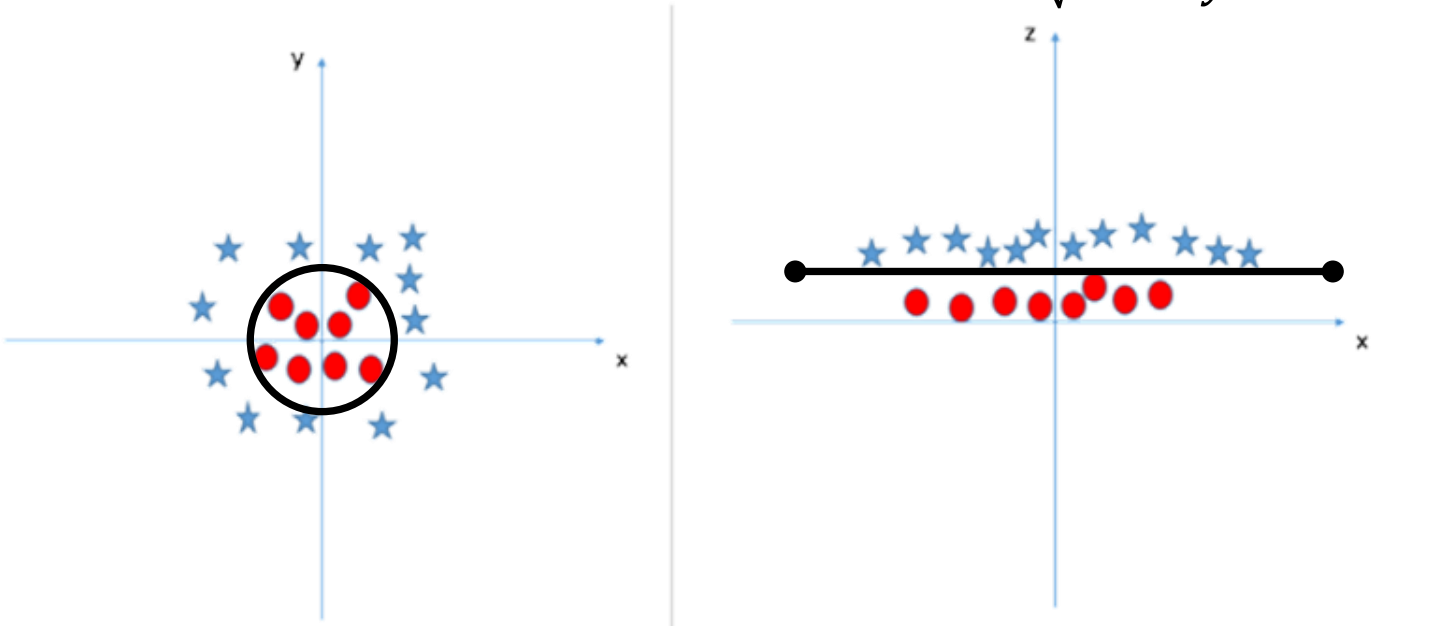
Feature transform

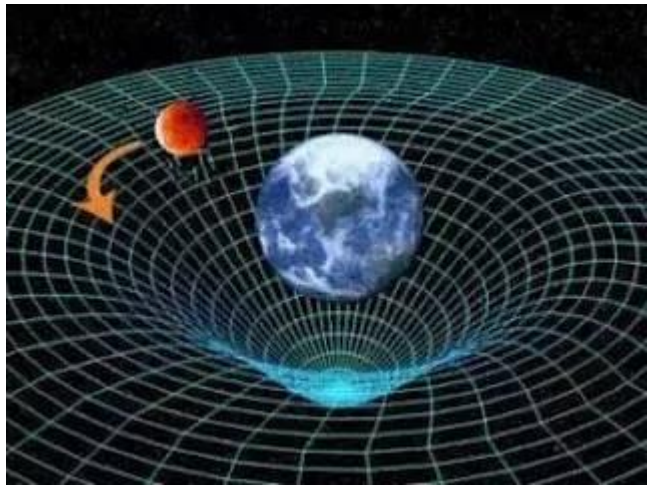
$$z = \sqrt{x^2 + y^2}$$

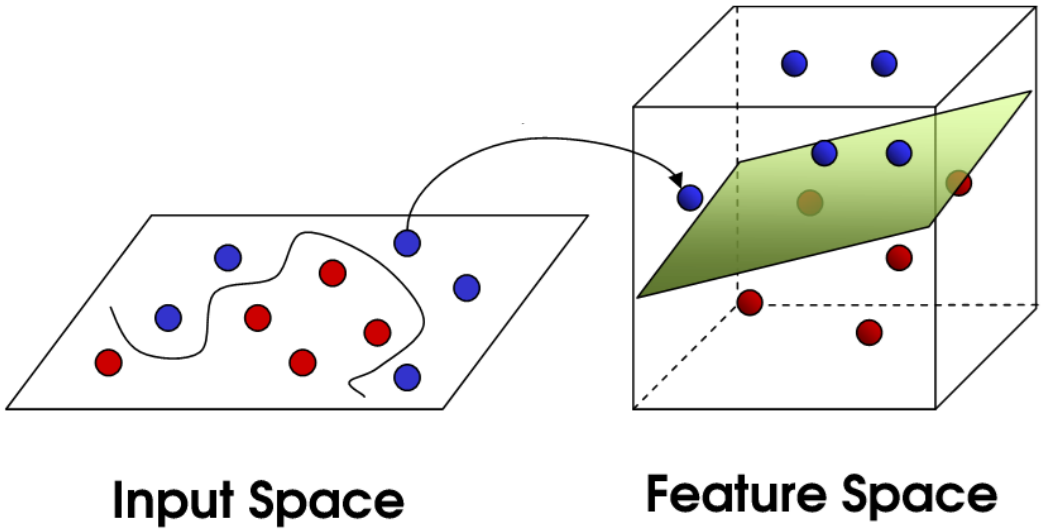


Feature transform

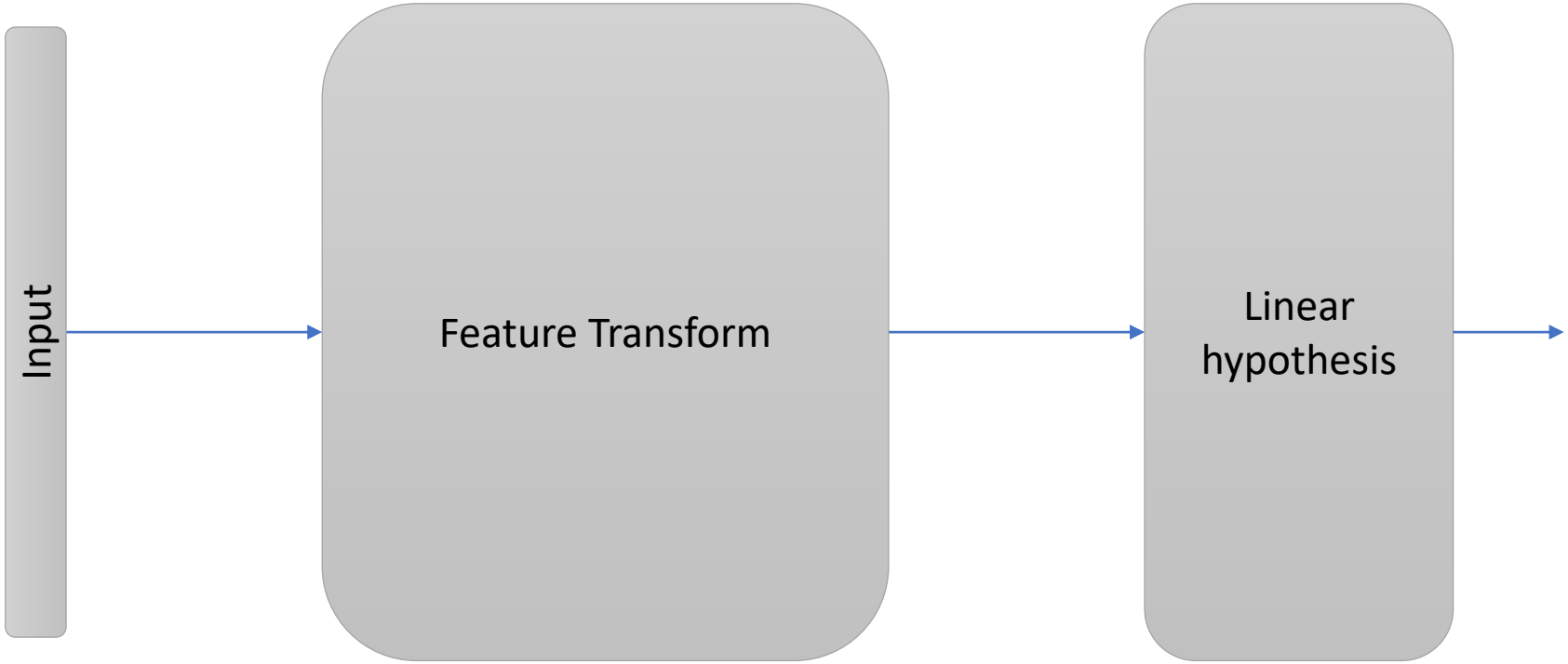
$$z = \sqrt{x^2 + y^2}$$



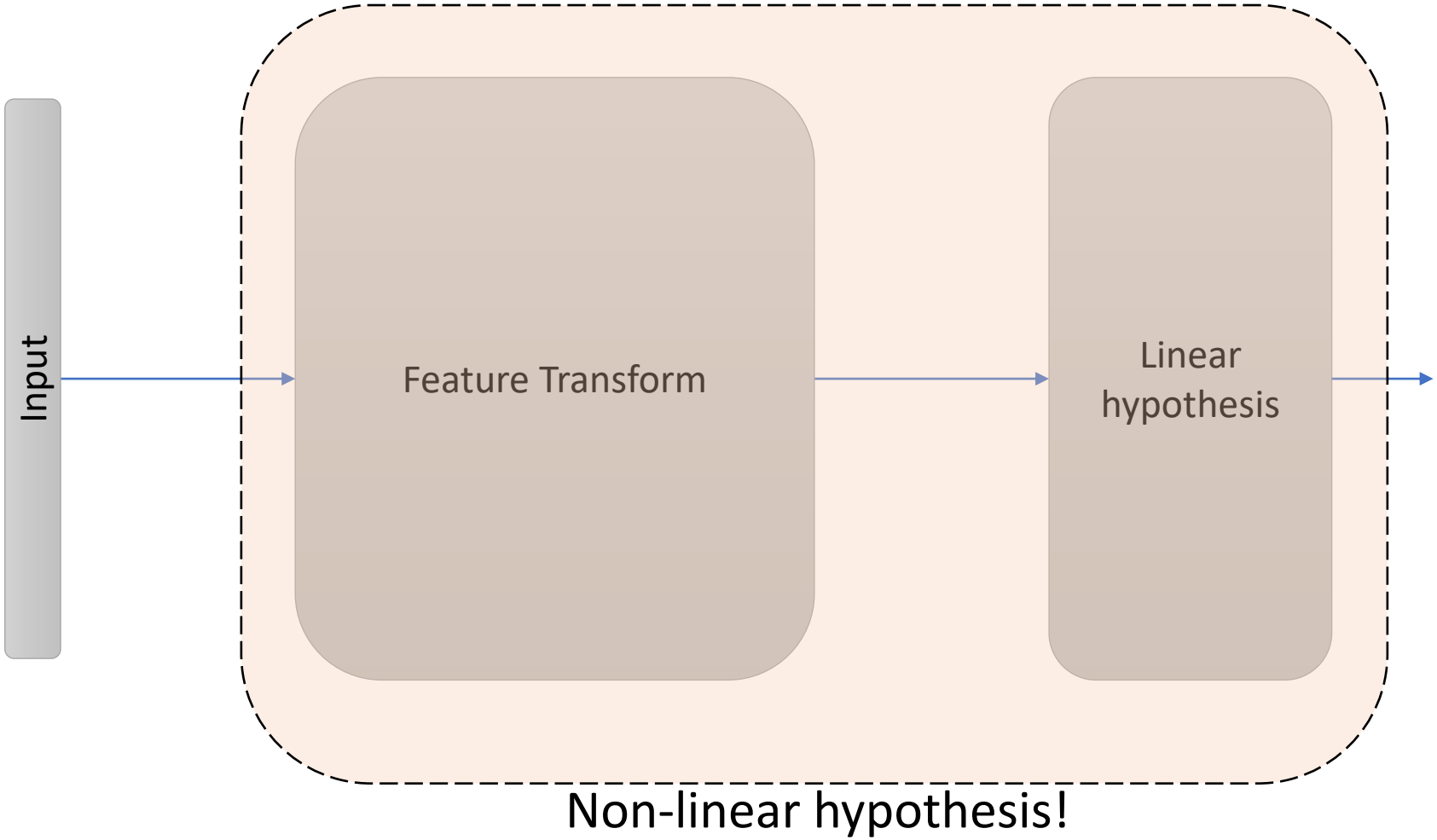




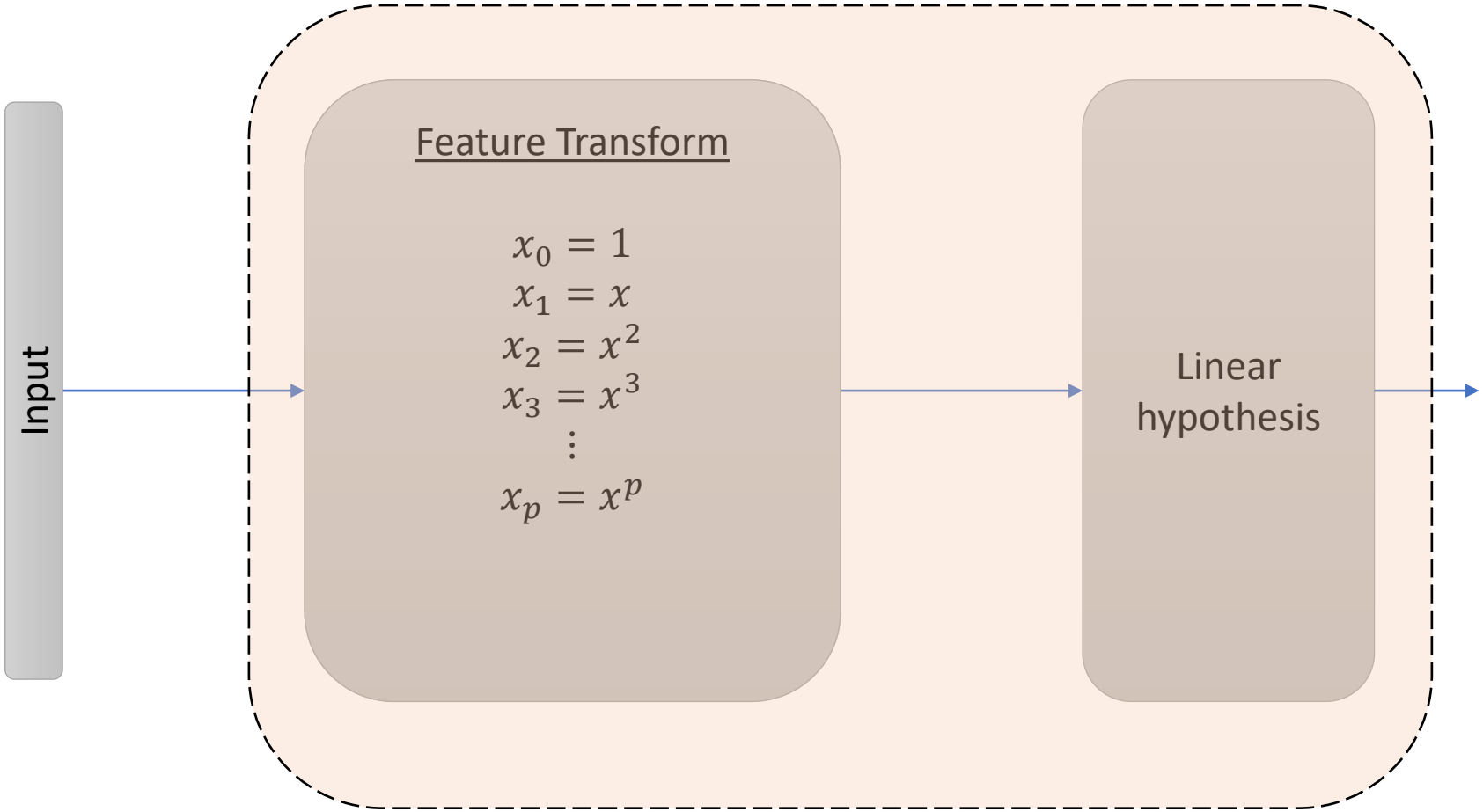
Feature transform



Feature transform

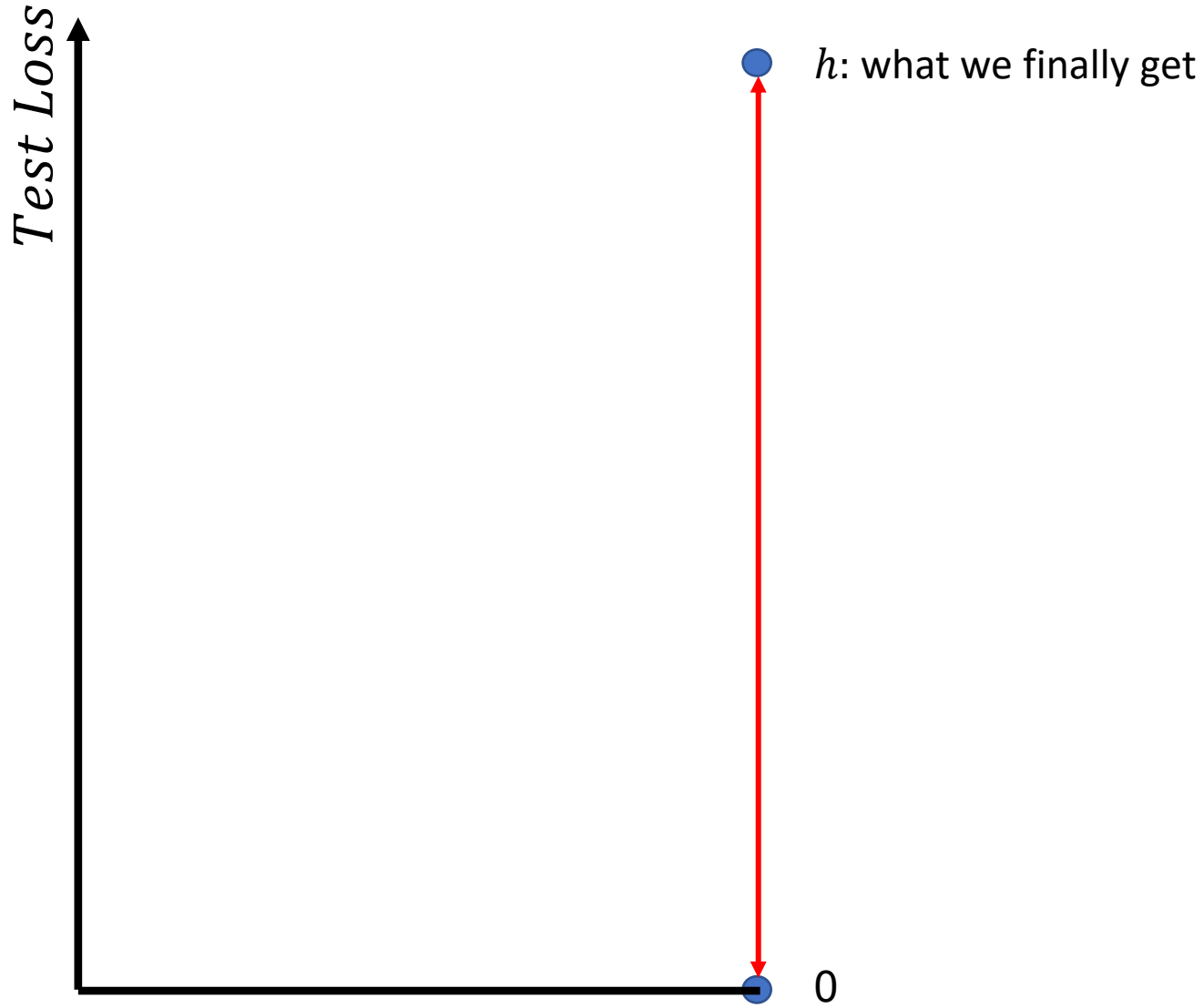


Feature transform

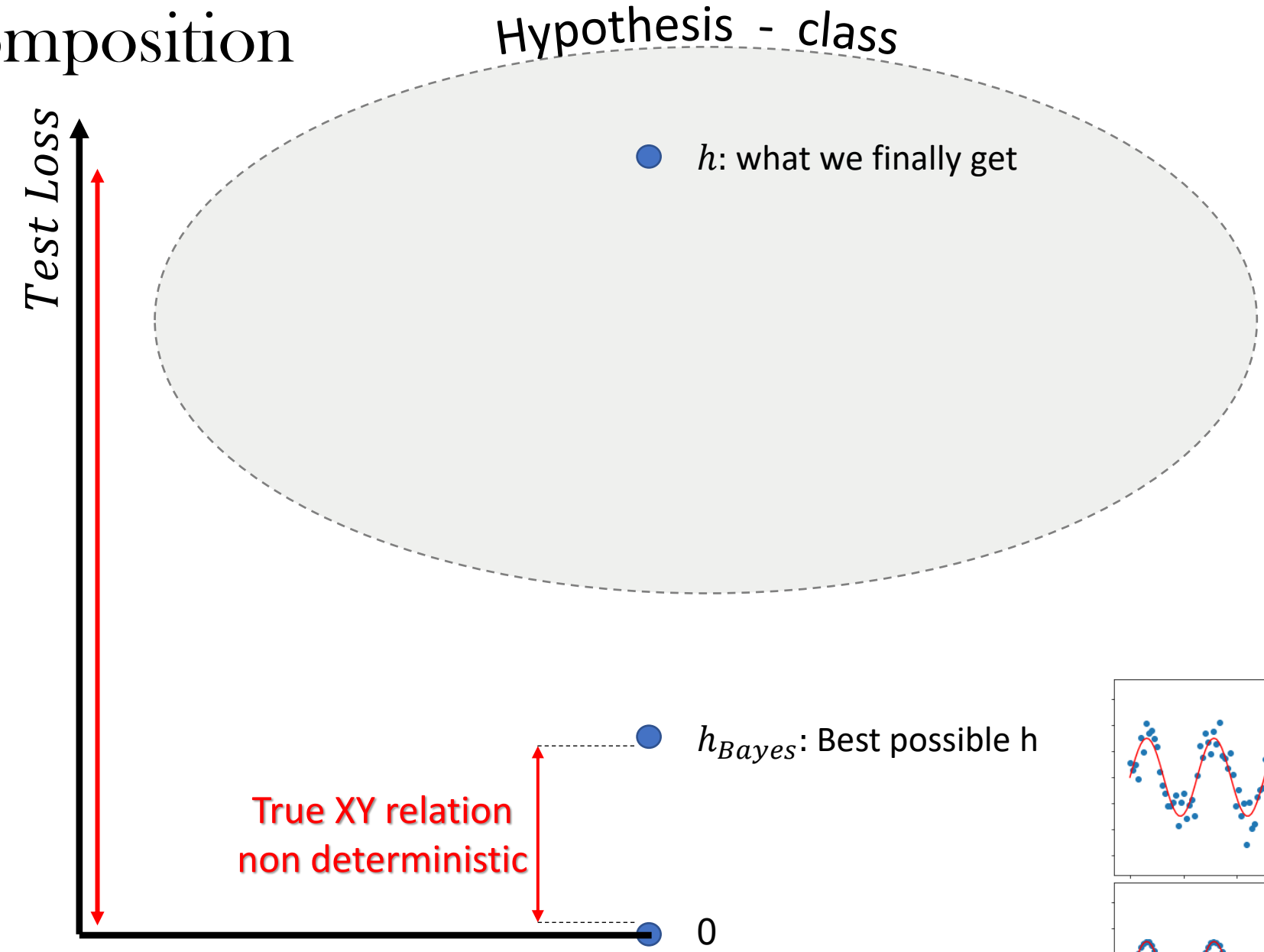


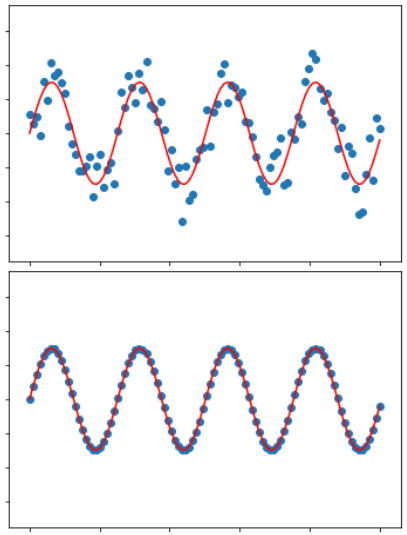
Non-linear hypothesis!
(Polynomial)

Error decomposition

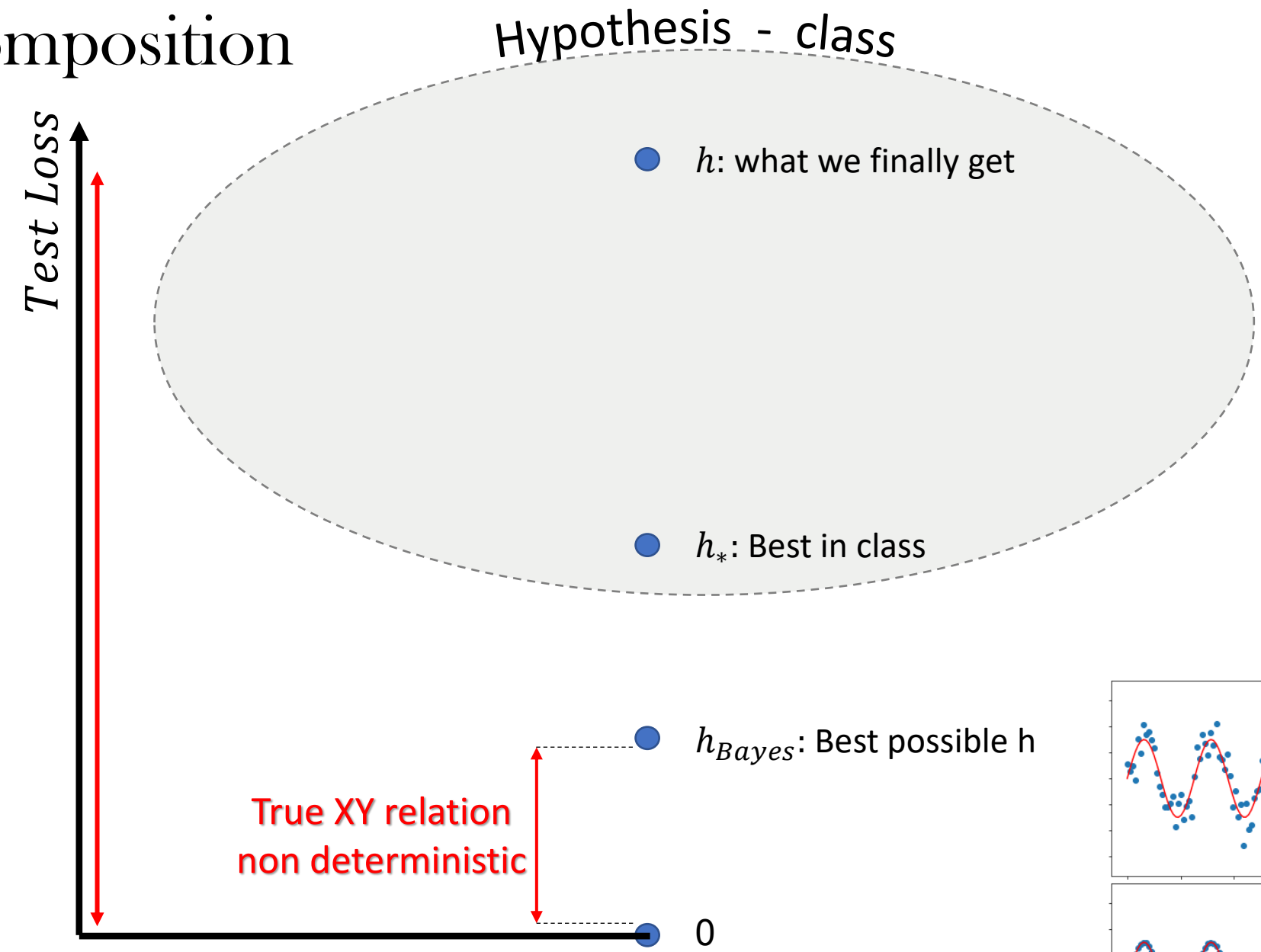


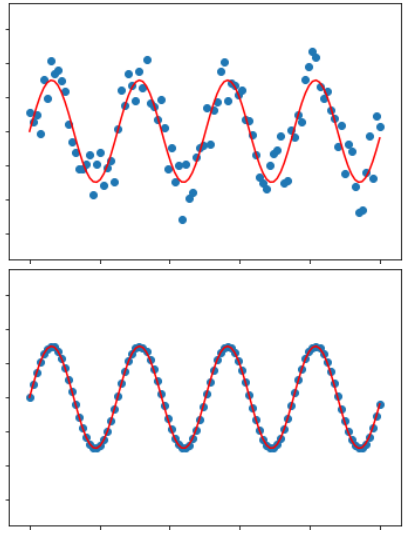
Error decomposition



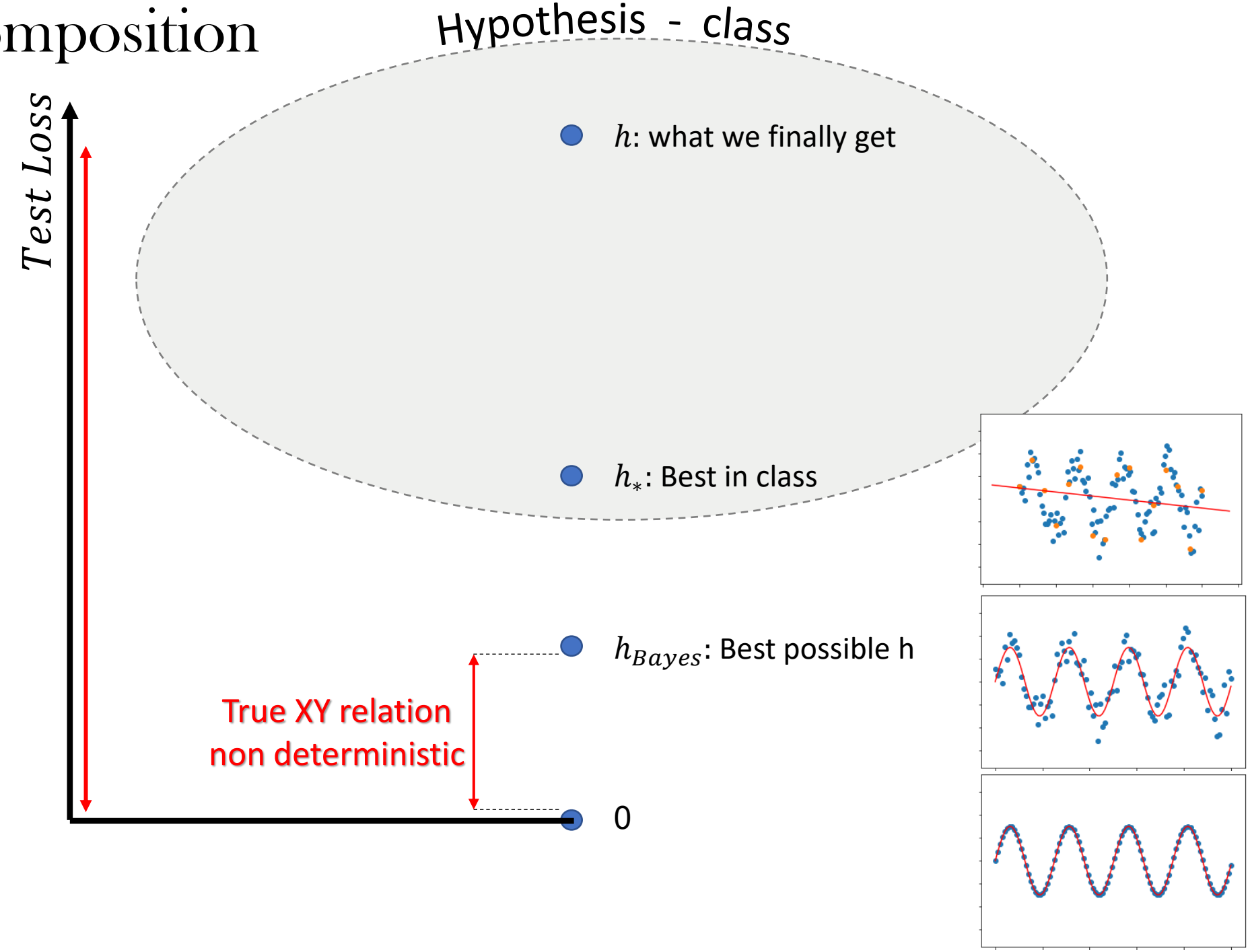


Error decomposition

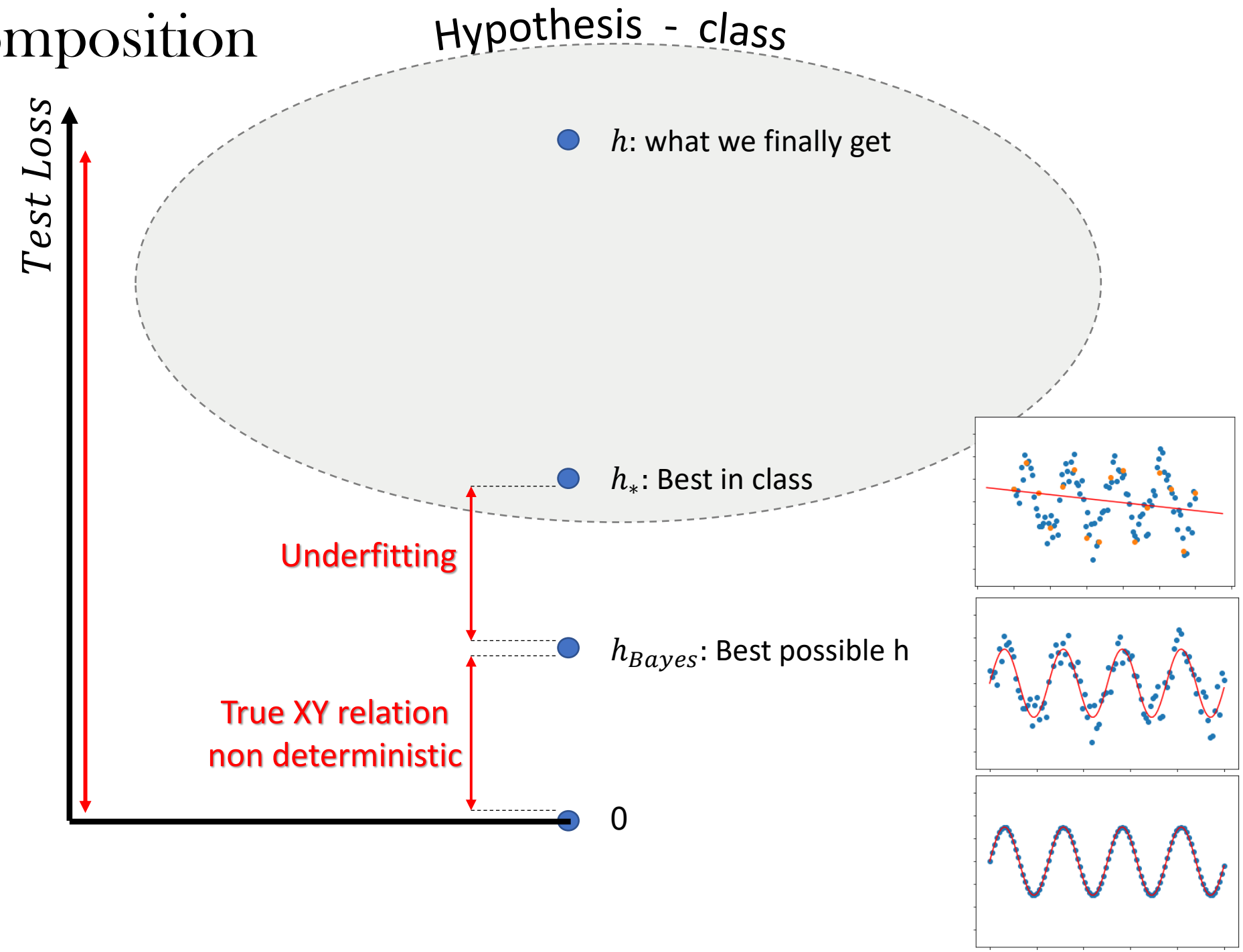




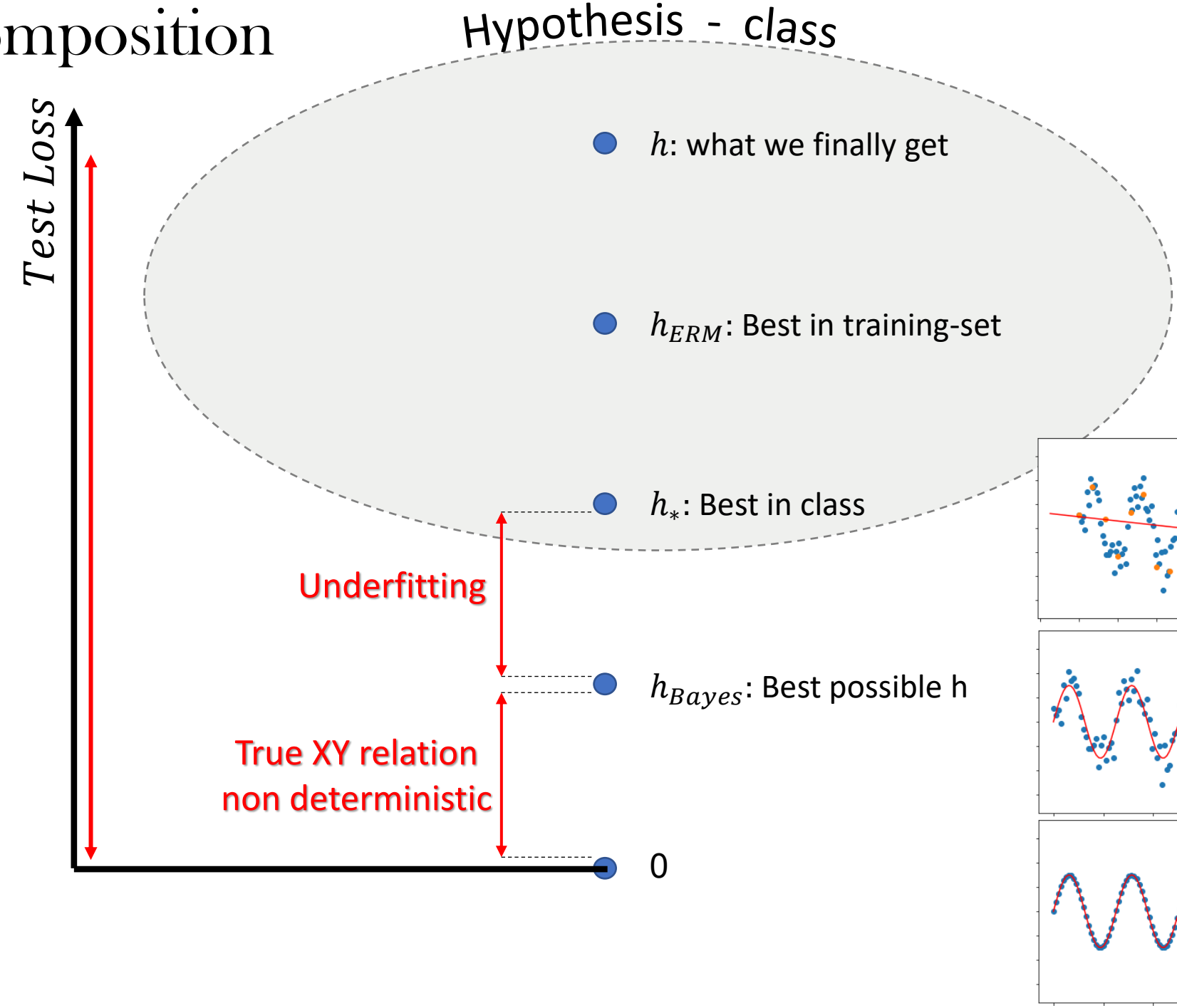
Error decomposition

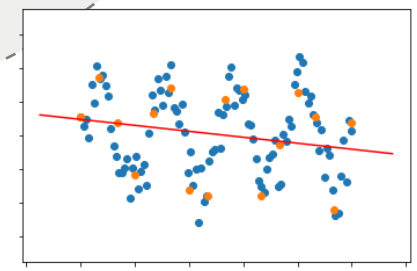


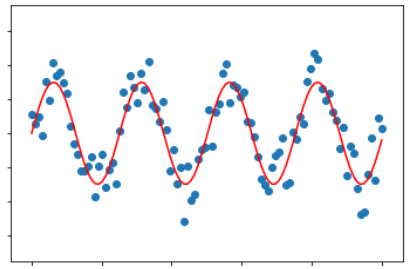
Error decomposition

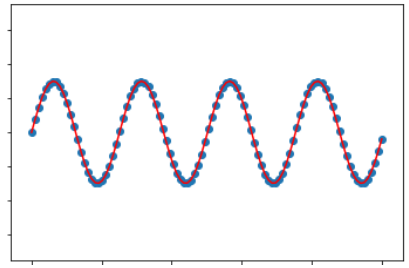


Error decomposition

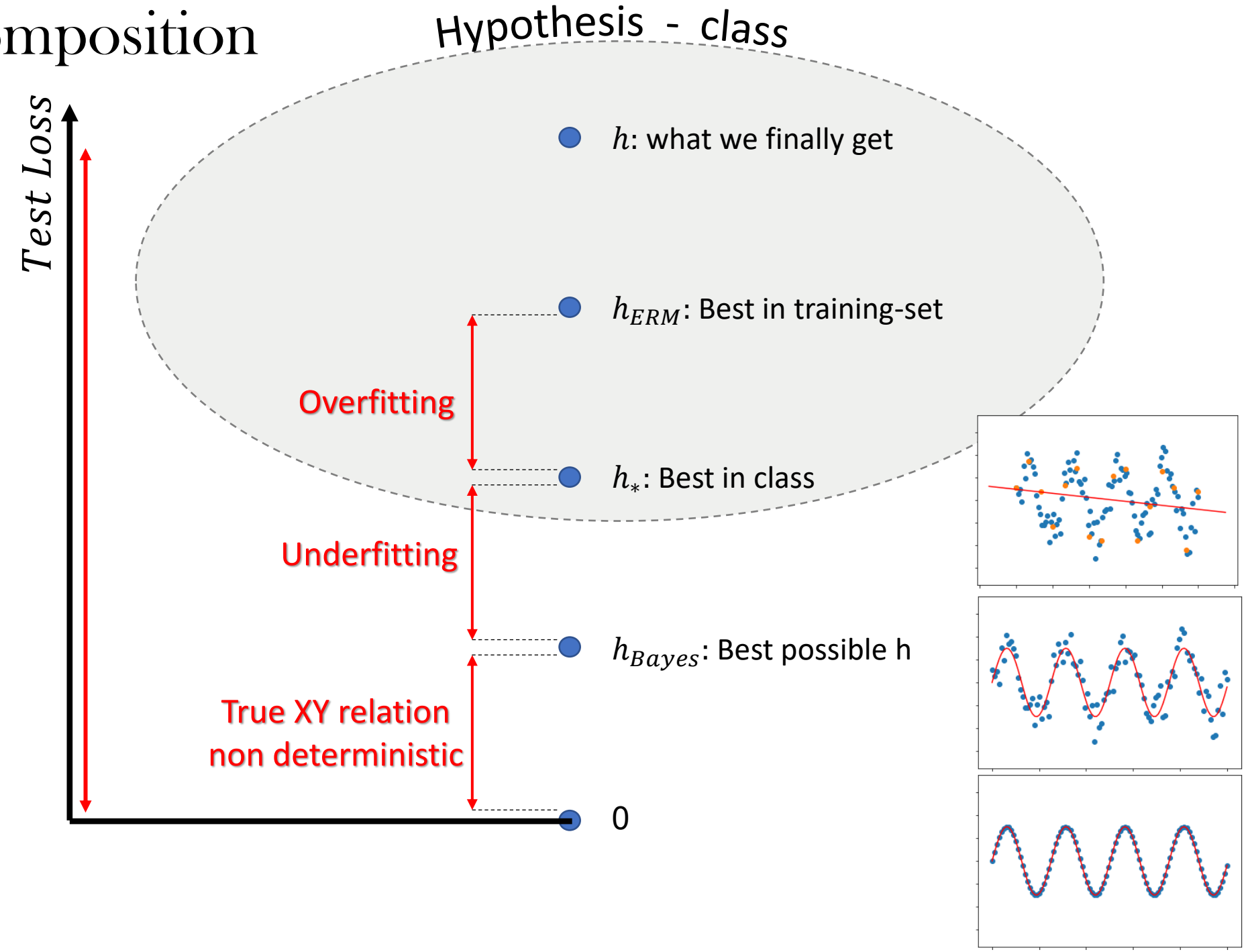




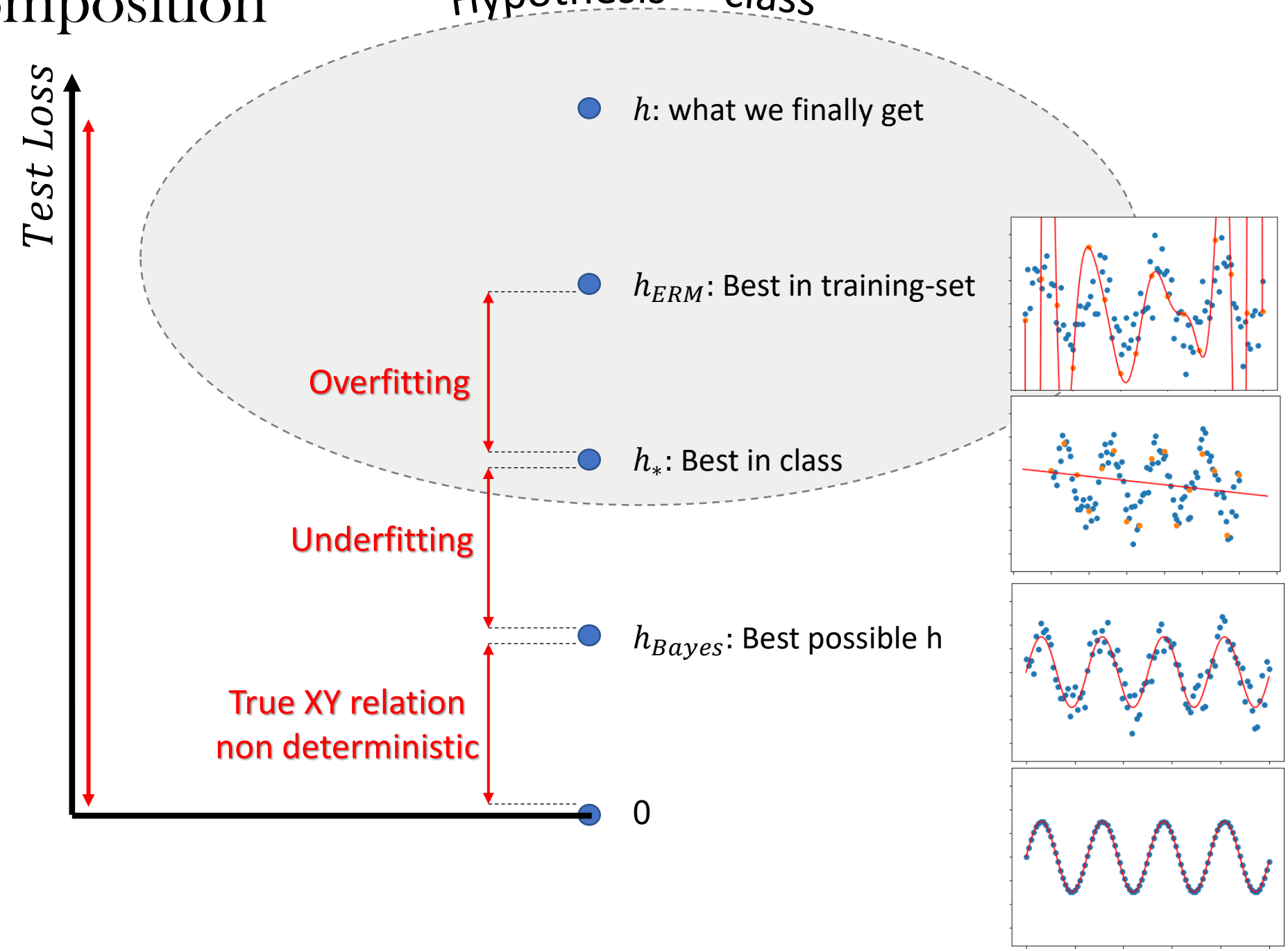




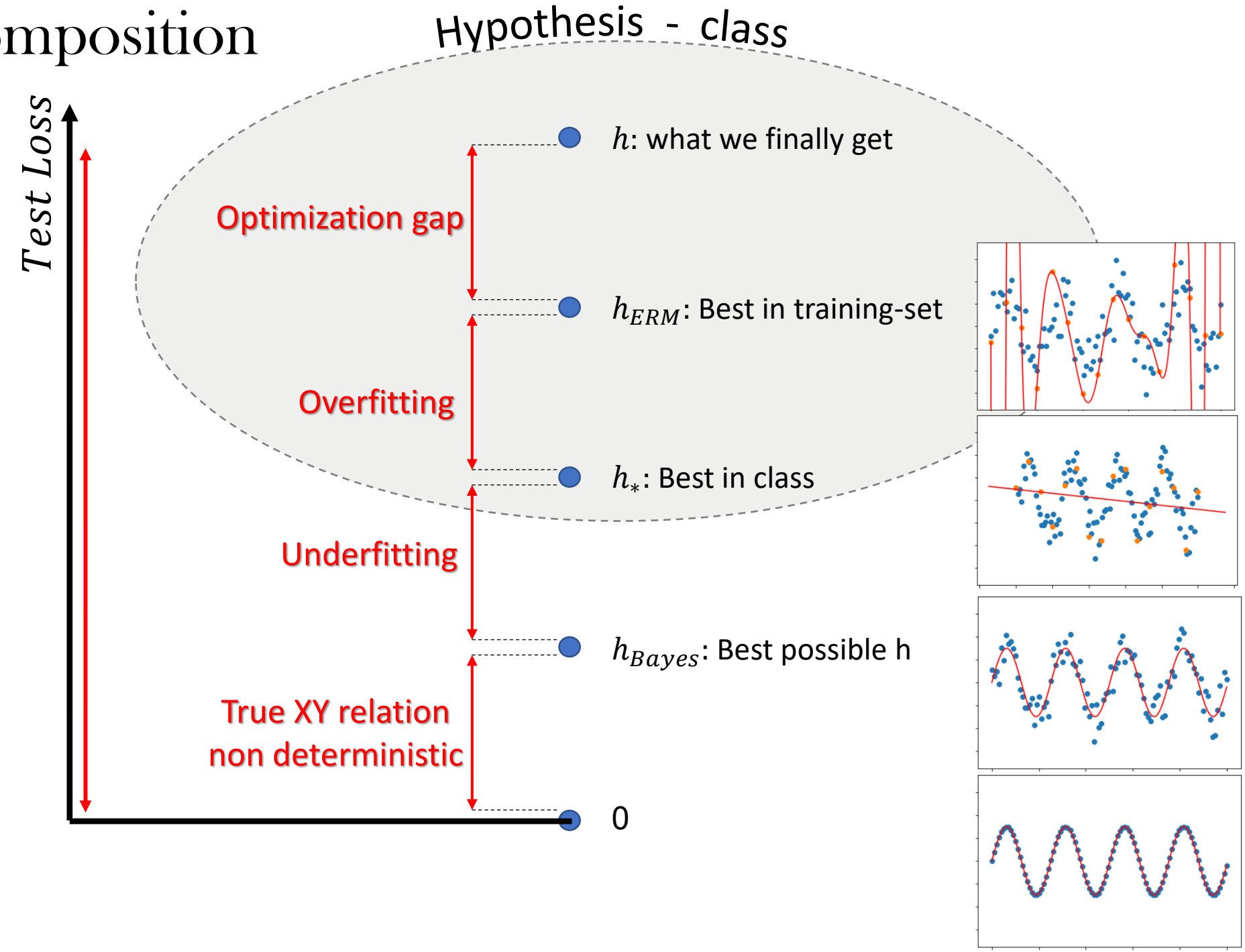
Error decomposition



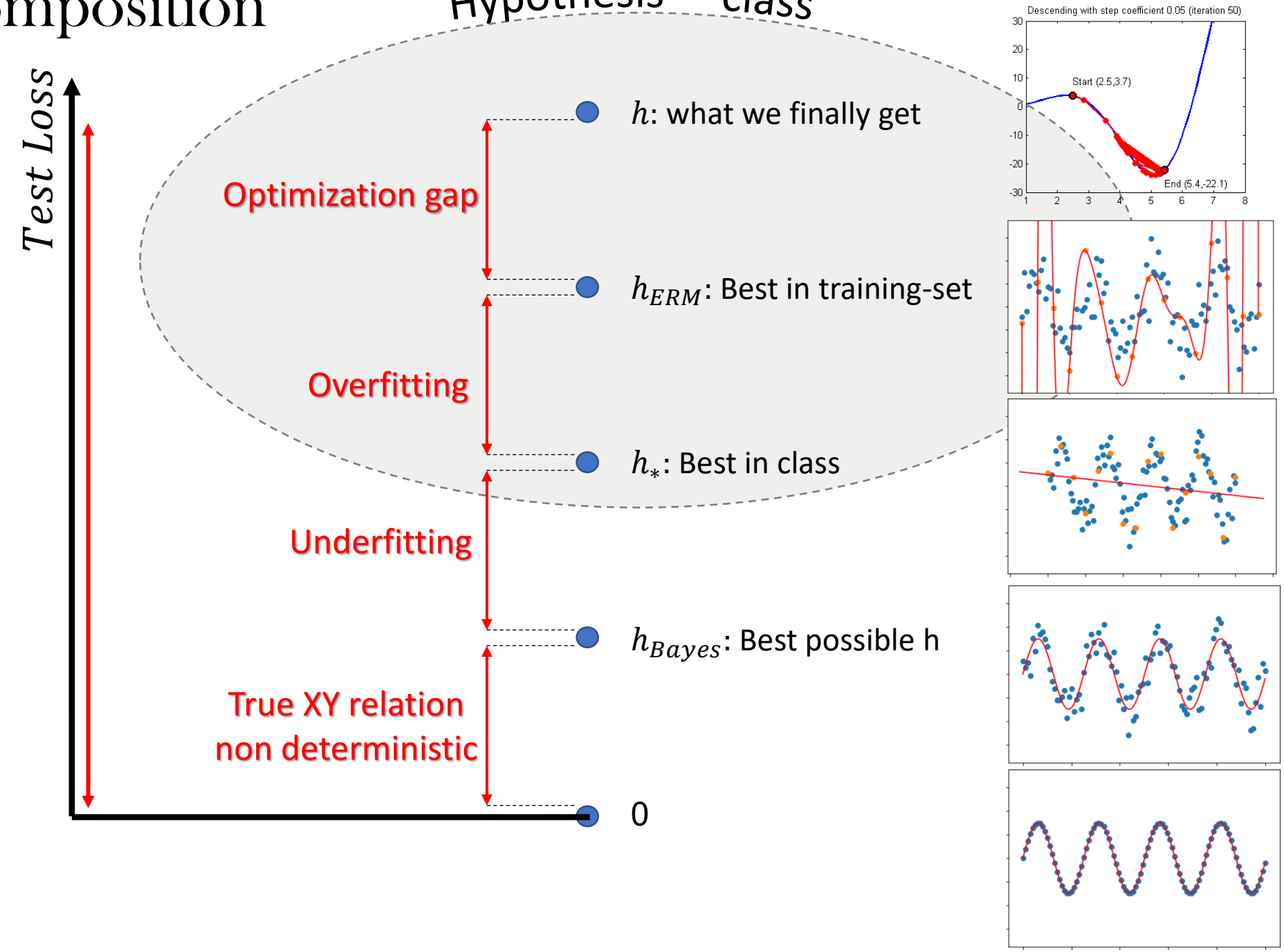
Error decomposition



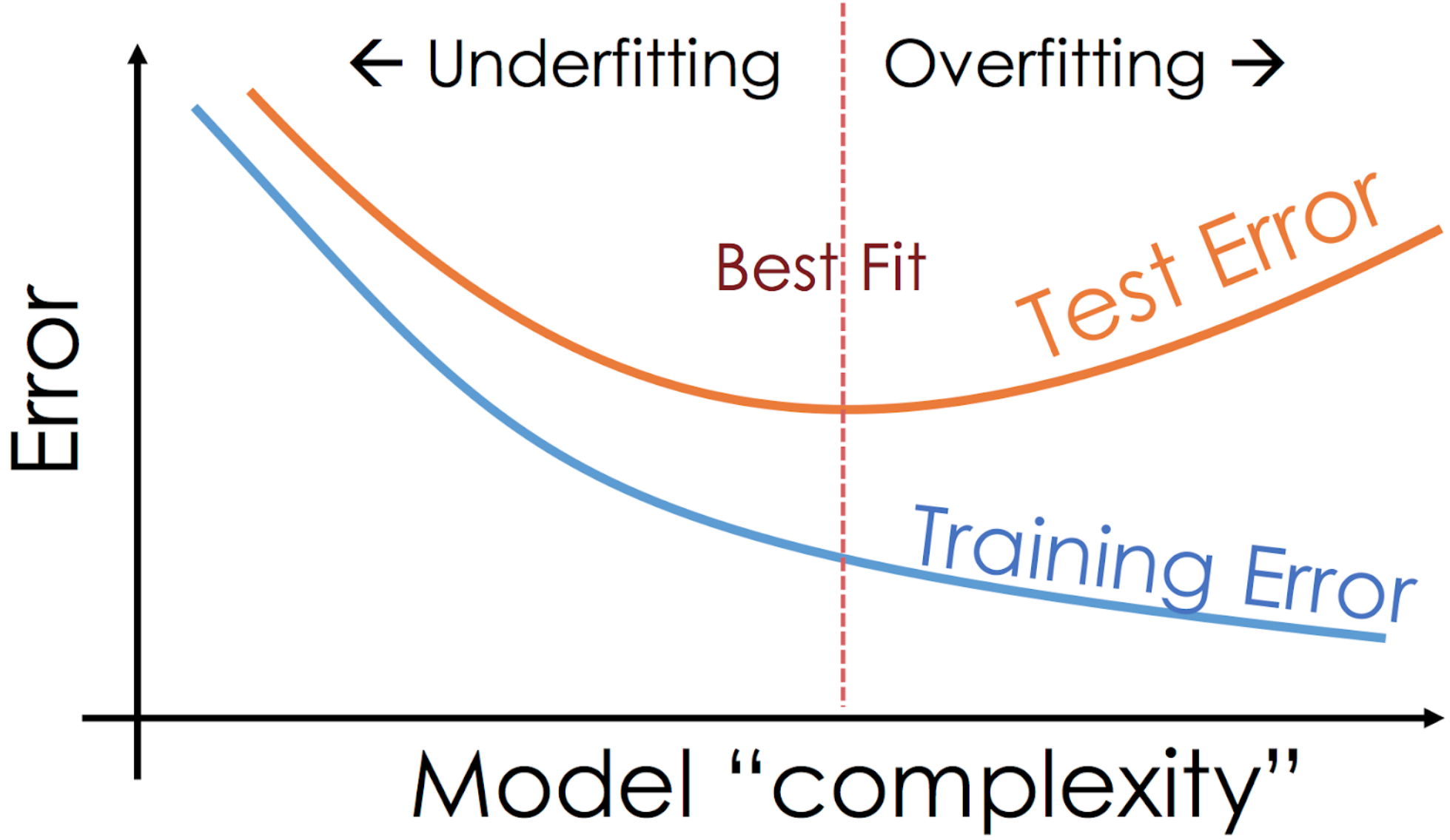
Error decomposition



Error decomposition



Generalization



Overfitting- Data influence



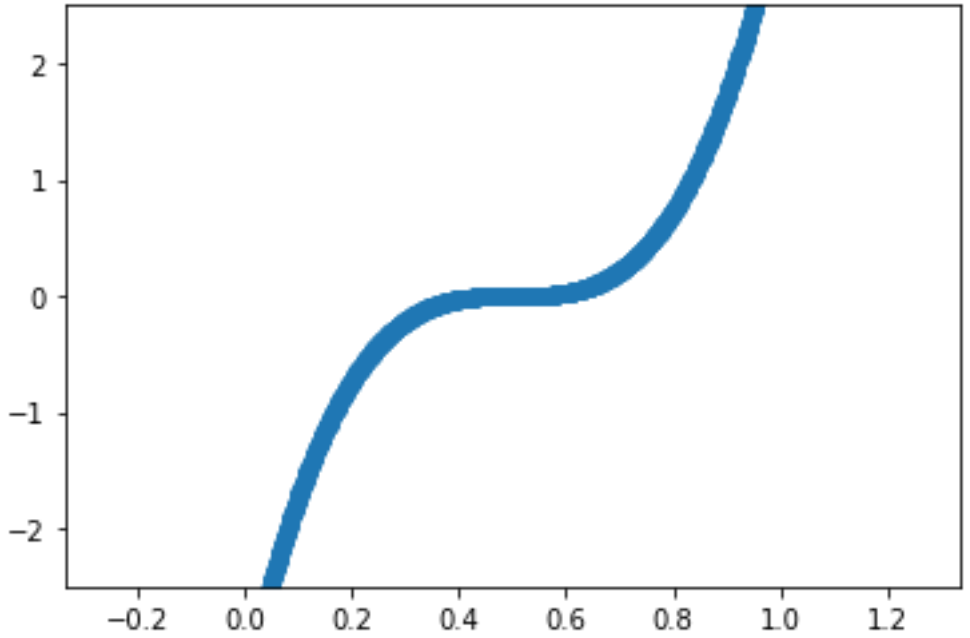
Matrix A

$$\begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}$$

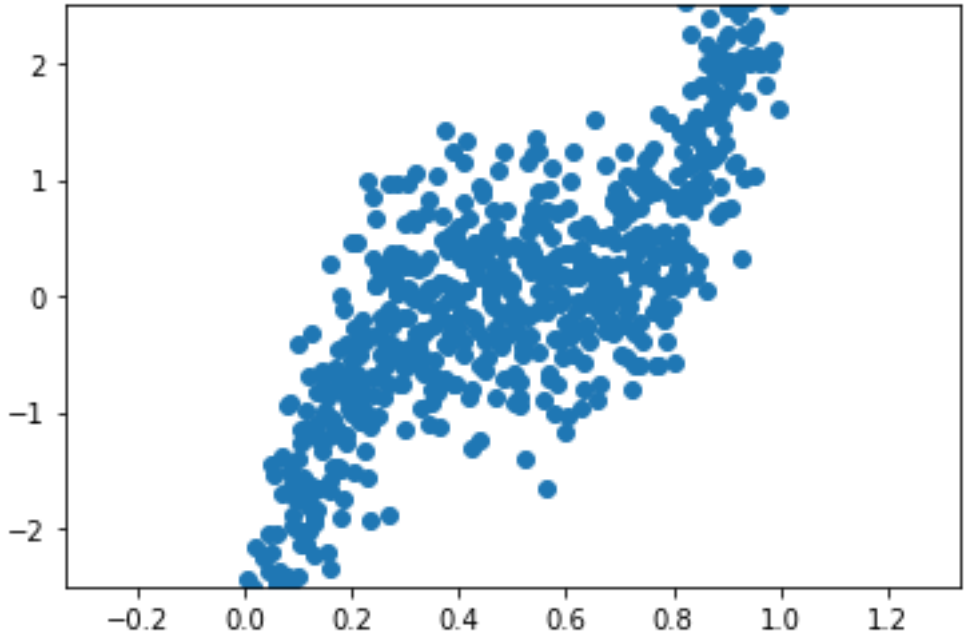
A^2

$$\begin{bmatrix} 33 & 44 \\ 66 & 88 \end{bmatrix}$$

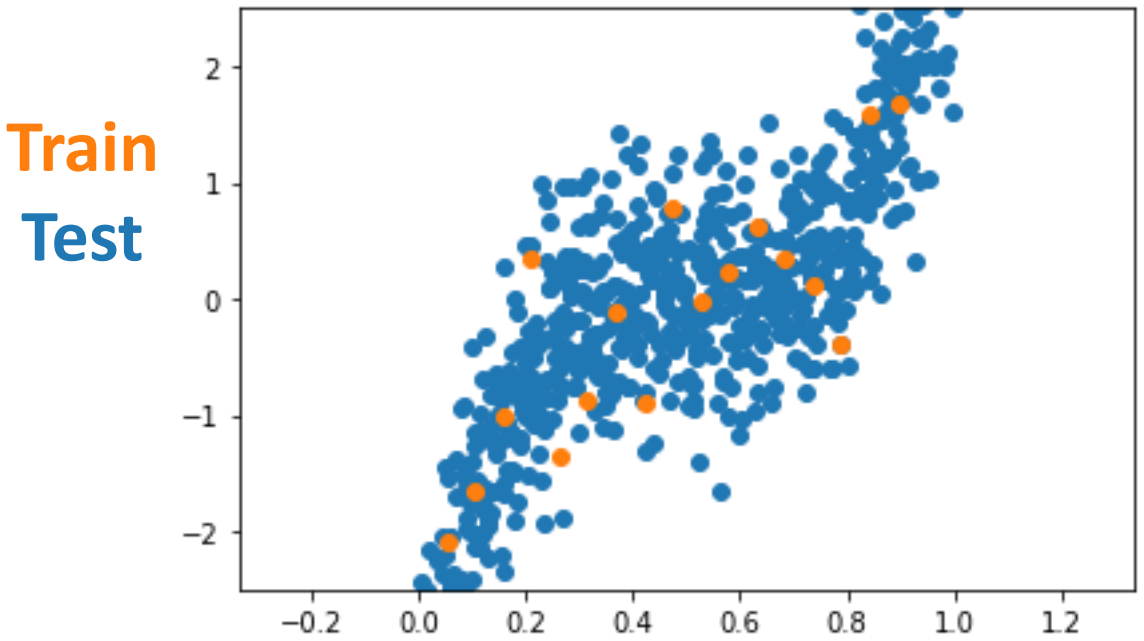
Polynomial fitting



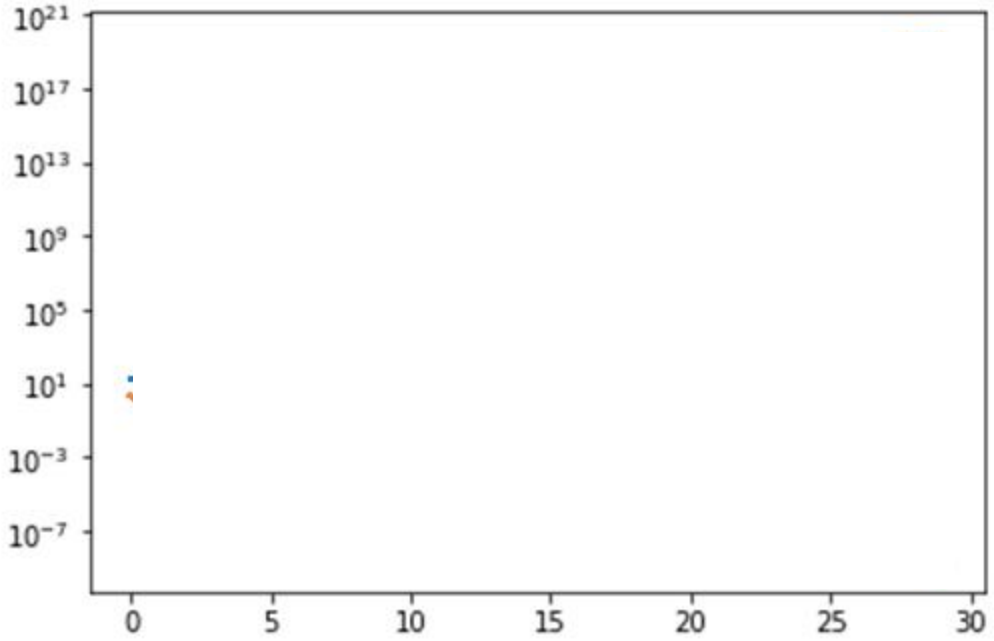
Polynomial fitting



Polynomial fitting

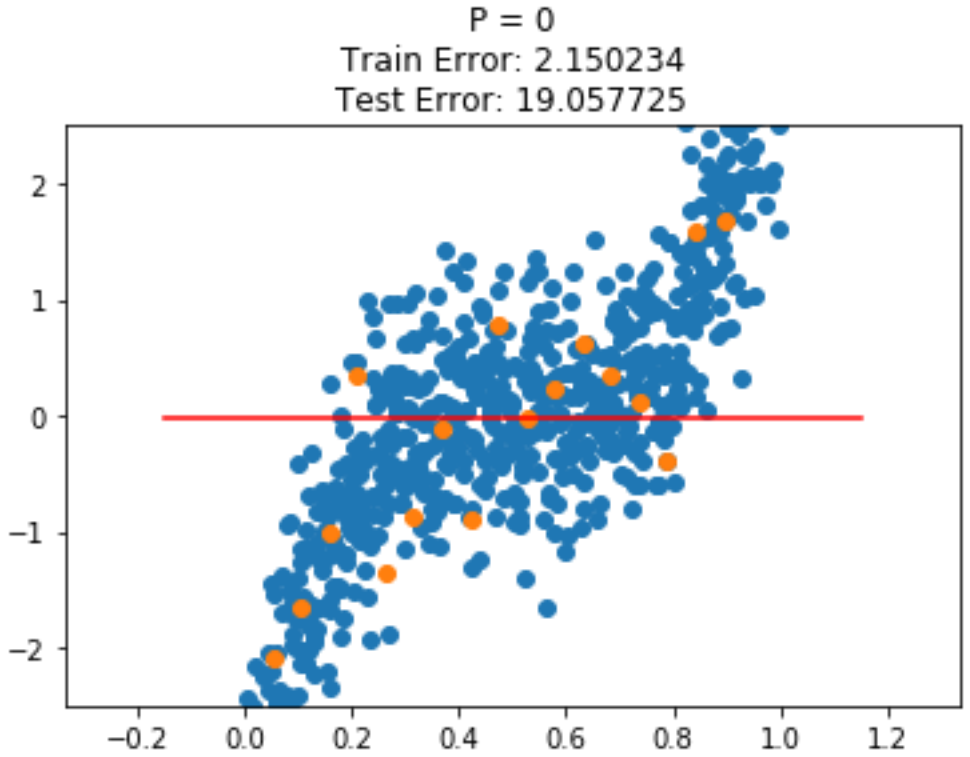


Polynomial fitting

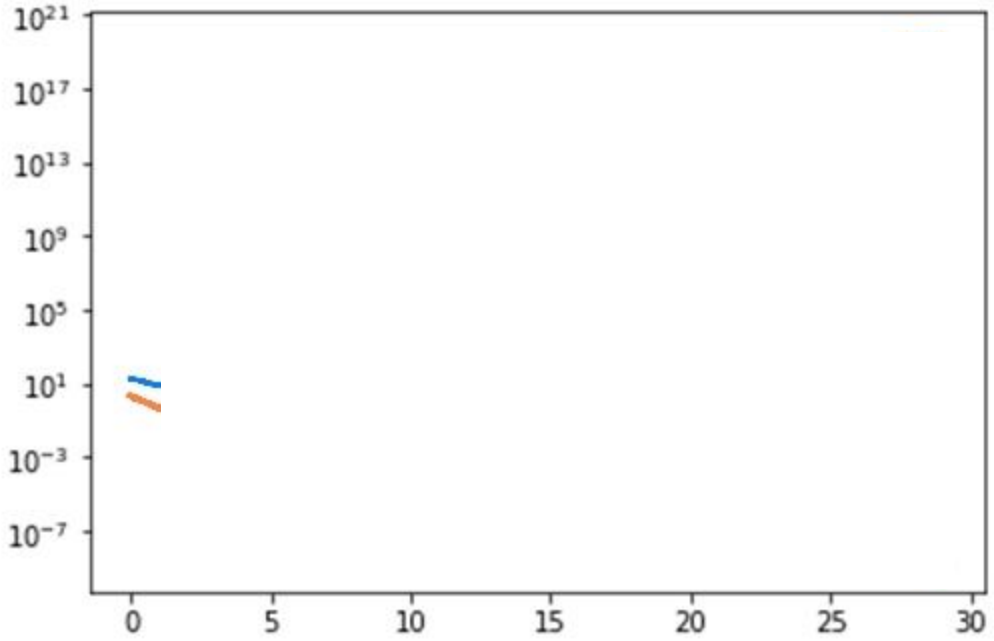


Train

Test

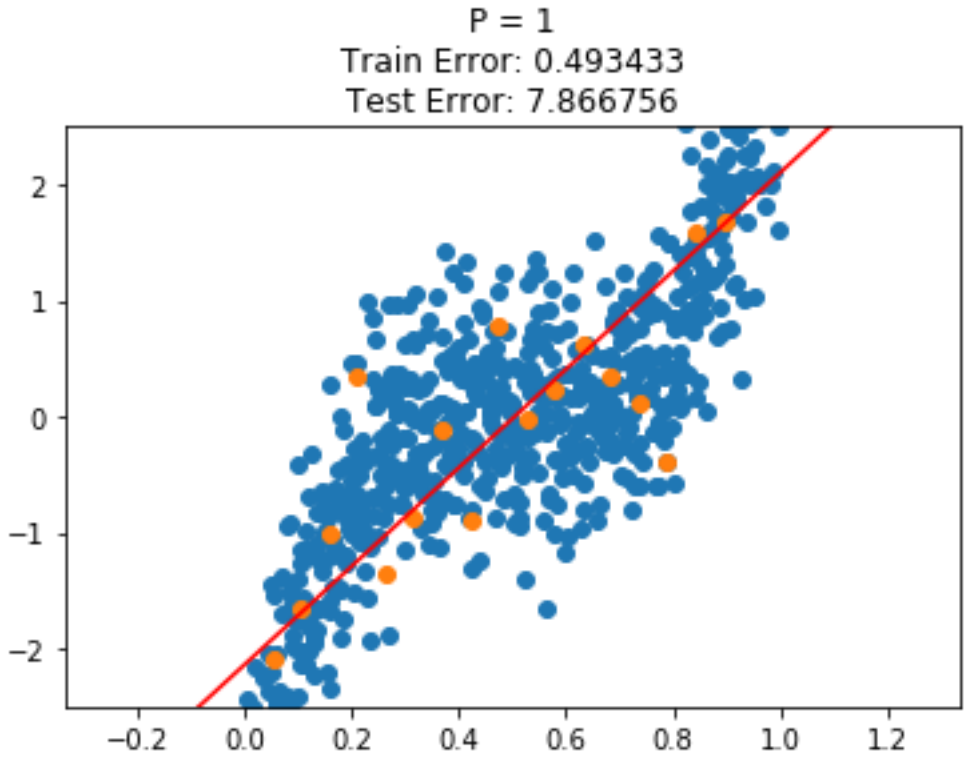


Polynomial fitting

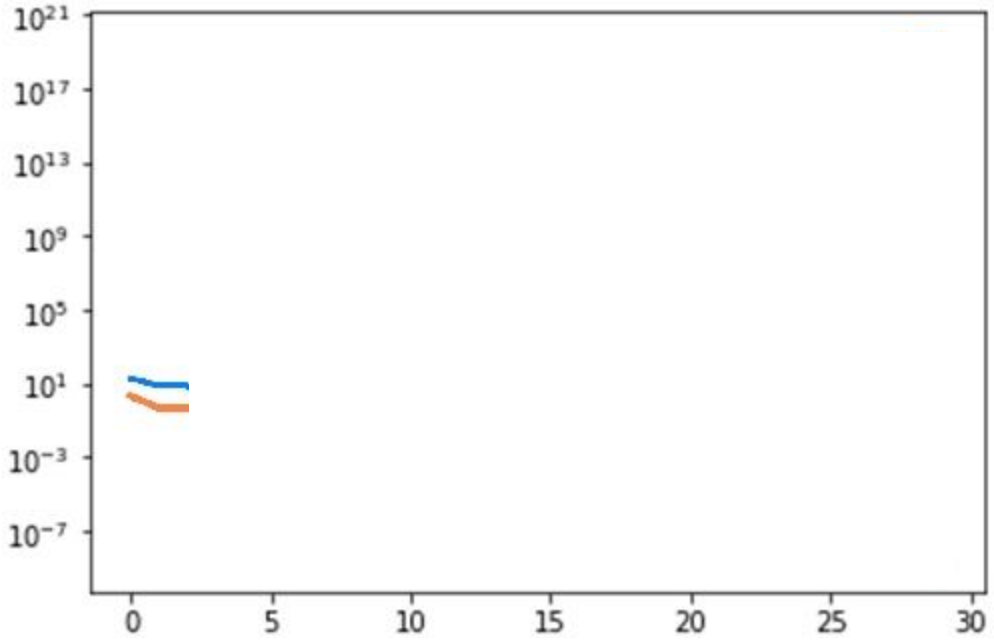


Train

Test



Polynomial fitting

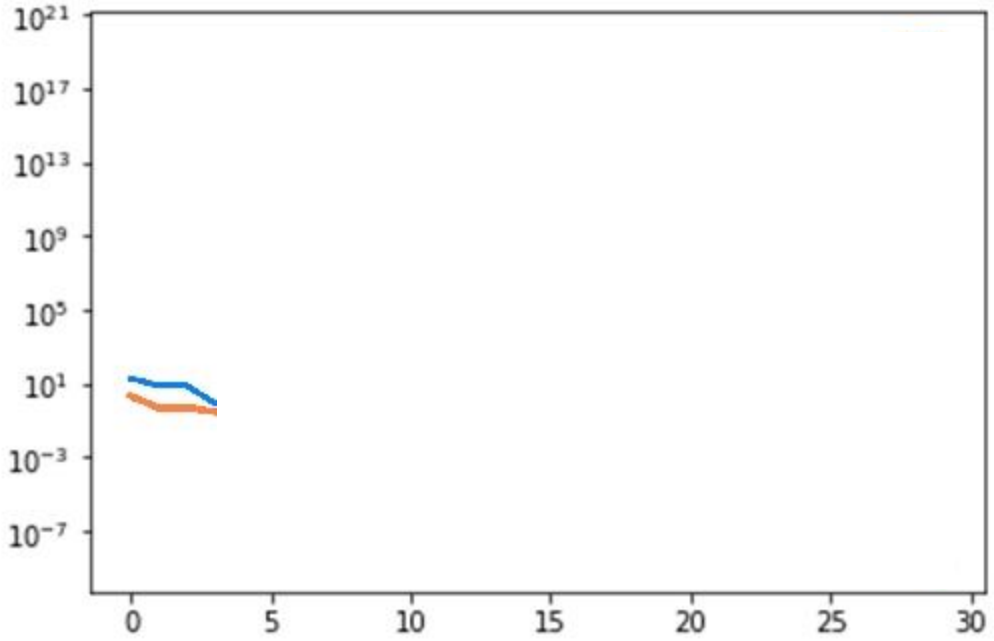


Train

Test



Polynomial fitting

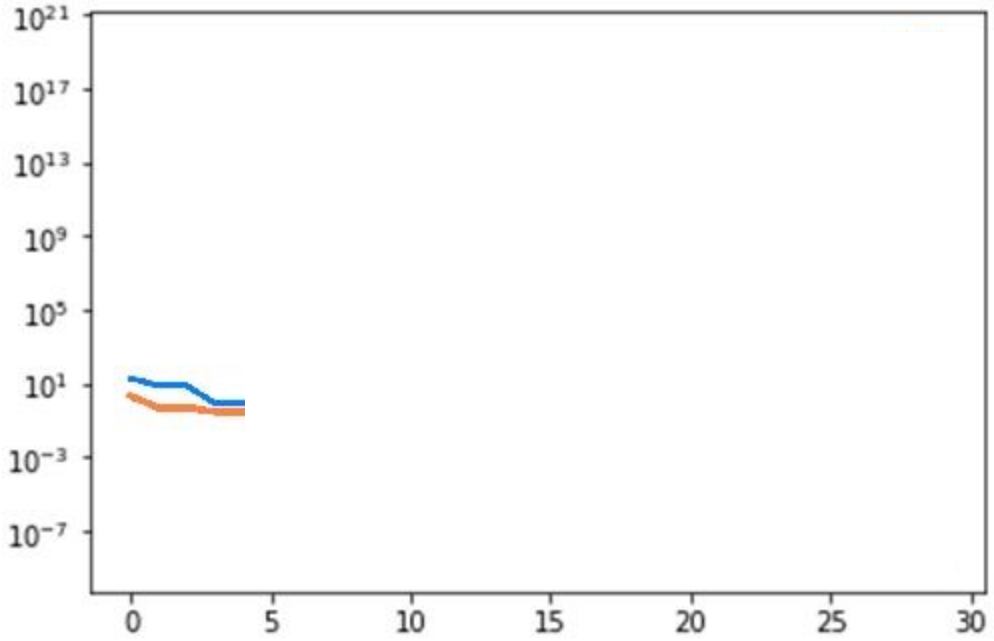


Train

Test



Polynomial fitting

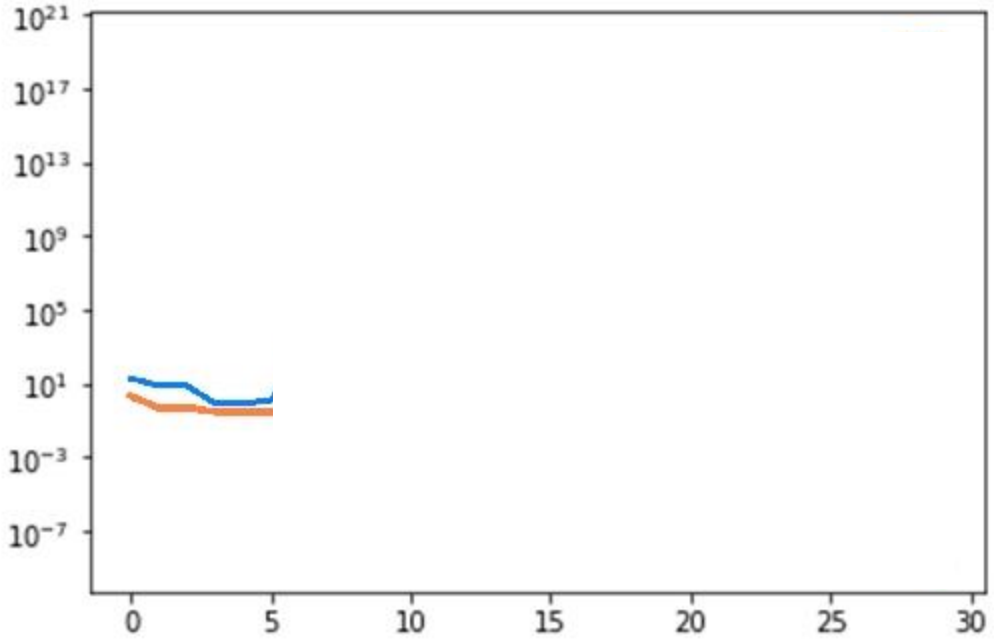


Train

Test



Polynomial fitting

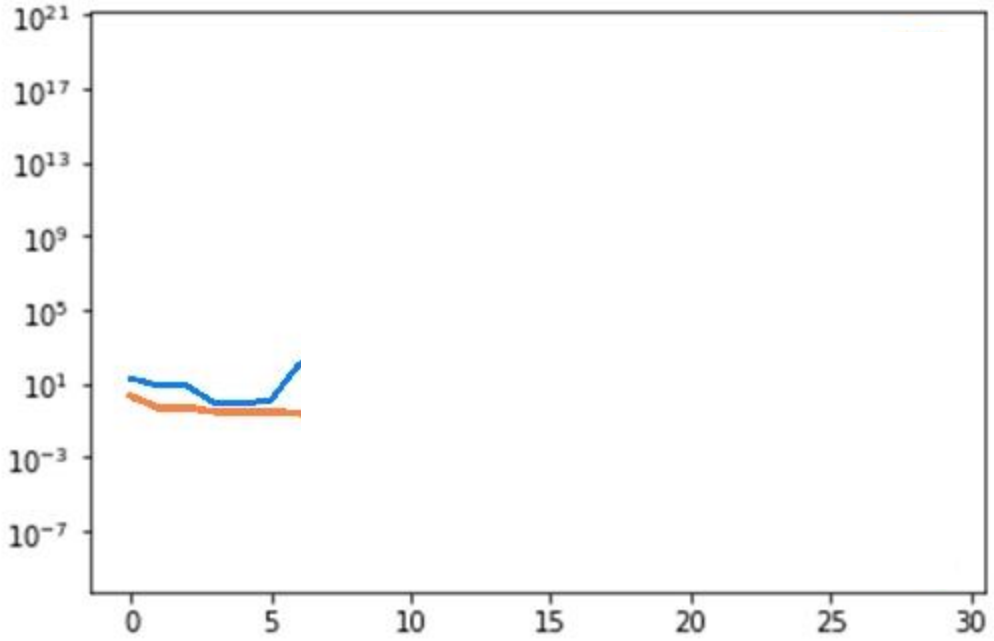


Train

Test



Polynomial fitting

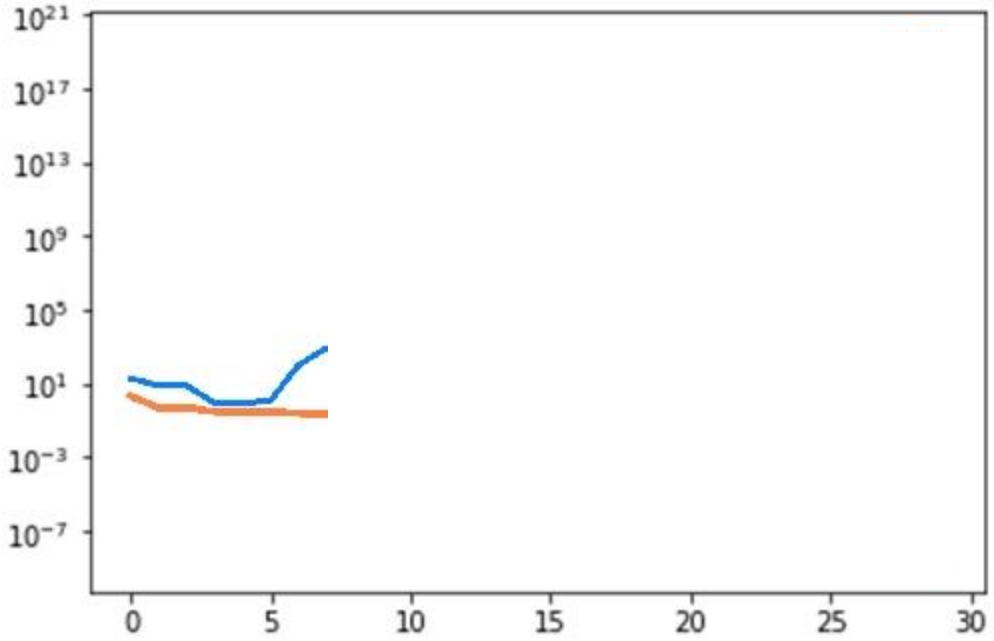


Train

Test

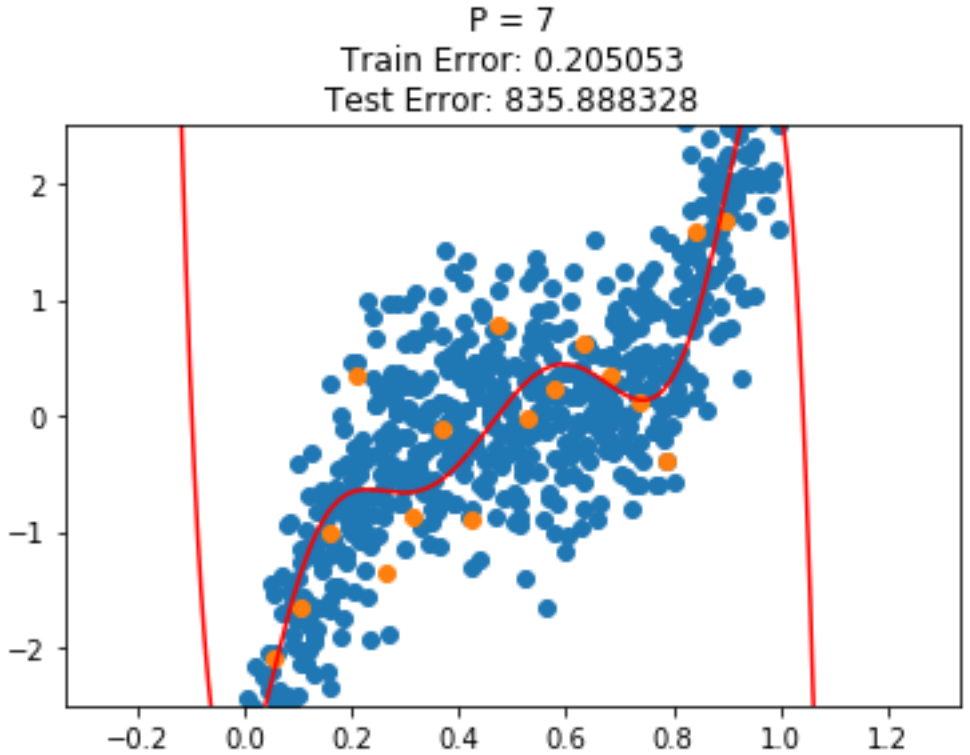


Polynomial fitting

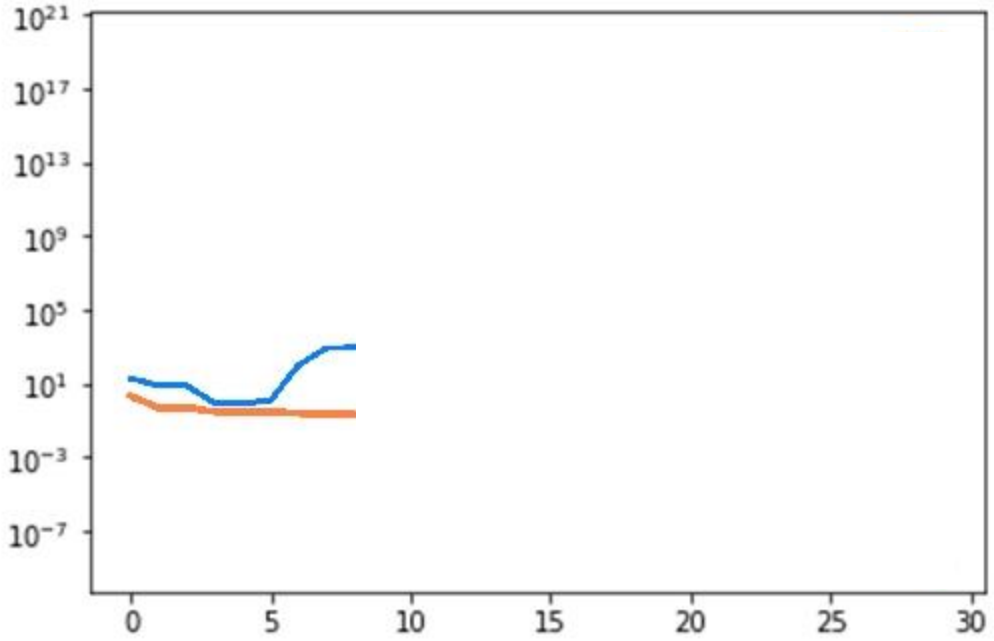


Train

Test

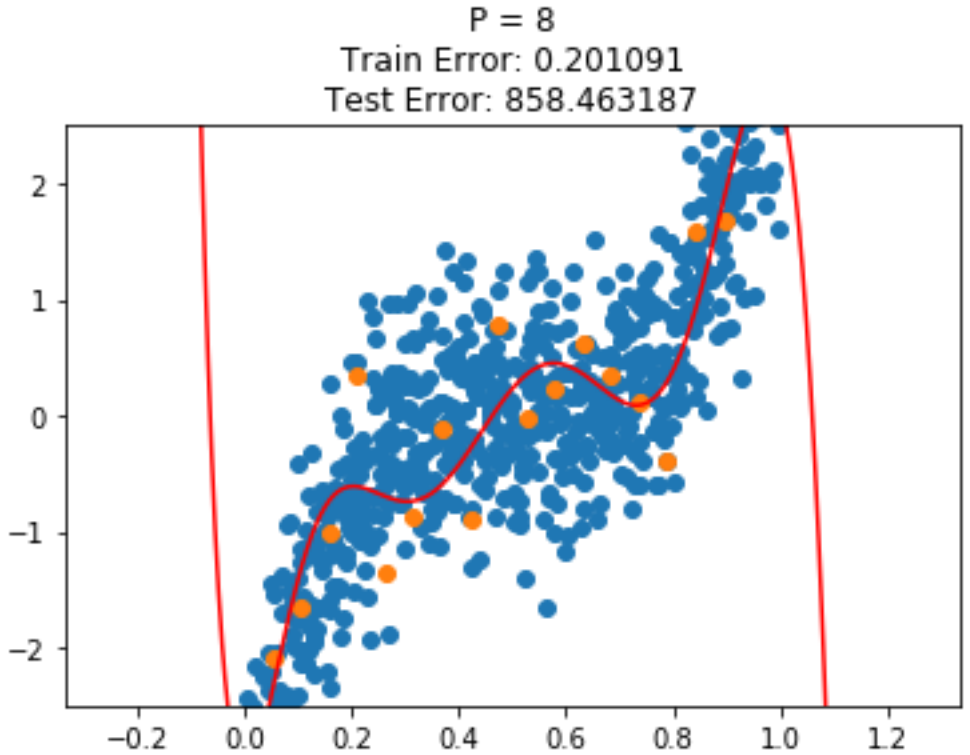


Polynomial fitting

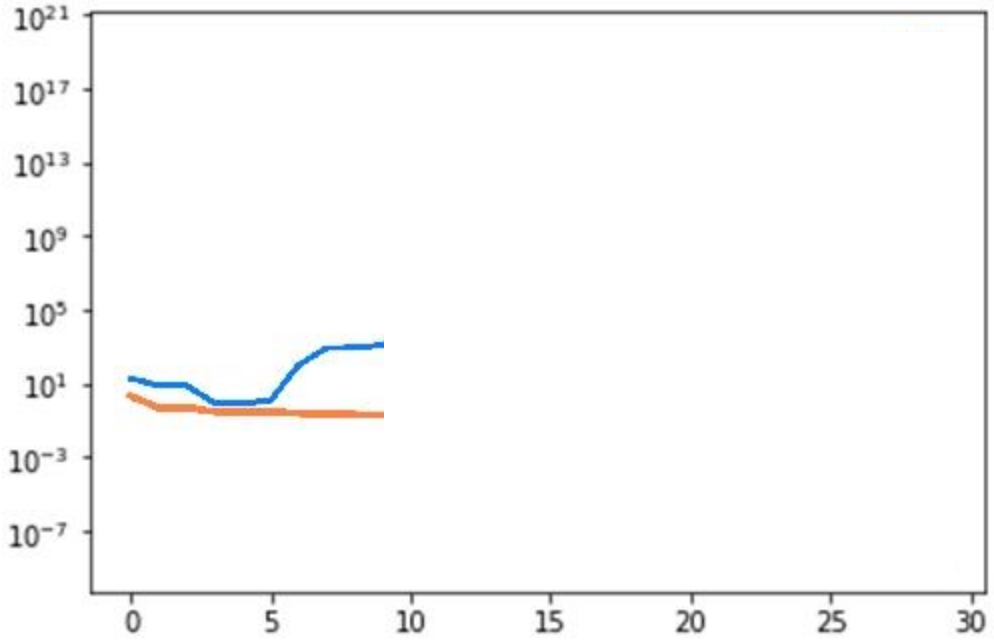


Train

Test

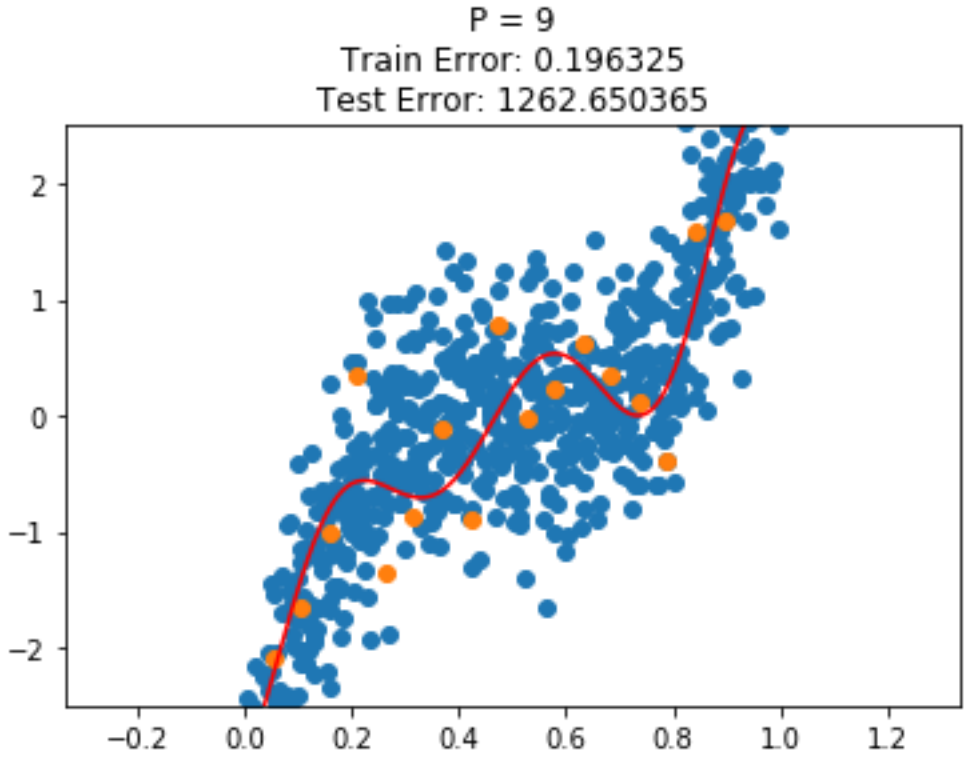


Polynomial fitting

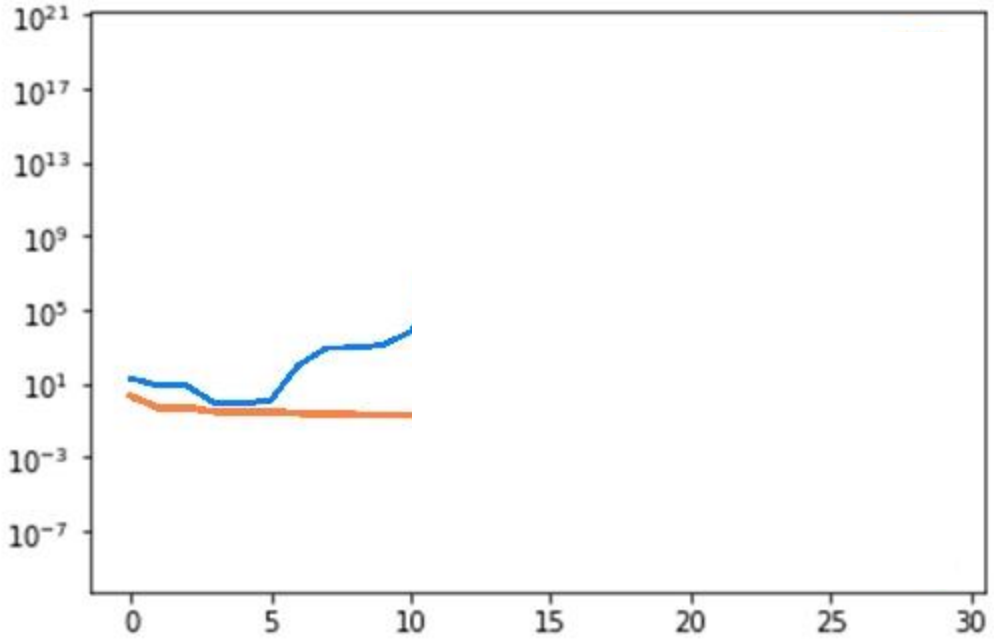


Train

Test

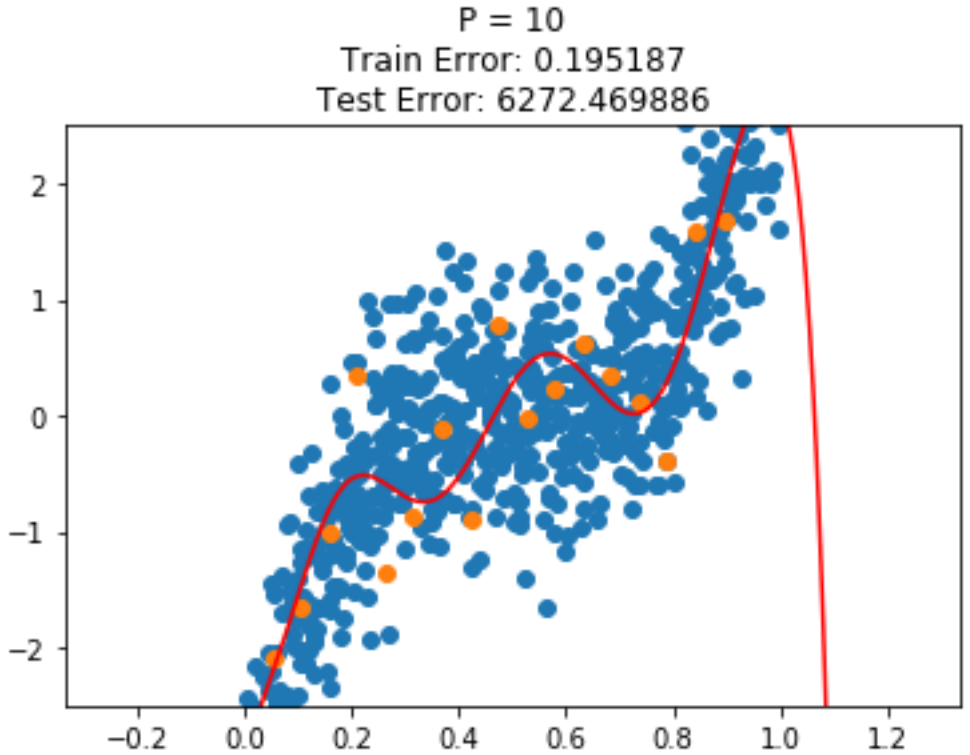


Polynomial fitting

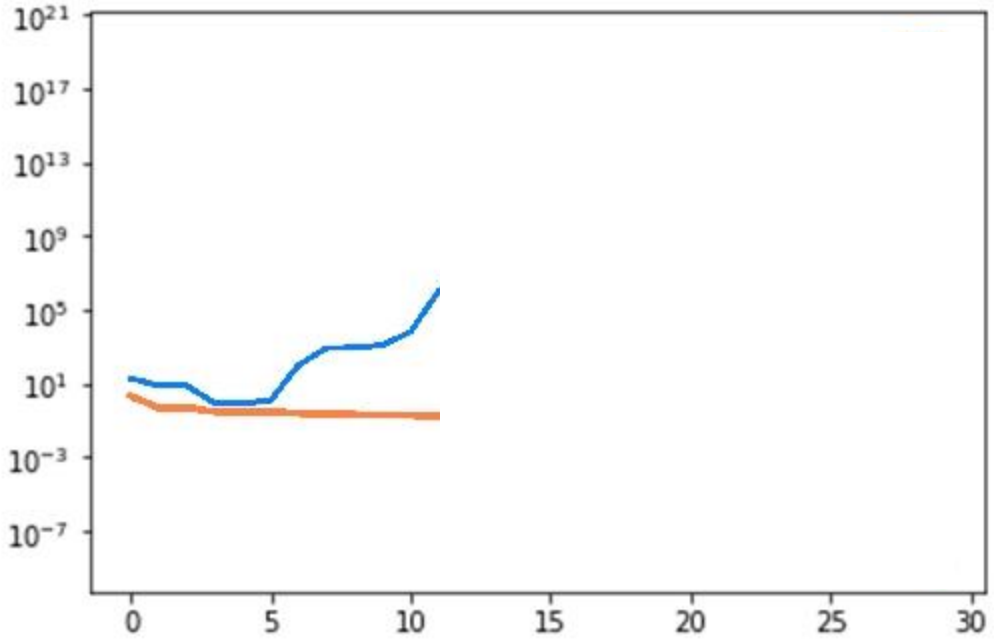


Train

Test

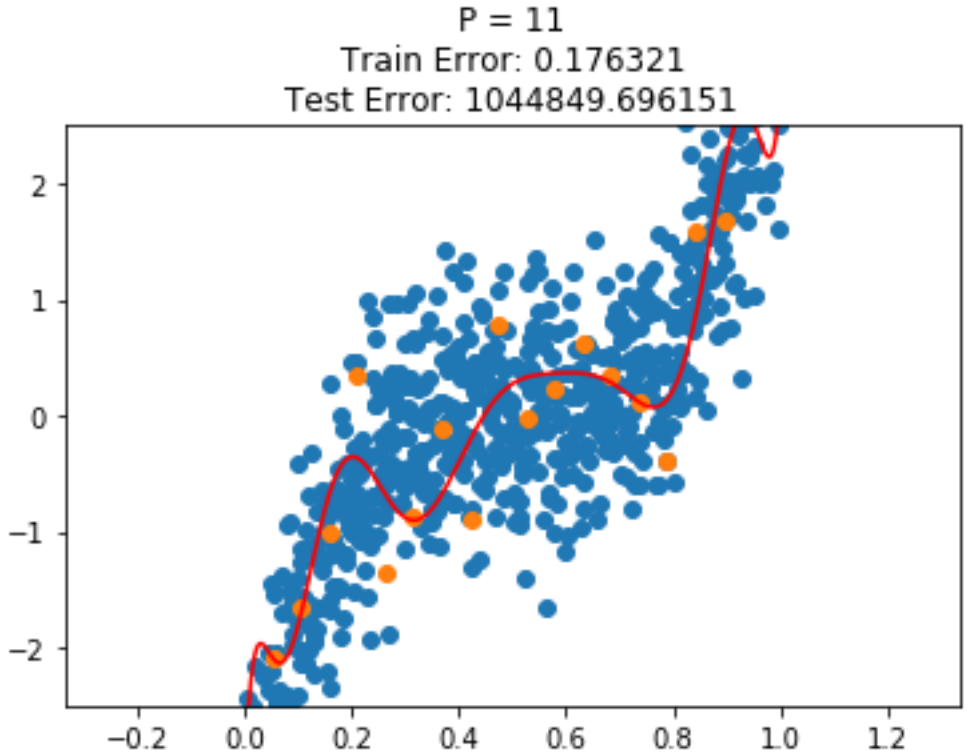


Polynomial fitting

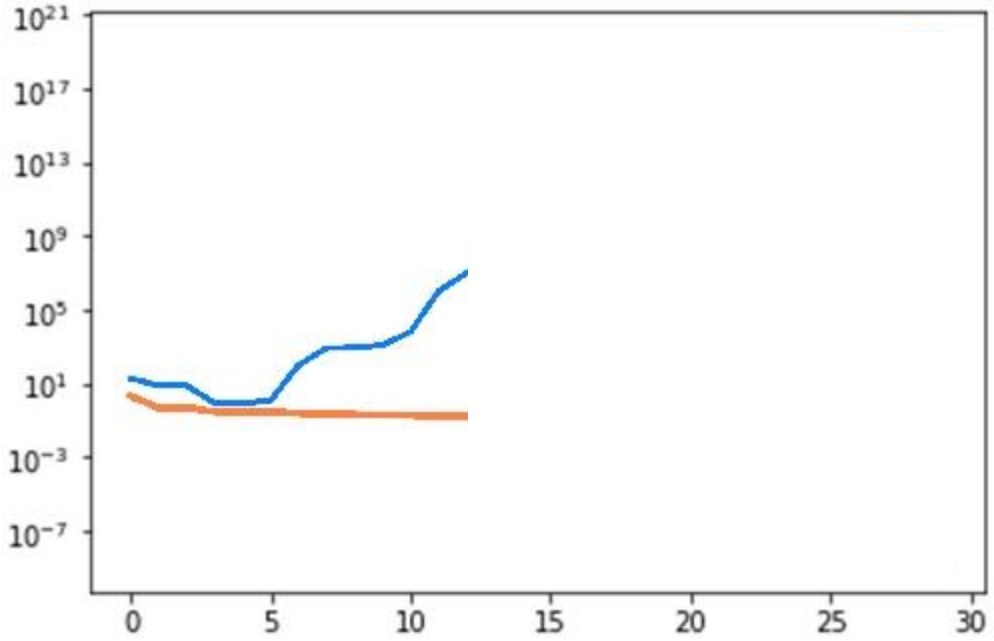


Train

Test

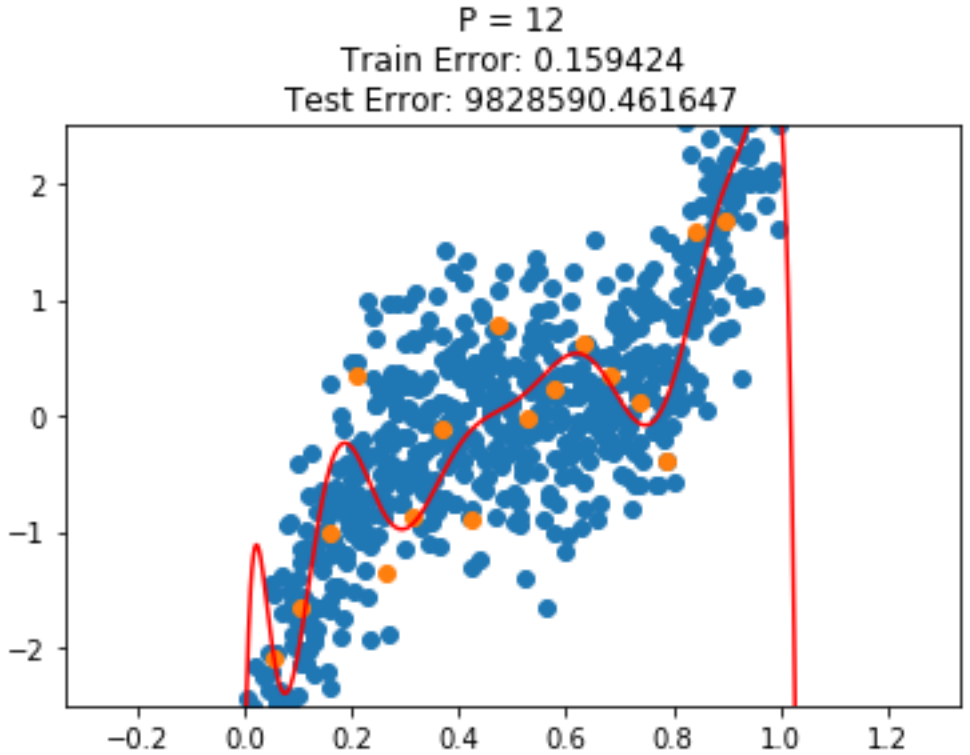


Polynomial fitting

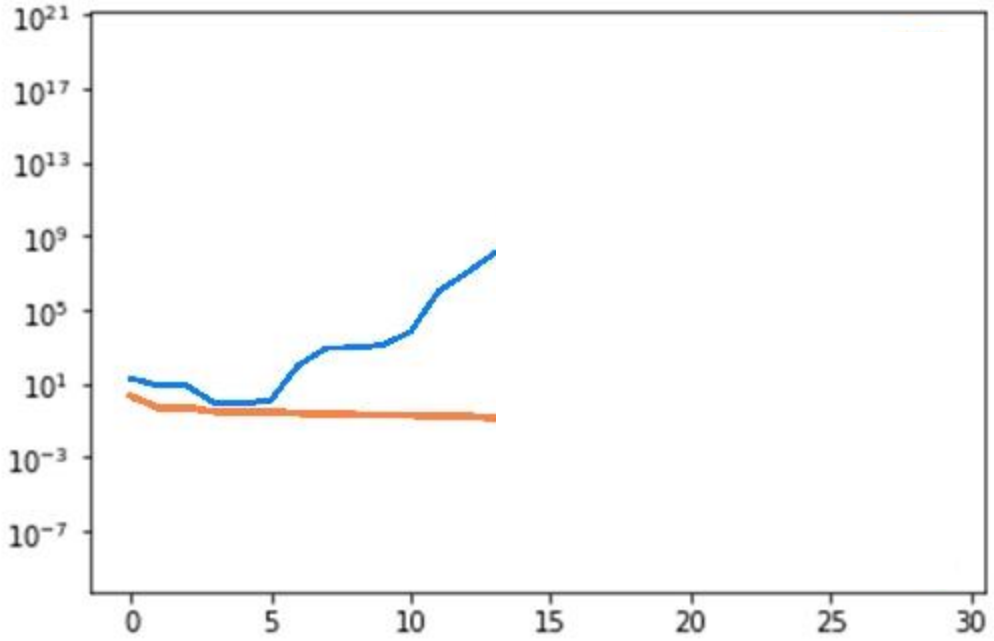


Train

Test

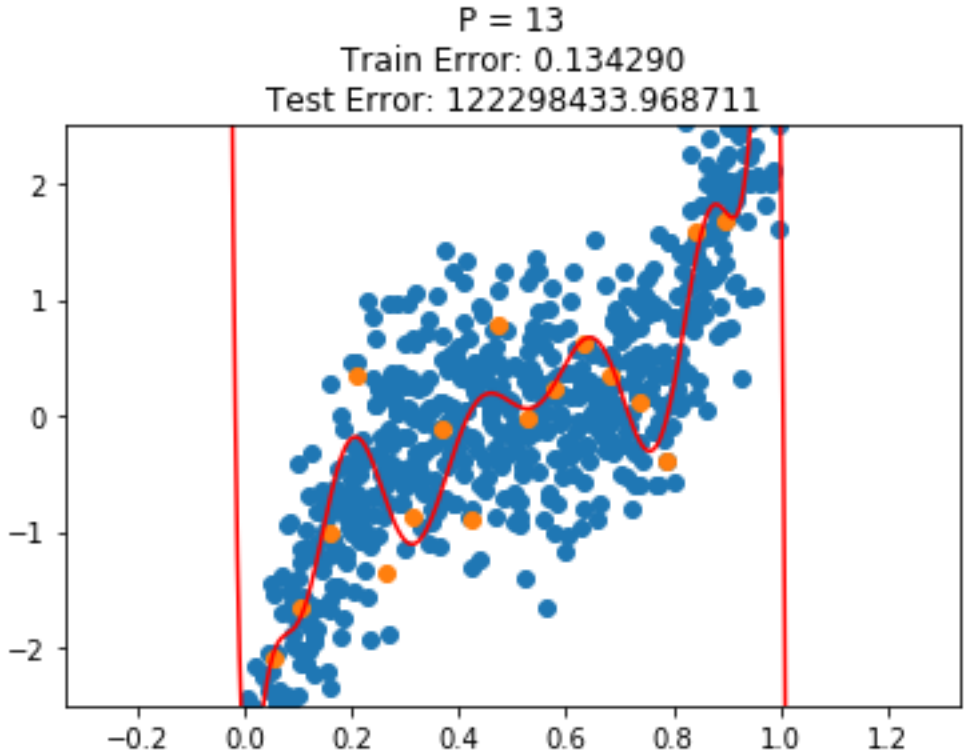


Polynomial fitting

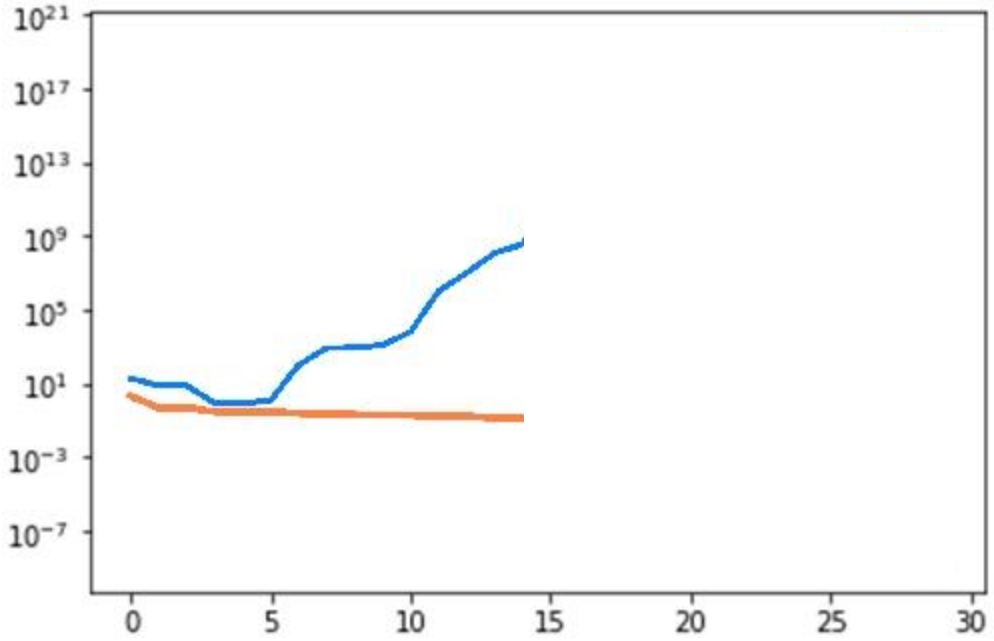


Train

Test

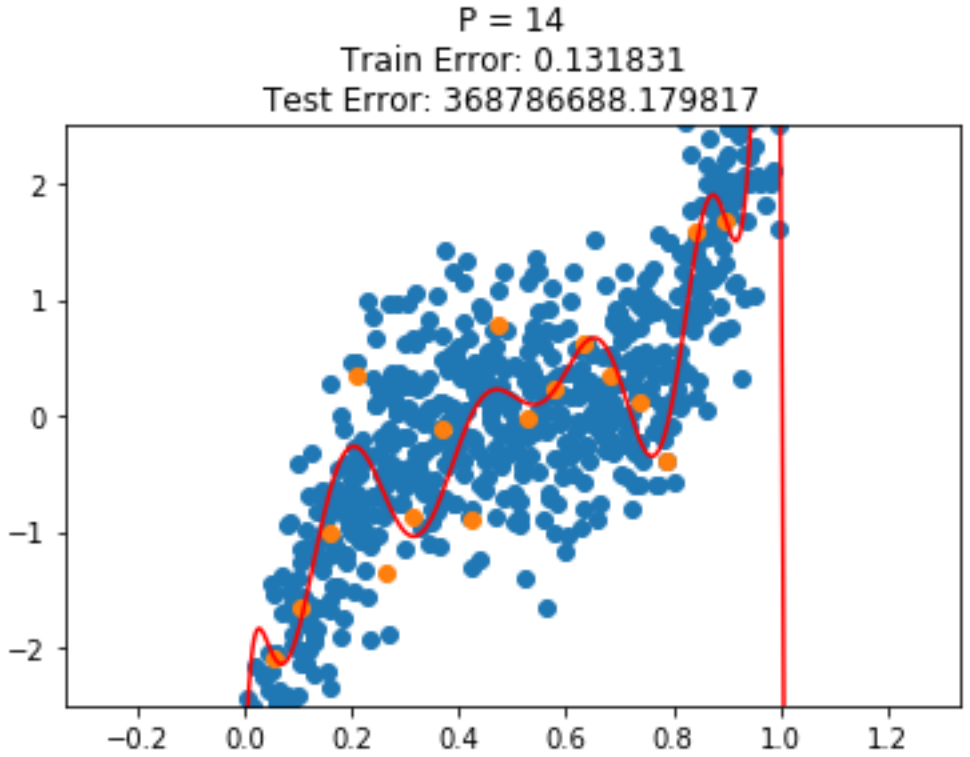


Polynomial fitting

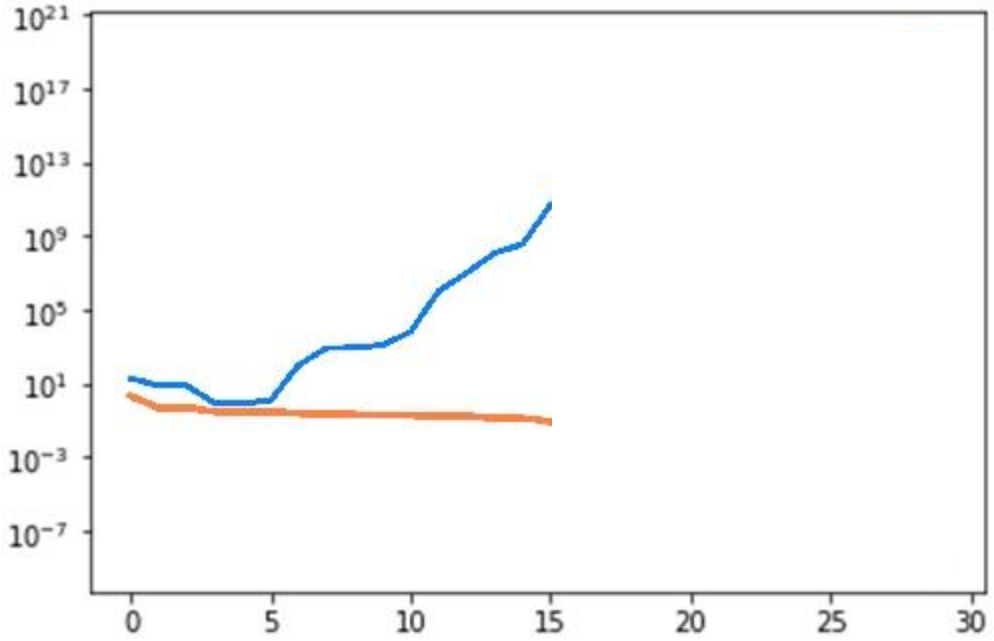


Train

Test

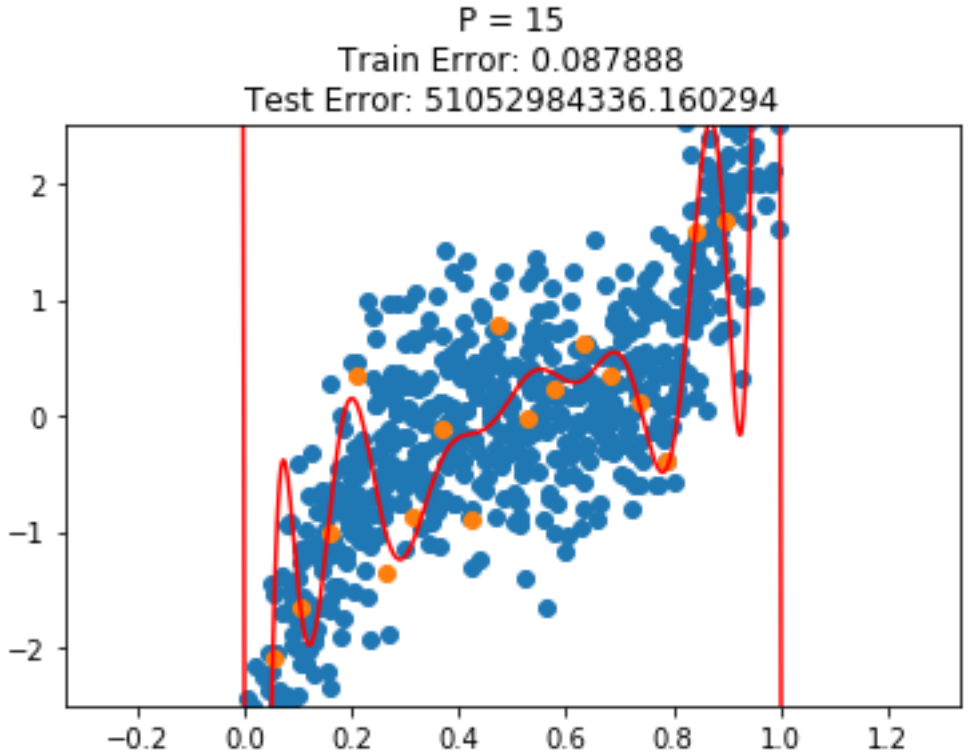


Polynomial fitting

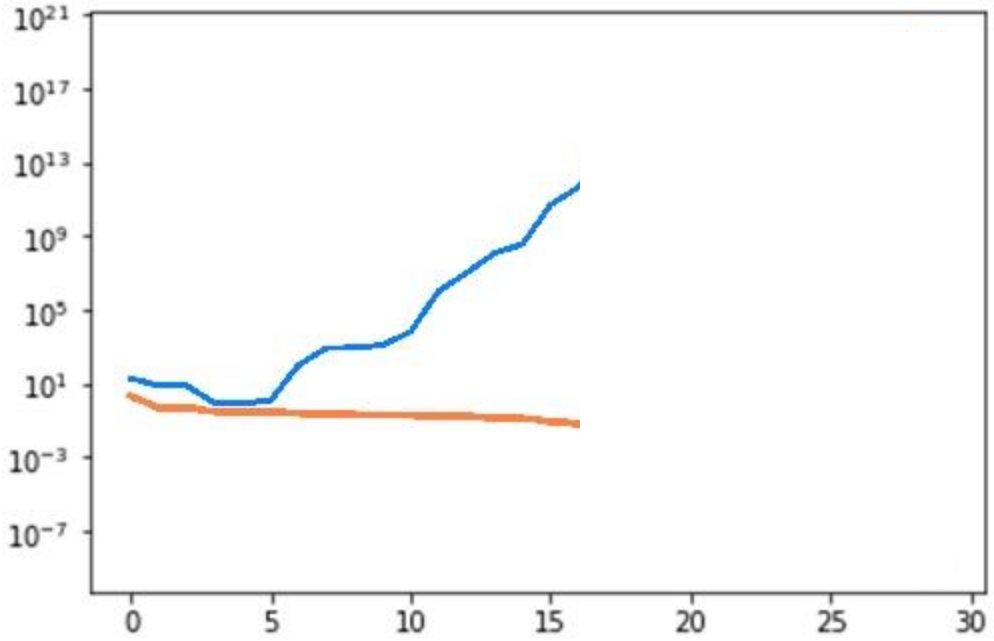


Train

Test

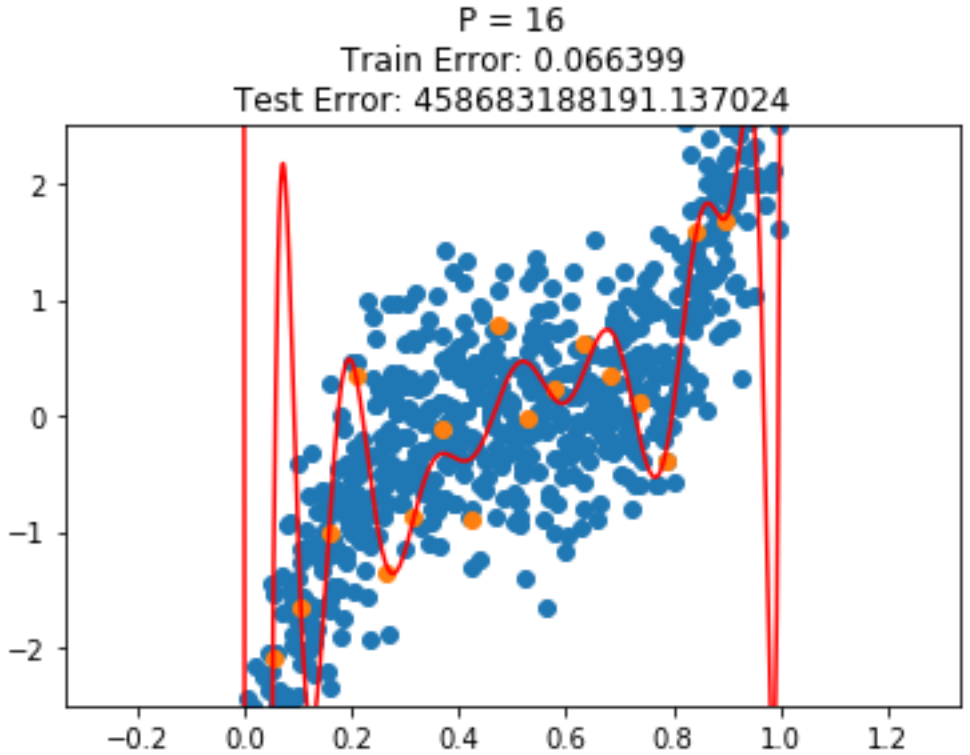


Polynomial fitting

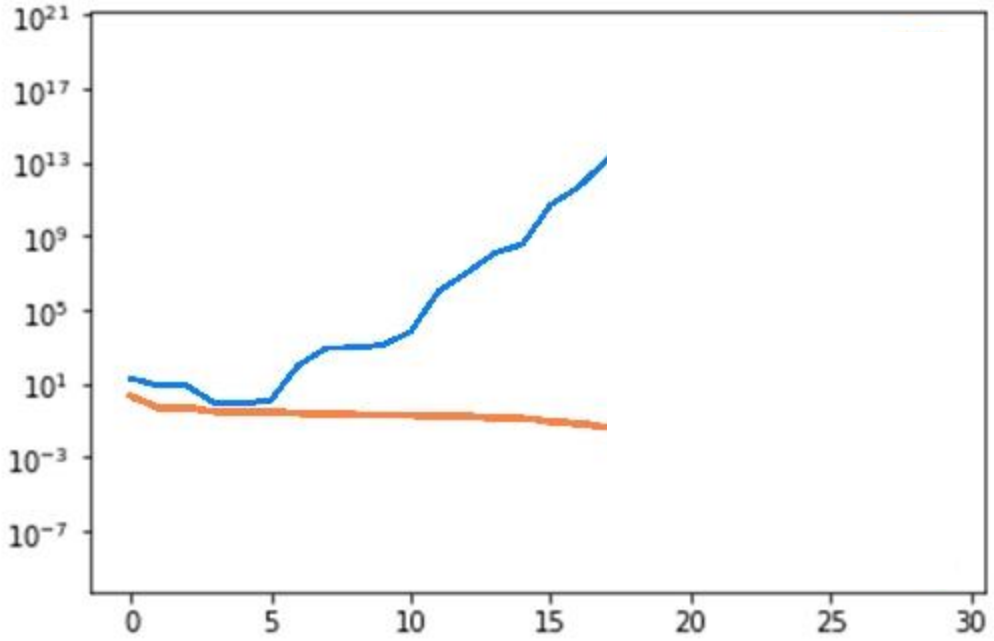


Train

Test

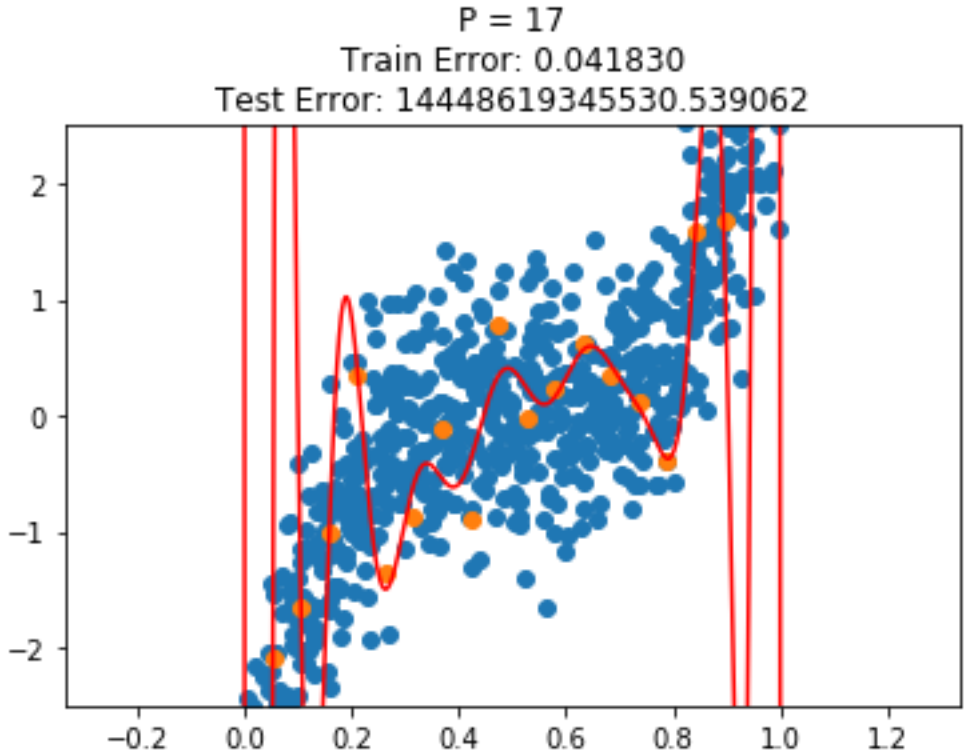


Polynomial fitting

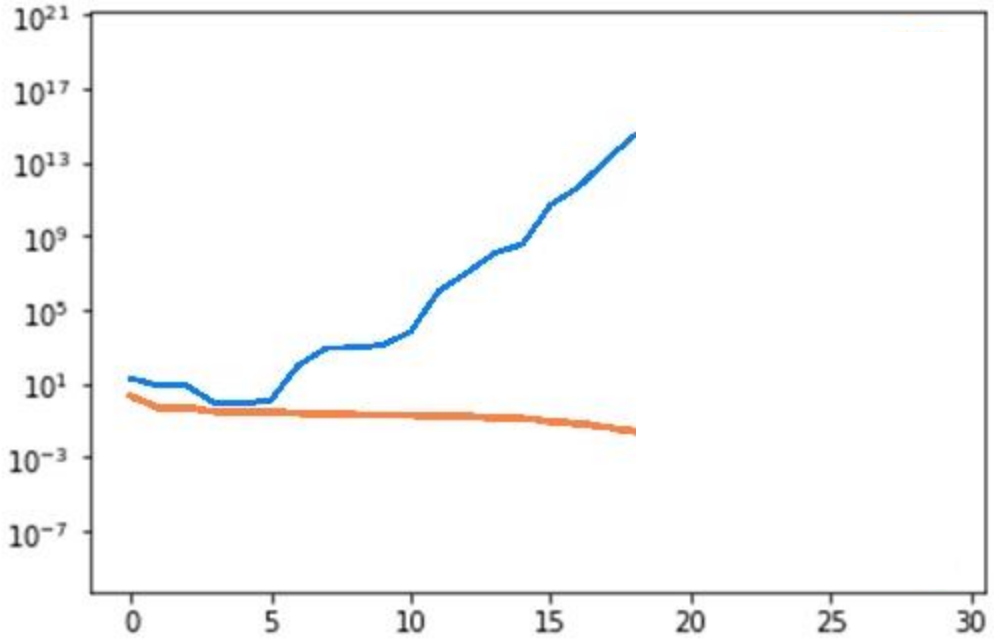


Train

Test

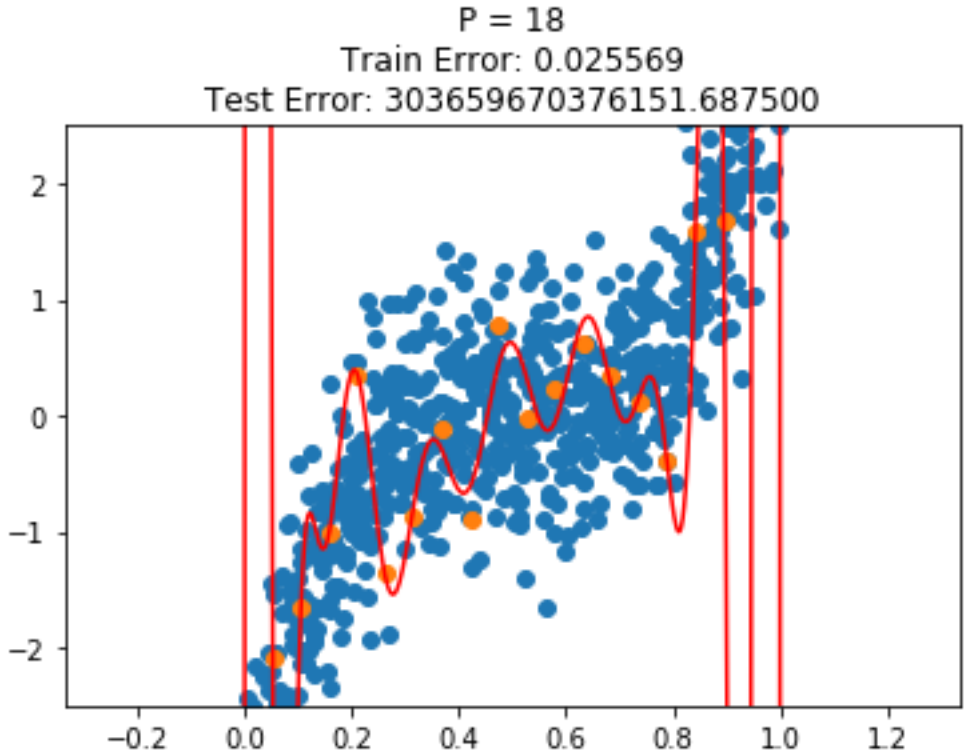


Polynomial fitting

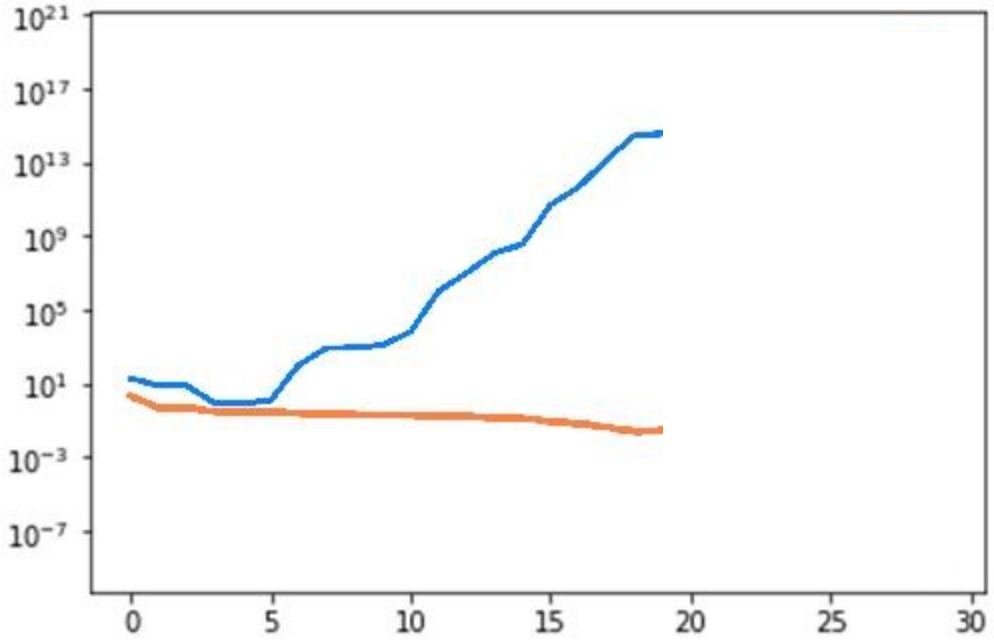


Train

Test



Polynomial fitting

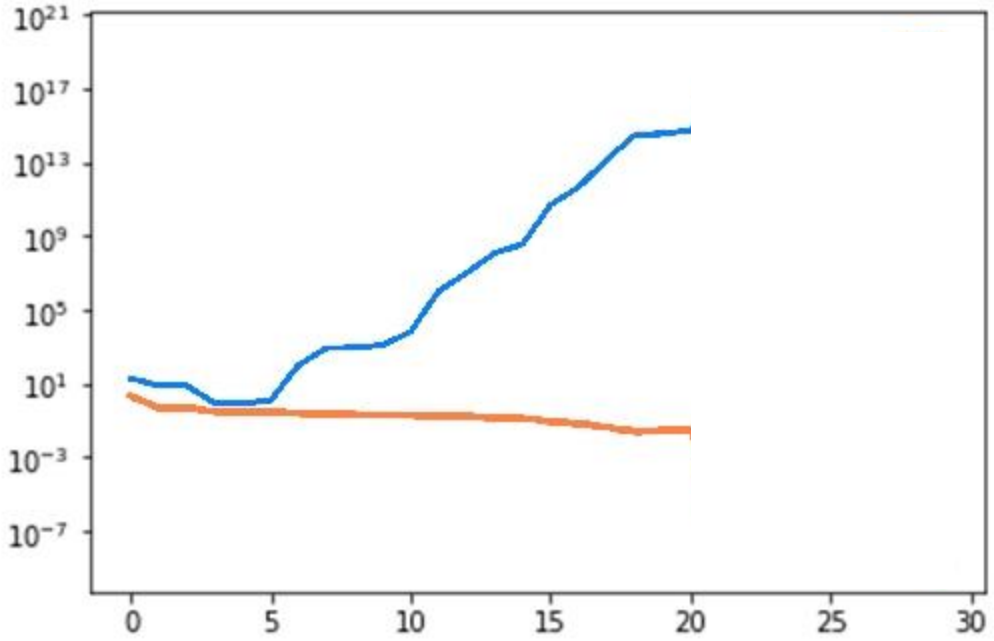


Train

Test

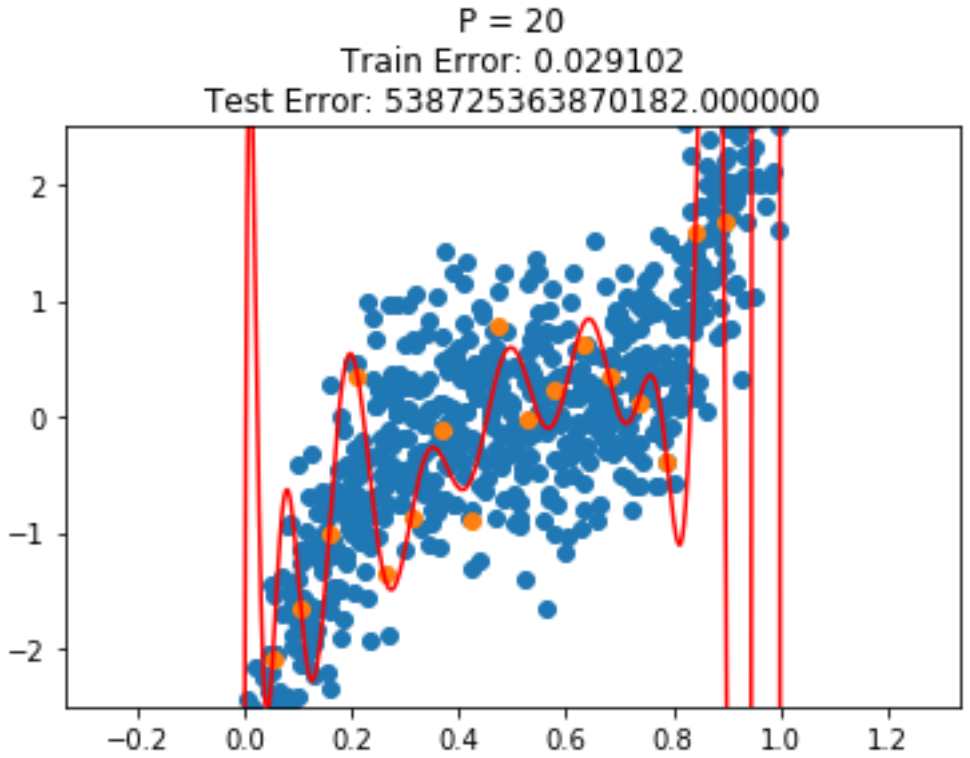


Polynomial fitting

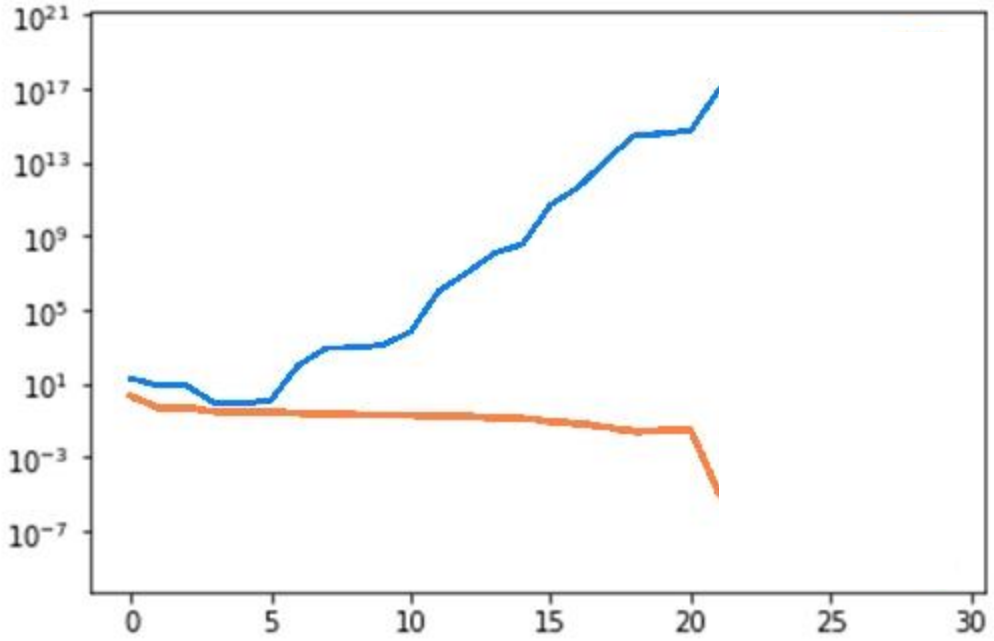


Train

Test

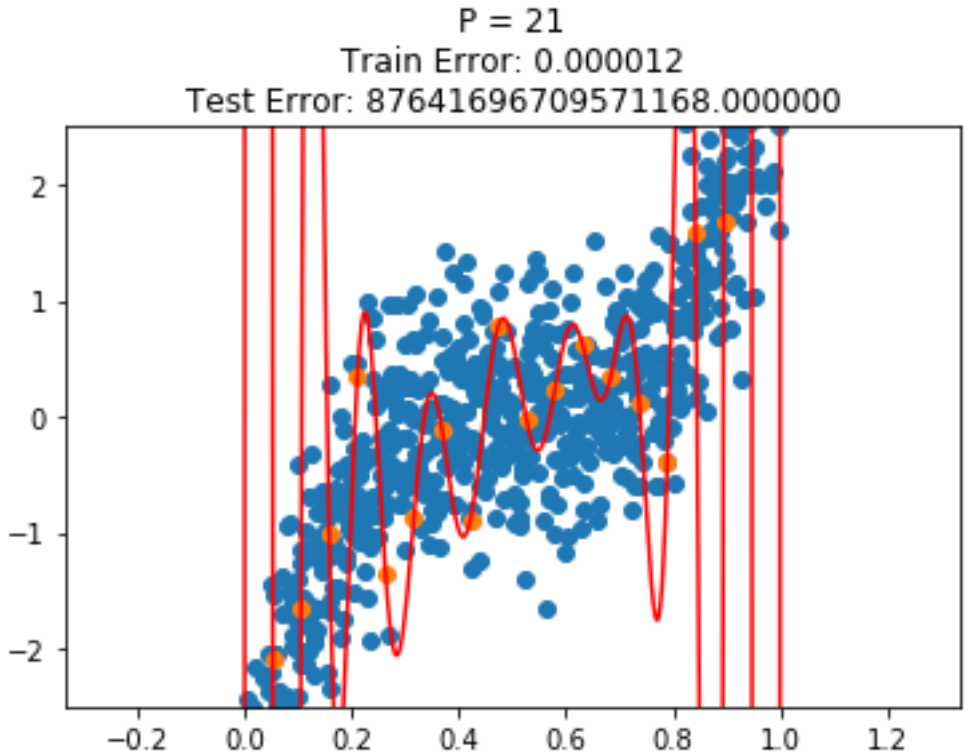


Polynomial fitting

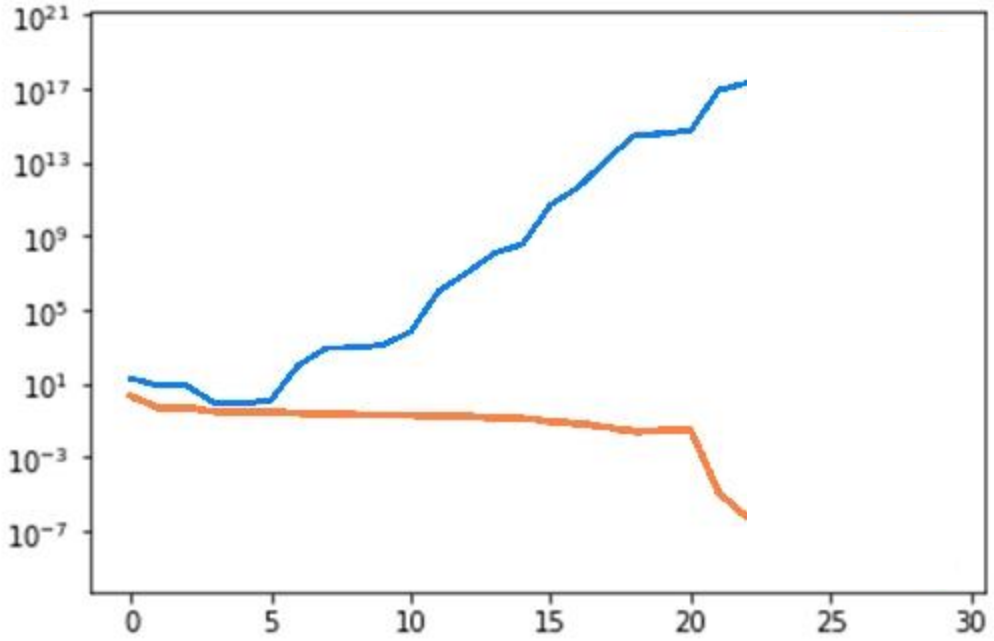


Train

Test

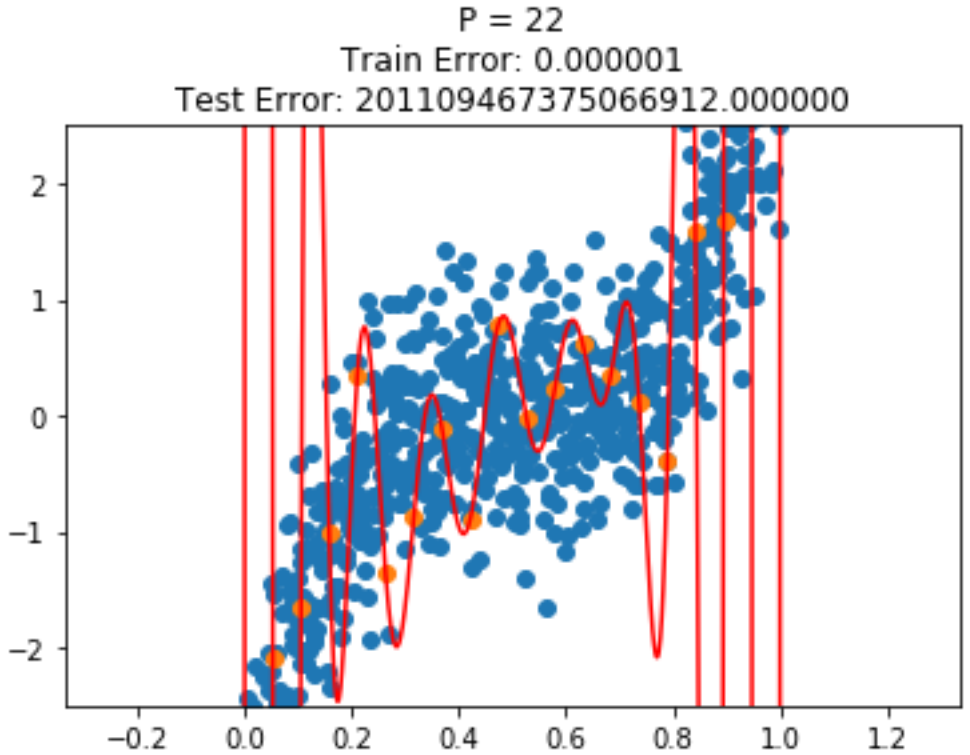


Polynomial fitting

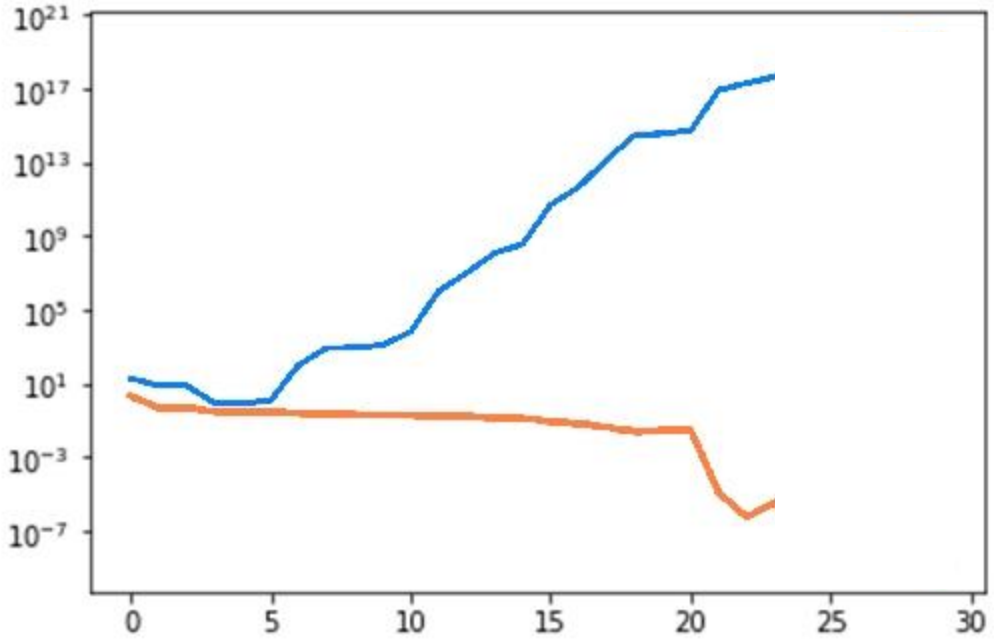


Train

Test

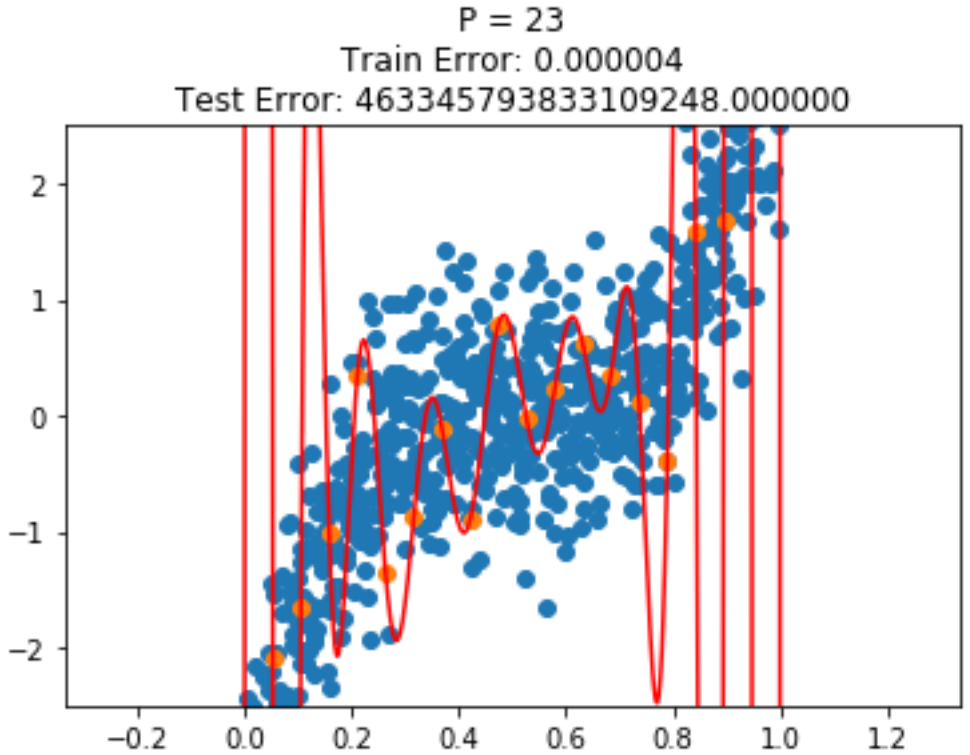


Polynomial fitting

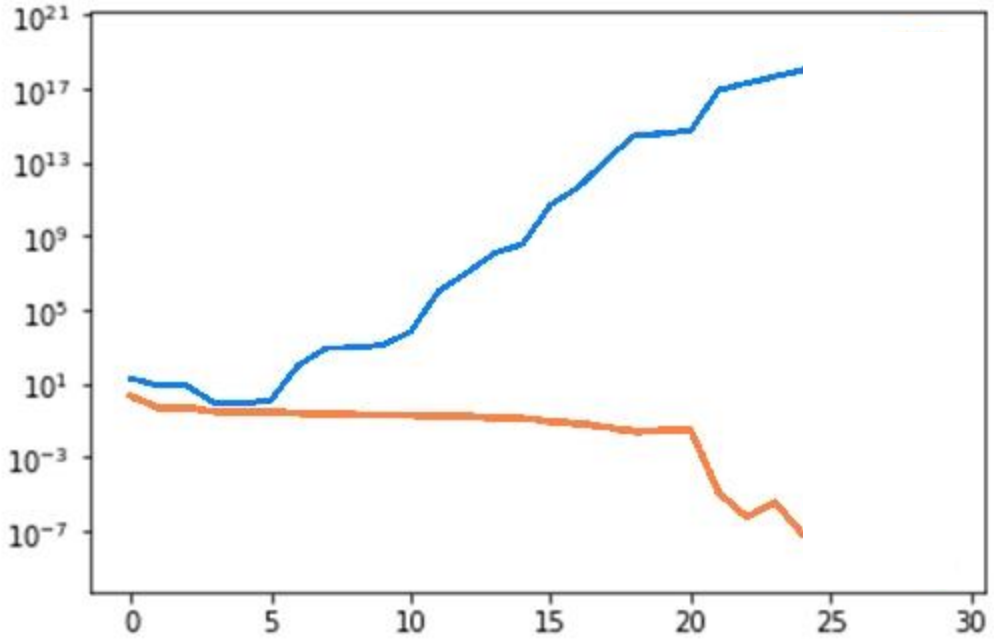


Train

Test

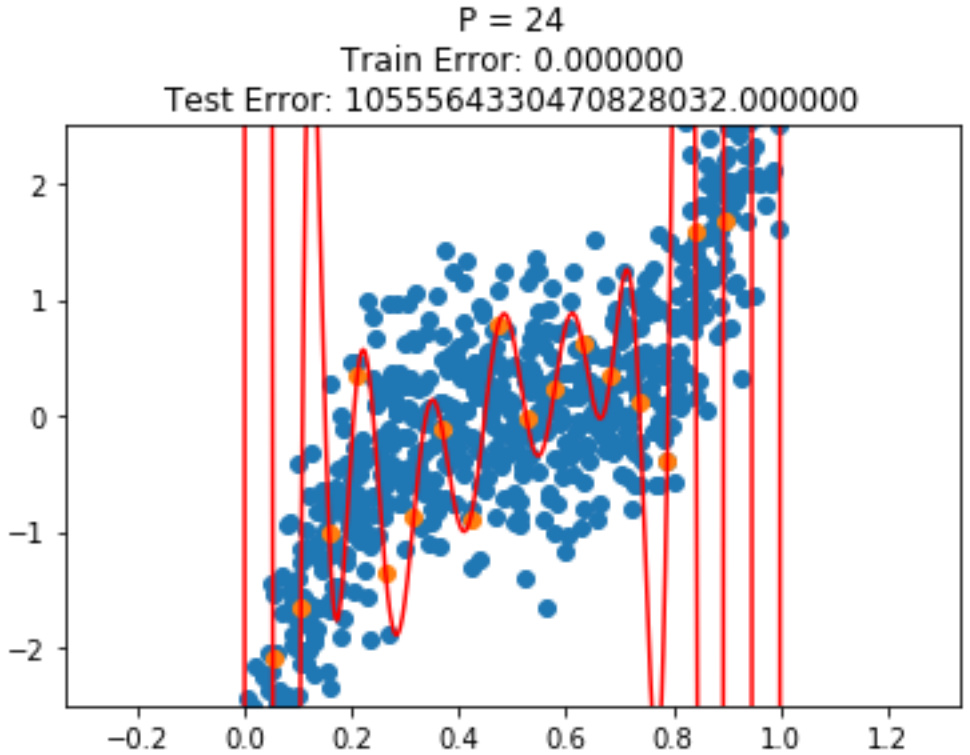


Polynomial fitting

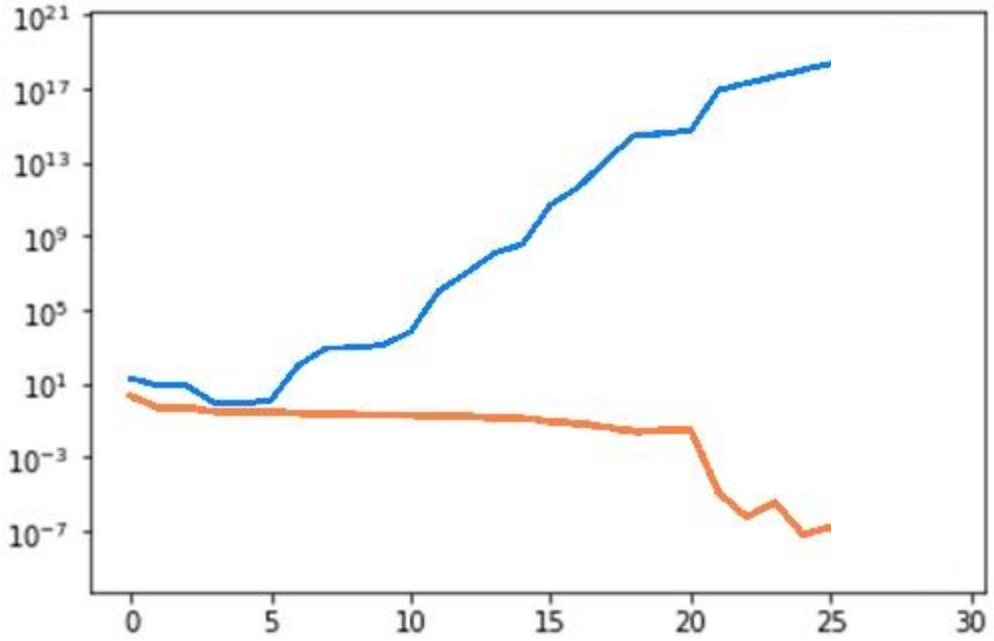


Train

Test

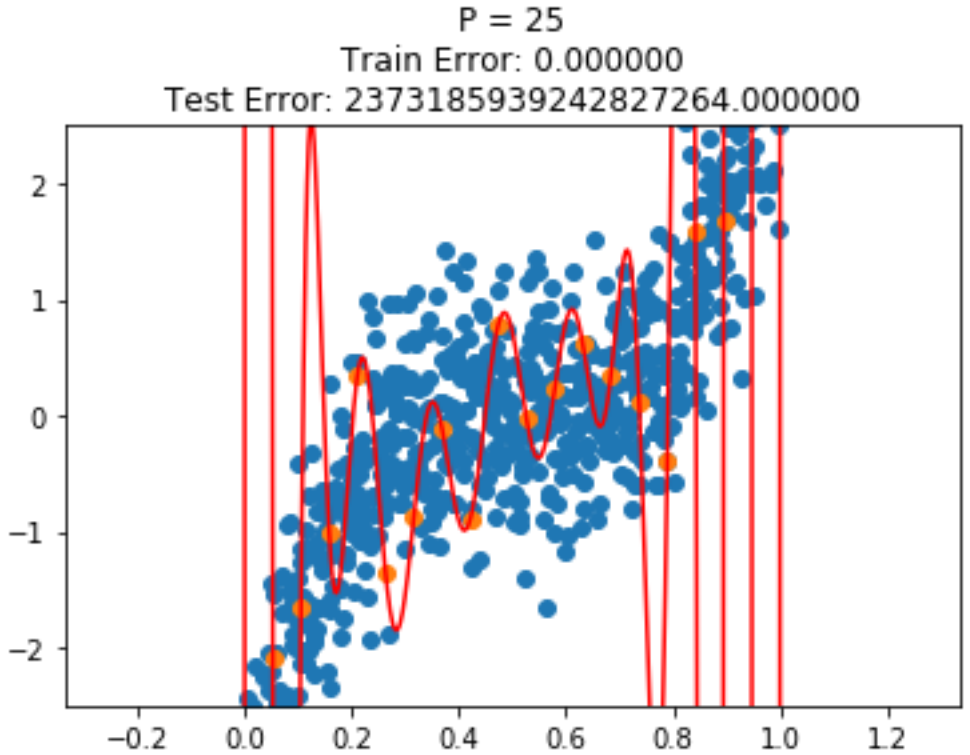


Polynomial fitting

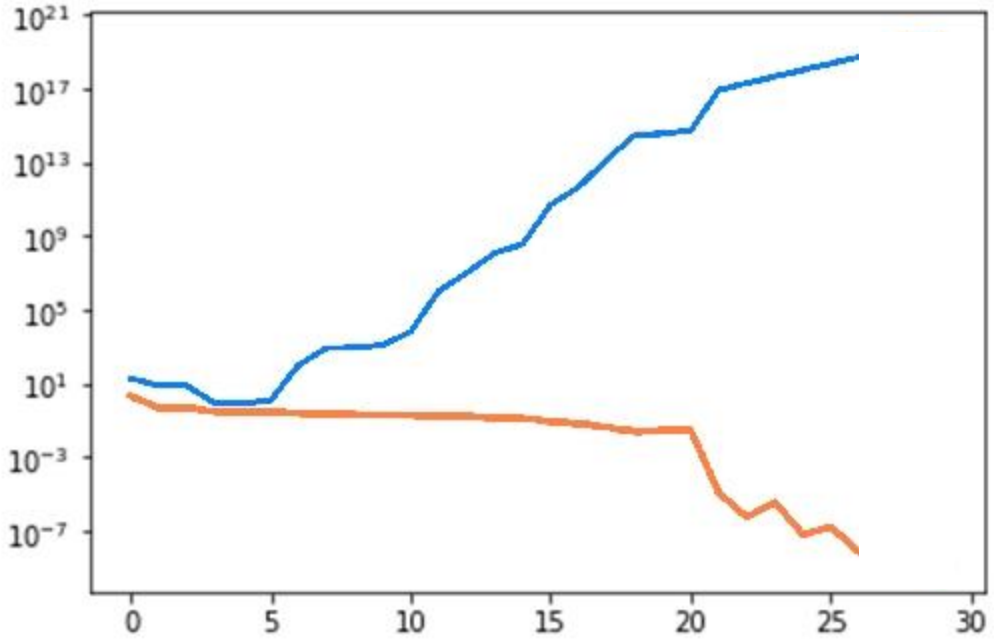


Train

Test

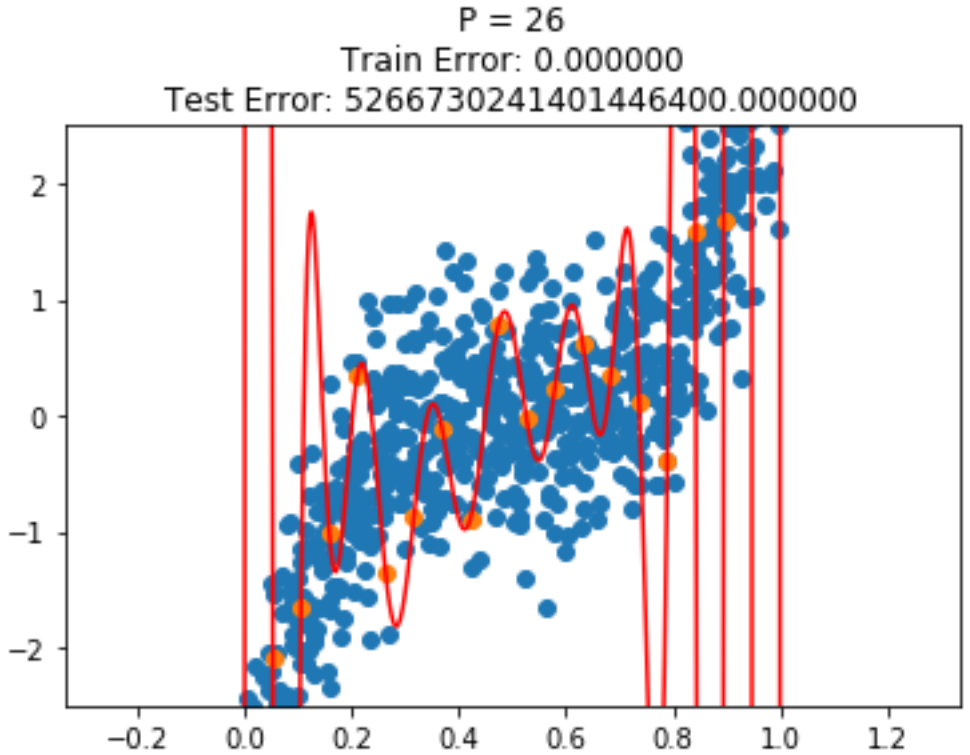


Polynomial fitting

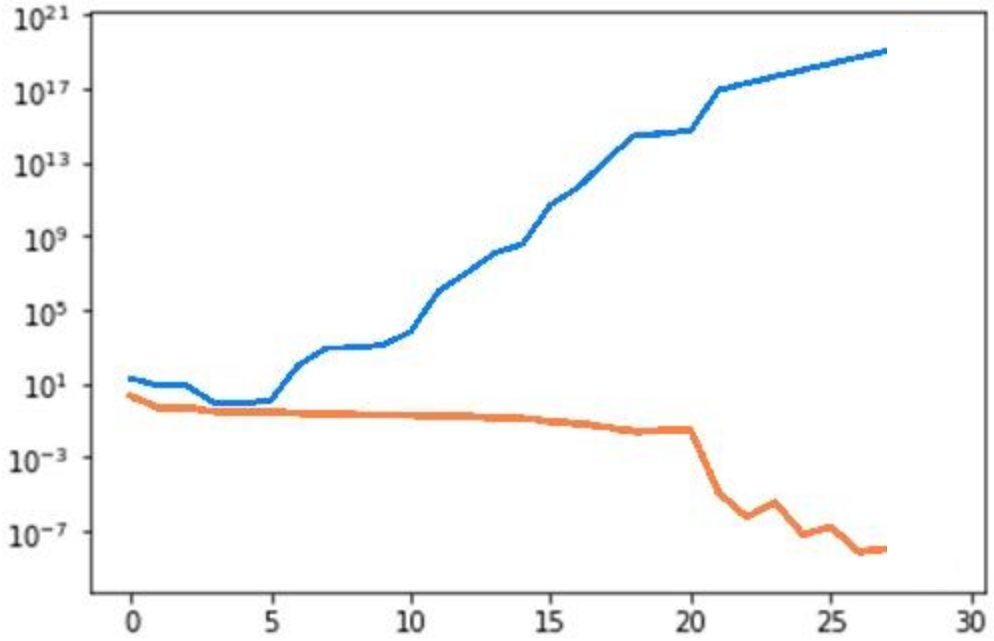


Train

Test

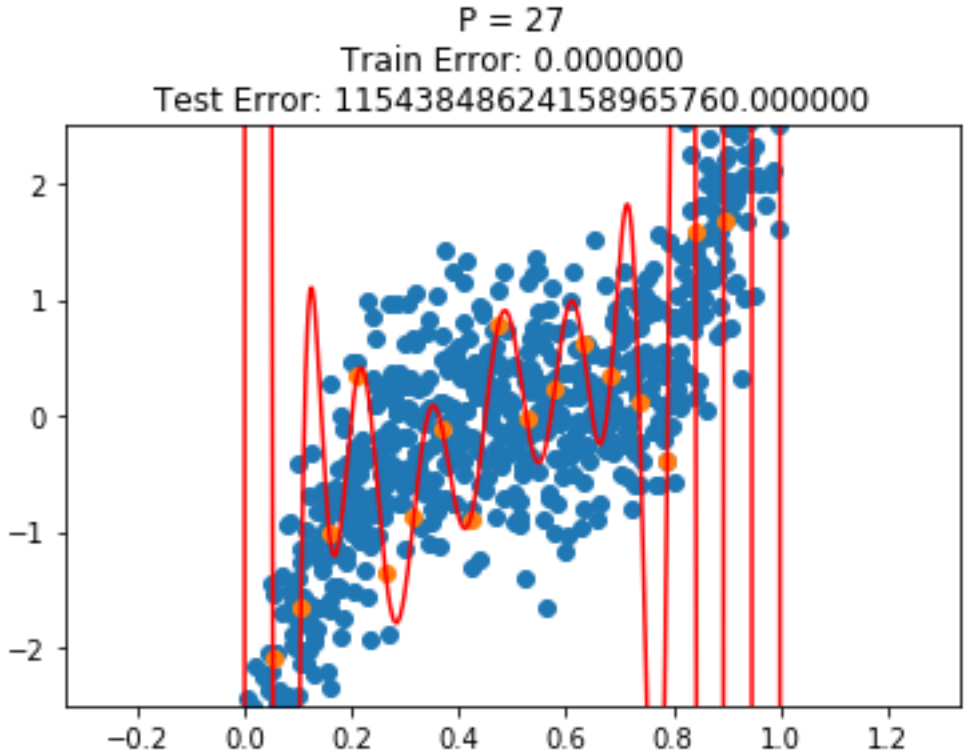


Polynomial fitting

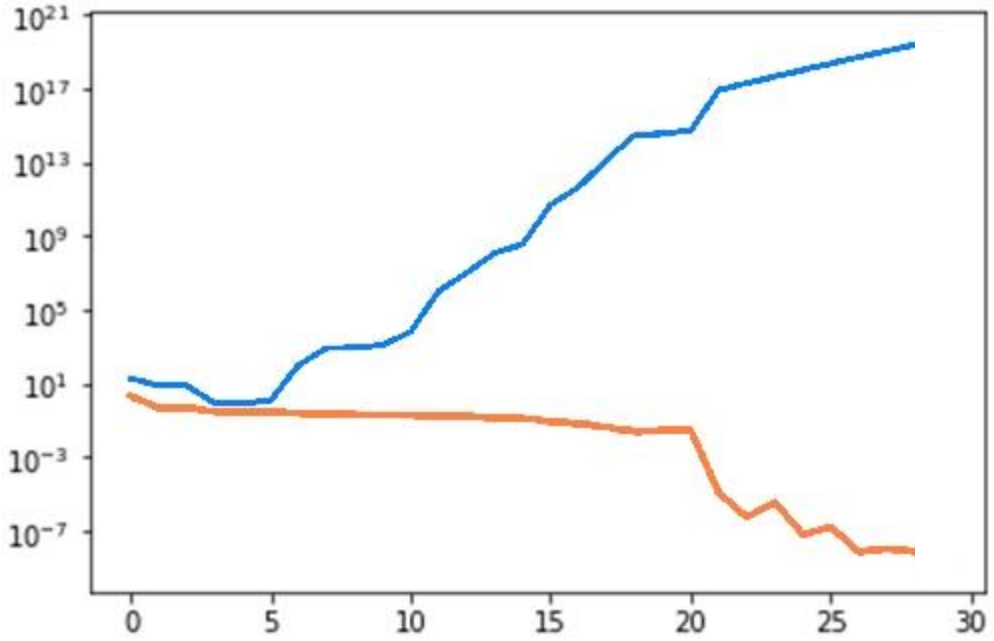


Train

Test

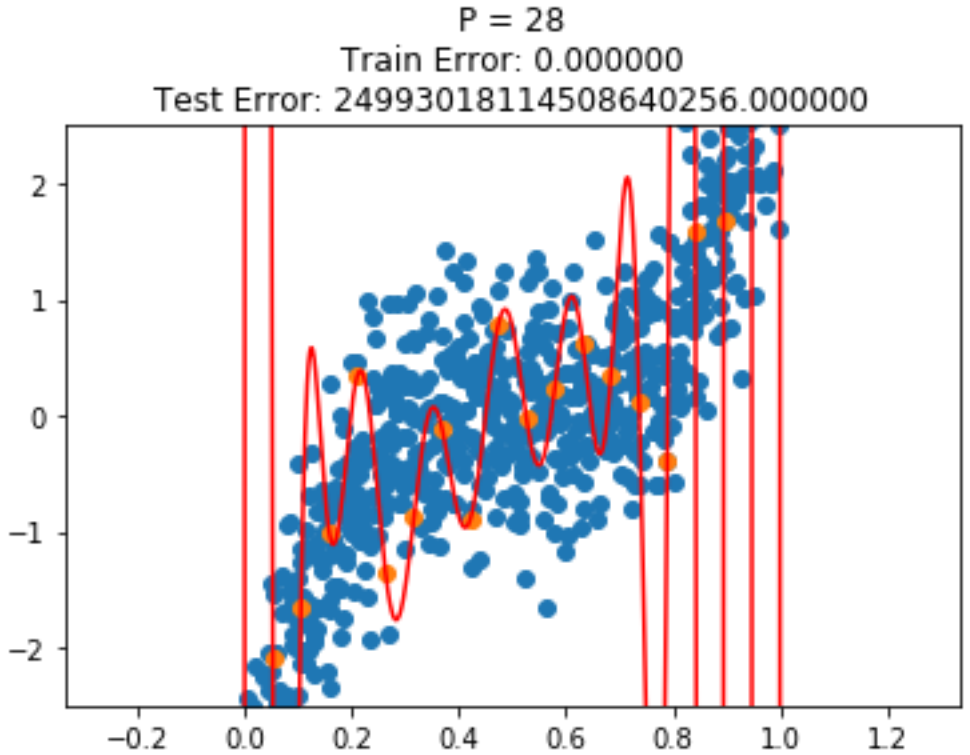


Polynomial fitting

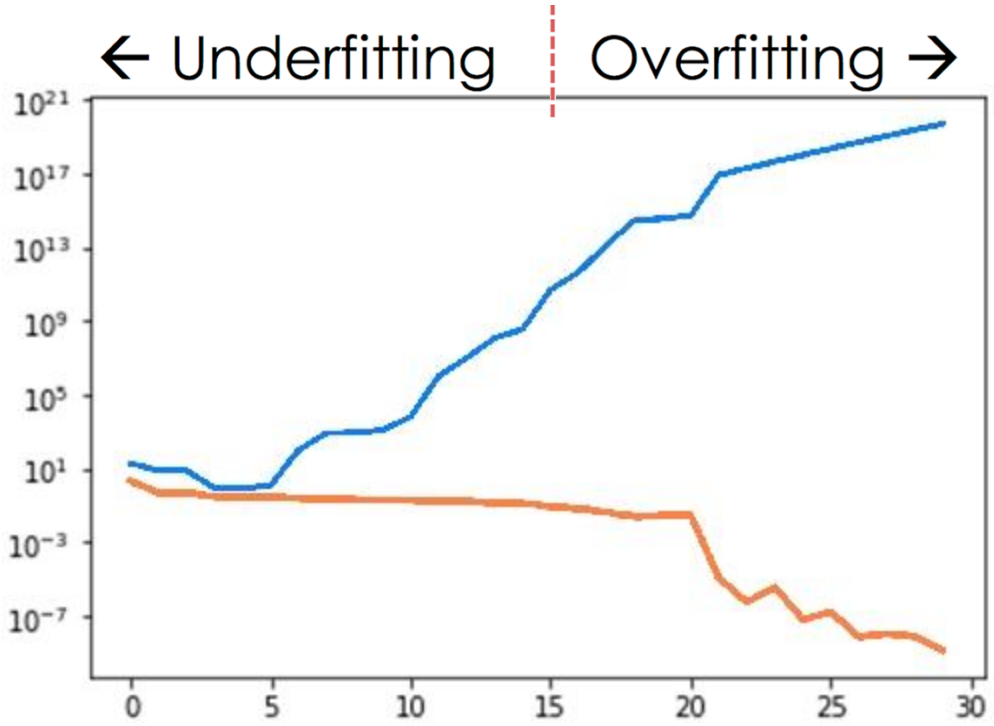


Train

Test

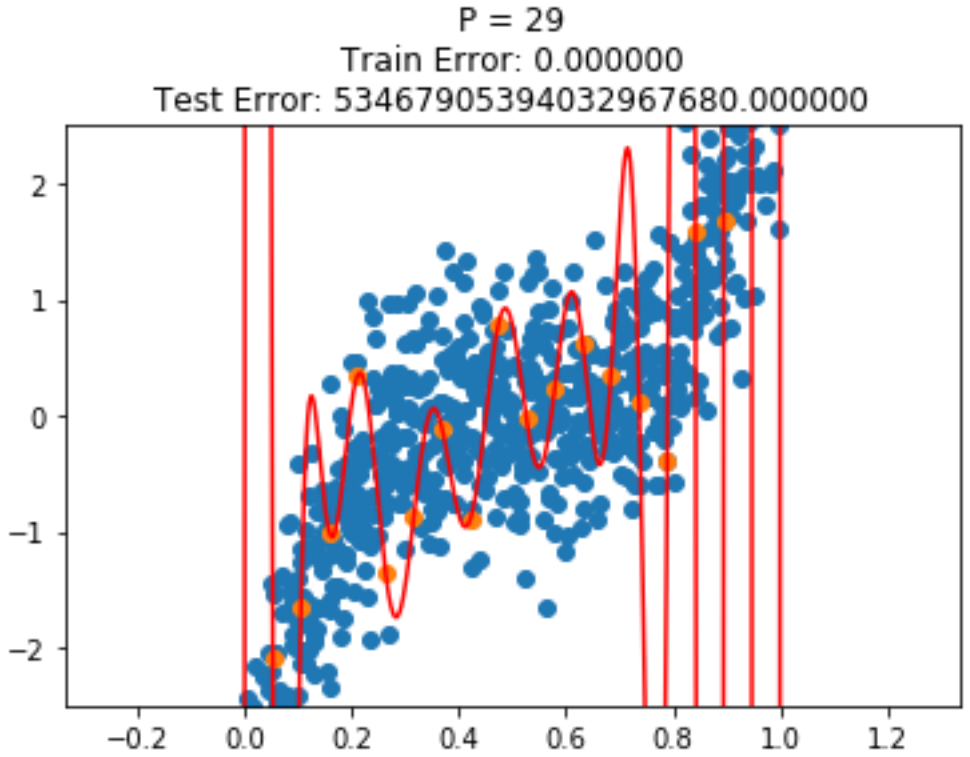


Polynomial fitting



Train

Test



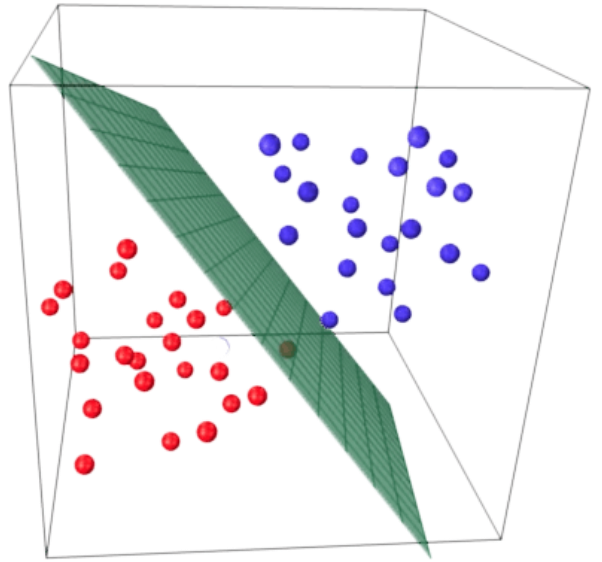
This week's tutorial:



Lior Yariv

Linear classification

+ some PyTorch



Next week's lecture:

(Me Again) # Neural Networks

